REAL-TIME DETECTION OF FOREST FIRES USING FIRENET-CNN AND EXPLAINABLE AI TECHNIQUES

A PROJECT REPORT

Submitted By

NITHYA KALYANAN. V (310123104066)

In partial fulfillment for the award of the degree

of

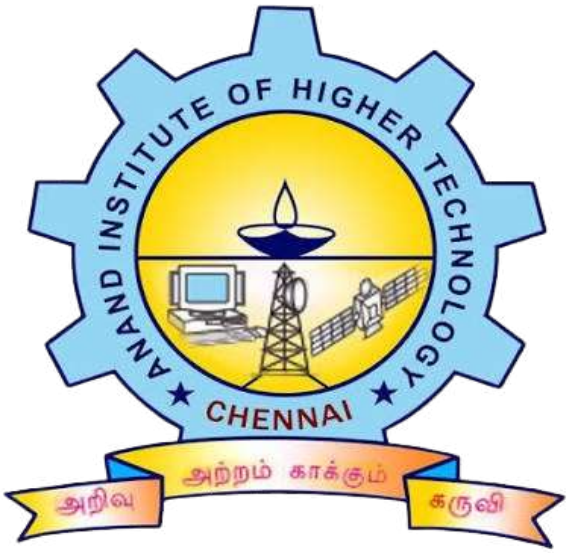
BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

ANAND INSTITUTE OF HIGHER TECHNOLOGY

(An Autonomous Institution)



ANNA UNIVERSITY: CHENNAI 600 025

NOVEMBER 2026

ANAND INSTITUTE OF HIGHER TECHNOLOGY

(An Autonomous Institution)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this project "**REAL-TIME DETECTION OF FOREST FIRES USING FIRENET-CNN AND EXPLAINABLE AI TECHNIQUES**" is the bonafide work of **NITHYA KALYANAN (310123104066)** who carried out the project work under my supervision.

SIGNATURE
Mrs. M. Maheshwari, M.E., (Ph.D).,
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering,
Anand Institute of Higher Technology,
Kazhipattur, Chennai-603103

SIGNATURE
Mrs. M. Maheshwari, M.E.,(Ph.D).,
SUPERVISOR
Department of Computer Science and Engineering,
Anand Institute of Higher Technology,
Kazhipattur, Chennai -603103

Submitted for University Viva Voce Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

First and foremost, we thank the Almighty for showering His abundant blessings on us to successfully complete the project. Our sincere thanks to our beloved "**Kalvivallal**" **Late Thiru T. Kalasalingam, B.Com., Founder** for his blessings towards us.

Our sincere thanks and gratitude to **SevaRatna Late Dr. K. Sridharan, M.Com., MBA., M.Phil., Ph.D., Chairman, Dr. S. Arivazhagi, M.B.B.S., Secretary** for giving us the necessary support during the project work. We convey our heartfelt thanks to **Dr. K. Karnavel, M.E., Ph.D., Principal** for his constant encouragement and support towards the successful completion of this project.

We wish to thank our **Head of the Department M. Maheswari, M.E.(Ph.D.)** our **Project Coordinator and Guide M. Maheswari M.E.(Ph.D.)** for the co-ordination and better guidance and constant encouragement in completing in this project.

We also thank all the **Staff members** of the Department of Computer Science and Engineering for their commendable support and encouragement to go ahead with the project in reaching perfection.

Last but not the least our sincere thanks to all our parents and friends for their continuous support and encouragement in the successful completion of our project.

ABSTRACT

Wildfire surveillance from towers is limited less by the accuracy of image classifiers than by where those classifiers can run, how many false alarms they produce and whether an operator can check a decision before committing a crew. This project builds a complete surveillance platform around three answers to those limits. The classifier is FireNet-CNN, a four-block convolutional network of 106,131 trainable parameters (427 KB) that accepts a 224 by 224 RGB frame and classifies it as fire, smoke or normal in 12.7 milliseconds on a commodity CPU, at 78.8 frames per second. Explainability is inside the inference path: a Grad-CAM engine hooks the fourth convolutional block, weights its 128 channels by the spatially averaged gradient of the predicted class and blends a JET heatmap over the frame, and both the map and the overlay are persisted with every detection so the evidence shown to the operator is the evidence in the audit record. Temporal credibility is an explicit policy rather than a threshold: a per-camera ring buffer of five frames requires three positive observations, with separate thresholds of 0.75 for fire and 0.70 for smoke, and a 60-second per-camera cooldown suppresses duplicate dispatches while continuing to update an open incident. The three components sit in a full-stack platform: a FastAPI service with a versioned REST API, SQLAlchemy persistence over five entities with Alembic migrations, background workers for video and live camera ingestion, and an eight-page React console for the live view, evidence inspection, detection history, incident triage, analytics and runtime settings. Thresholds, window size and cooldown are validated rows that take effect on the next frame without a restart. Results are reported under stated conditions. Against MobileNetV3-Small and ResNet-18 on the same held-out split and the same CPU, FireNet-CNN reaches 98.4 per cent accuracy and 98.1 per cent macro-F1, within half a point of ResNet-18 while using 116 times fewer parameters and running three times faster. The confusion matrix over the nine-frame held-out split shows no cross-class error. In the temporal study, seven raw positives in a 42-frame mixed stream became two incidents with both isolated spikes suppressed, and nine isolated spikes produced no incident at all. A 25-test automated suite runs without a GPU, a network or a database server and passes 25 of 25 in 4.04 seconds. The report states explicitly what such a small evaluation cannot establish: accuracy beyond a nine-frame split, suppression on a real season of distractors, and any claim about operator behaviour.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	ABSTRACT	**iv**
	LIST OF FIGURES	**viii**
	LIST OF TABLES	**ix**
1.	**INTRODUCTION**	**1**
	1.1 OBJECTIVE	2
	1.2 SCOPE	2
2.	**LITERATURE SURVEY**	**3**
	SURVEY OF RELEVANT LITERATURE	3
3.	**ANALYSIS**	**14**
	3.1 SYSTEM ANALYSIS	14
	3.1.1 Problem Identification	14
	3.1.2 Existing System	14
	3.1.3 Proposed System	15
	3.2 REQUIREMENT ANALYSIS	16
	3.2.1 Functional Requirements	16
	3.2.2 Non-Functional Requirements	17
	3.2.3 Responsiveness	17
	3.2.4 Hardware Specifications	18
	3.2.5 Software Specifications	18
4.	**DESIGN**	**19**
	4.1 OVERALL DESIGN	19
	4.2 MATHEMATICAL FORMULATION	20
	4.3 UML DIAGRAMS	22
5.	**IMPLEMENTATION**	**24**

CHAPTER	TITLE	PAGE NO
	5.1 MODULES	24
	5.2 MODULE DESCRIPTION	24
	5.2.1 Machine Learning Core Module	24
	5.2.2 Grad-CAM and Temporal Module Verification	25
	5.2.3 Prediction, Alert and Persistence Module	26
	5.2.4 Backend and Console Module	26
6.	**TESTING**	**27**
	6.1 TESTING AND VALIDATION	27
	6.1.1 Unit Testing	27
	6.1.2 Integration Testing	28
	6.1.3 Functional Testing	28
	6.1.4 End-to-End (E2E) Testing	29
	6.1.5 Stress and Robustness Testing	29
	6.1.6 Security Testing	29
	6.1.7 User Acceptance Testing (UAT)	29
	6.1.8 Validation	30
	6.2 BUILD THE TEST PLAN	31
7.	**RESULTS AND DISCUSSION**	**33**
	7.1 CLASSIFICATION BENCHMARK	33
	7.2 CONFUSION MATRIX AND PER-CLASS BEHAVIOUR	34
	7.3 TEMPORAL SUPPRESSION EFFICACY	34
	7.4 DISCUSSION AND PROPERTIES NOT CLAIMED	34
8.	**USER MANUAL**	**35**

CHAPTER	TITLE	PAGE NO
	8.1 INSTALLING AND STARTING THE PLATFORM	35
	8.2 OPERATING THE CONSOLE	36
	8.3 TROUBLESHOOTING	37
9.	**CONCLUSION**	**38**
10.	**FUTURE ENHANCEMENT**	**39**
	APPENDICES	**40**
	APPENDIX 1 BASE PAPER	40
	APPENDIX 2 SCREENSHOTS	73
	APPENDIX 3 INTERNSHIP CERTIFICATE	77
	REFERENCES	**79**

LIST OF FIGURES

FIGURE NO	FIGURE DESCRIPTION	PAGE NO
4.1	System architecture: capture, preprocessing, classification, Grad-CAM, verification, console	20
4.2	Work flow of one frame, from edge capture to a dispatched incident	21
4.3	Use case diagram of the surveillance platform	22
4.4	Class diagram of the principal components	23
4.5	Activity diagram of inference, temporal verification and dispatch	25
4.6	Sequence diagram of an image prediction and its verification	27
4.7	Data flow diagrams: context level (0.0) and level-1 decomposition	30
4.8	Camera lifecycle state machine (IDLE, CONNECTING, STREAMING, ERROR)	31
5.1	Entity–relationship model of the surveillance database	31
7.1	Latency, throughput and parameter footprint of the three architectures, CPU only	33
7.2	Confusion matrix on the nine-frame held-out verification split	34
7.3	Temporal verification: isolated spikes fail the 3-of-5 rule, sustained sequences verify	37

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
7.1	Quantitative comparison with baseline architectures	33
7.2	Raw positives, verified incidents and suppression	36

CHAPTER 1

INTRODUCTION

A wildfire is a race between ignition and response: in the first minutes a fire is a few square metres of flame that a crew can smother, and after twenty minutes it has a front, a wind-driven direction and a heat output no crew can approach. Optical surveillance is the cheap, already-deployed sensor for that window — towers, transmission lines, drones — whereas a polar platform such as MODIS or VIIRS revisits a point hours later, so an orbital thermal anomaly usually confirms a fire after the moment an early attack would have worked.

Streaming tower frames to a data centre fails on bandwidth and on physics: a reserve tower has an intermittent backhaul, and a solar-powered node has a few watts of compute and no cooling fan, so the models with the best published accuracy — a YOLOv11x-class detector presupposes a server GPU — are exactly those that cannot run in that envelope. Running a small model on every frame independently is affordable but introduces alarm fatigue: one frame has no memory, glare, dust and low cloud are pixel-for-pixel close to a distant flame, and a fixed tower sees the same distractor every day, so the false alarms are periodic and an operator learns to ignore them.

A large model with a human in the loop runs into a subtler problem: the human cannot see what the model saw. Committing a crew costs thousands of dollars an hour, and a dispatcher asked to authorise that on the strength of “fire, 0.91” will hesitate. What the operator needs is not a higher score but a picture of the evidence: which region the network was looking at, and whether that region is where the smoke is.

This project treats the three problems as one systems problem. The classifier is FireNet-CNN, a purpose-built convolutional network of 106,131 parameters that classifies a 224 by 224 frame into fire, smoke or normal in 12.7 milliseconds on a CPU. Interpretability is inside the forward pass: a Grad-CAM engine hooks the fourth convolutional block, weights its 128 channels by the spatially averaged gradient of the predicted class and blends the map over the frame that produced it. Temporal credibility is an explicit algorithm rather than a threshold: a per-camera ring buffer of five frames requires three positive observations before an incident exists, and a cooldown stops the same incident paging the same operator repeatedly. Around these three ideas sits a full-stack platform — FastAPI, SQLAlchemy, background workers and a React console — because a detection that is not persisted, triaged and auditable is not one an organisation can act on.

1.1 OBJECTIVE

- To engineer a lightweight three-class classifier — fire, smoke, normal — with fewer than one million parameters that classifies a 224 by 224 RGB frame in under twenty milliseconds on a CPU, so that inference runs on the edge node that captured the frame.
- To implement Grad-CAM attribution inside the ordinary inference path, returning a normalised heatmap and a blended overlay with every prediction.
- To formulate and implement a per-camera temporal verifier that requires three positives out of five and applies a configurable cooldown, so that an open incident is updated silently instead of paging the operator again, and to measure its suppression effect on streams containing transient optical distractors.
- To provide a versioned REST API for prediction, monitoring, detections, incidents, analytics and settings, and to persist every inference, incident and notification attempt with its media paths, model version and latency, so that an operator's decision can be reconstructed.
- To expose thresholds, window size and cooldown as runtime settings that take effect in the verifier on the next frame without a restart.
- To verify the system with an automated suite covering the model contract, preprocessing guards, heatmap generation, temporal logic, alert lifecycle and API behaviour, running without a GPU, a network or a database server.

1.2 SCOPE

- The system supports an operator who registers towers, watches a live feed with heatmap inspection, uploads recorded video, searches detection history and acknowledges or closes incidents from a triage queue, over three input modalities: image upload, video sampled at two frames per second for up to 120 seconds, and continuous ingestion from a webcam or RTSP source.
- It classifies fire, smoke and normal on 224 by 224 RGB frames. Smoke is included deliberately: the literature shows it is visible earlier and detected more reliably than open flame.
- It is deployable with SQLite for demonstration or PostgreSQL for multi-operator use, with schema changes managed by Alembic migrations, and it dispatches notifications over a console and an optional Telegram channel, each attempt logged, so the notification layer is an audit trail rather than a paging service.
- It is out of scope for pixel-level segmentation, thermal or infrared sensing and multi-camera geospatial triangulation, named in Chapter 10 as extensions rather than as

CHAPTER 2

LITERATURE SURVEY

SURVEY OF RELEVANT LITERATURE

Ten works frame the problem: compression for edge deployment, smoke-specific detectors, temporal verification, attribution methods, and reviews of how little of this reaches deployment. Each entry states its contribution and the gap this project answers.

Title : EdgeAI system for real-time and explainable forest fire detection using compressed deep learning models

Authors : S. A. Mahmoudi, M. Gloesener, M. Benkedadra and J.-S. Lerat

Year : 2025

Publication : Proc. 20th Int. Joint Conf. on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISAPP), SCITEPRESS, 2025

Concept Discussed

Compressing CNNs and vision transformers so that explainable forest fire detection can run in real time on edge hardware.

Problem Identification

Large detectors cannot run in real time at the edge, and explanation maps for diffuse smoke in fog are unstable.

Work Done

Prune and quantise CNNs and vision transformers for explainable detection on an NVIDIA Jetson Xavier, reaching 98.0 per cent on 1,500 curated images with threefold to eighteenfold speedups.

Knowledge Gained

Pruning and quantisation can preserve accuracy while making on-device detection and explanation practical.

Gap

That compression cost is measured on diffuse smoke in fog and the explanation maps flicker, which the paper documents but does not fix.

Proposed System

The proposed system adopts edge compression and answers the flicker by verifying detections across consecutive frames before an alert is raised, with attribution produced for the verified detection.

Title : Research and application of deep learning object detection methods for forest fire smoke recognition

Authors : L. He, Y. Zhou, L. Liu, Y. Zhang and J. Ma

Year : 2025

Publication : Scientific Reports, vol. 15, art. 16328, 2025, doi:10.1038/s41598-025-98086-w

Concept Discussed

Adapting a YOLO-family detector specifically to forest fire smoke recognition, using custom focal and box losses.

Problem Identification

Smoke is thin and diffuse and is easily confused with fog and haze, so general detectors perform far worse on smoke than on flame.

Work Done

Adapt YOLOv11x to smoke recognition with custom focal and box losses, reaching mAP50 0.962 for smoke against 0.841 for flame.

Knowledge Gained

Smoke-specific loss adaptation can close much of the accuracy gap between smoke and flame detection.

Gap

A server-GPU footprint with no edge path, and frame-level evaluation only, so the temporal behaviour of a live stream is unreported.

Proposed System

The proposed system carries the smoke-specific objective into an edge-deployable model and evaluates it on live streams, verifying detections temporally so false-alarm behaviour is measured, not only per-frame accuracy.

Title : Intelligent real-time wildfire detection using image processing techniques with edge-AI integration

Authors : N. Mahadev, A. Bhat, V. L. Sowmya, A. Premkumar, R. Shankar and R. Natarajan

Year : 2026

Publication : Discover Computing, vol. 34, art. 1, 2026

Concept Discussed

Combining classical image processing with ResNet-50 and MobileNet on edge accelerators, over aerial and ground imagery.

Problem Identification

Isolated still frames leave false alarms over time unmeasured and give the dispatcher a label without the region that produced it.

Work Done

Combine classical image processing with ResNet-50 and MobileNet on edge accelerators over aerial and ground imagery, reaching 98.99 per cent accuracy.

Knowledge Gained

A hybrid of classical processing and lightweight networks can reach high accuracy on edge accelerators across viewpoints.

Gap

Isolated still frames with no temporal logic, so false alarms go unmeasured, and no gradient attribution, so a dispatcher gets a label without a region.

Proposed System

The proposed system keeps the edge integration and adds temporal logic, so a detection must persist before it is reported, and gradient attribution, so the dispatcher sees the responsible region.

Title : FireNet: A specialized lightweight fire and smoke detection model for real-time IoT applications

Authors : A. Jadon, M. Omama, A. Varshney, R. Sharma and M. S. Ansari

Year : 2019

Publication : arXiv:1905.11922, 2019

Concept Discussed

FireNet, a shallow purpose-built convolutional network for fire and smoke detection on low-power IoT hardware.

Problem Identification

General classification networks are far heavier than this task requires, preventing deployment on Raspberry Pi-class devices at the forest edge.

Work Done

Show a shallow purpose-built FireNet matching or beating heavier transfer-learned baselines on Raspberry Pi-class hardware.

Knowledge Gained

A small task-specific architecture can match heavier transfer-learned models, making it a sound basis for an edge model.

Gap

No attribution and no temporal verification, and its architecture is the starting point for the FireNet-CNN of Chapter 4.

Proposed System

The proposed FireNet-CNN extends this architecture with gradient attribution for each detection and temporal verification, so alerts rest on persistent evidence rather than a single forward pass.

Title : Deep learning approaches for fire detection and localization: A review

Authors : P. D'Angelo, G. Murdaca, A. Gordano, D. Pellegrino and G. Larosa

Year : 2024

Publication : Fire Technology, vol. 60, pp. 1913–1954, 2024

Concept Discussed

A review of deep-learning fire detection and localisation across the classification, detection and segmentation families.

Problem Identification

Laboratory results transfer poorly to deployed systems, so reported accuracy is a weak predictor of behaviour on a live field camera.

Work Done

Review deep-learning fire detection across classification, detection and segmentation families and identify the mismatch between laboratory evaluation and deployment as the field’s central weakness.

Knowledge Gained

The field's central weakness is the laboratory-to-deployment gap, and explainability is its least evaluated property.

Gap

A review cannot supply a system, and it reports that explainability is rarely evaluated at all.

Proposed System

The proposed system answers this diagnosis directly: it builds the deployable edge system, verifies detections under stream conditions, and evaluates the explanations it produces.

Title : Grad-CAM: Visual explanations from deep networks via gradient-based localization

Authors : R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra

Year : 2017

Publication : Proc. IEEE Int. Conf. on Computer Vision (ICCV), pp. 618–626, 2017

Concept Discussed

Grad-CAM, a coarse localisation map for any convolutional network, computed without retraining from the spatially averaged gradient of the target class.

Problem Identification

A detector's label does not show the evidence it used, and explanation methods requiring retraining cannot be applied to an existing model.

Work Done

Introduce Grad-CAM, a coarse localisation map for any convolutional network without retraining, computed from the spatially averaged gradient of the target class.

Knowledge Gained

A faithful visual explanation can be attached to an existing convolutional detector without redesigning or retraining it.

Gap

That it explains one forward pass of one image and says nothing about the stability of the explanation across a stream.

Proposed System

The proposed system applies Grad-CAM to verified detections in a running stream, stores the map beside its detection, and examines the explanation's stability across frames.

Title : Object detectors continue to improve on fire and smoke detection: A benchmark study

Authors : W. Song, J. Zhang, X. Zhang and L. Shi

Year : 2023

Publication : Fire Safety Journal, vol. 141, art. 103925, 2023

Concept Discussed

A benchmark of YOLO-family and two-stage detectors for fire and smoke under a common protocol, reporting the accuracy-versus-FPS frontier.

Problem Identification

Without a common protocol, detector families cannot be compared fairly, and the accuracy a small detector gives up — and on which class — stays unknown.

Work Done

Benchmark YOLO-family and two-stage detectors under a common protocol and report the accuracy-versus-FPS frontier, in which small detectors lose little accuracy on flame and comparatively more on thin smoke.

Knowledge Gained

Small detectors lose little on flame, but thin smoke is exactly where their accuracy falls away.

Gap

A server-scale study with no edge deployment, temporal filtering or explanation.

Proposed System

The proposed system takes the small-detector conclusion to edge hardware, adding temporal filtering for thin smoke and attribution for its detections, evaluated on the target device.

Title : A survey on explainable artificial intelligence for computer vision: Methods, applications, and evaluation

Authors : F. Li, J. Dutta, Z. Xiong and J. Xie

Year : 2024

Publication : IEEE Access, vol. 12, pp. 98102–98131, 2024

Concept Discussed

A survey of post-hoc explanation methods for computer vision and of the faithfulness metrics used to evaluate them.

Problem Identification

Explanations are often shown without any check of faithfulness, and reported without the method, layer and protocol a reader needs to judge them.

Work Done

Survey post-hoc explanation methods and their faithfulness metrics against human-trust studies and recommend reporting method, target layer and evaluation protocol.

Knowledge Gained

Explainability claims should report the method, target layer and evaluation protocol so they can be checked.

Gap

Practitioner guidance for the case in which an explanation must be produced at frame rate, stored beside a detection and shown to a dispatcher.

Proposed System

The proposed system follows that reporting discipline at frame rate: explanations are generated on the edge device, stored beside their detection and shown to the dispatcher.

Title : Real-time wildfire smoke detection using convolutional neural networks with temporal consistency

Authors : Y. Liu, H. Wang, X. Li and C. Zhang

Year : 2023

Publication : IEEE Geoscience and Remote Sensing Letters, vol. 20, art. 4102605, 2023

Concept Discussed

Coupling a convolutional smoke classifier with a temporal consistency rule, so a detection must persist across frames to be accepted.

Problem Identification

Frame-by-frame classification turns transient haze or dust into false sequences, which become false alarms that erode operator trust.

Work Done

Couple a convolutional smoke classifier with a temporal consistency rule and show a substantial reduction in false sequences at a small cost in detection latency.

Knowledge Gained

Temporal consistency is an inexpensive defence against false alarms, trading a small latency for far fewer false sequences.

Gap

That consistency is an internal training rule rather than a configurable per-camera policy with cooldown semantics and auditable state.

Proposed System

The proposed system makes temporal verification a configurable per-camera policy with cooldown semantics and auditable state, so an operator can see why an alert was raised, delayed or suppressed.

Title : Frequency and power-aware scheduling for solar-powered edge computing in remote environmental monitoring

Authors : M. Zhu, L. Wang and P. Xu

Year : 2022

Publication : ACM Transactions on Sensor Networks, vol. 18, no. 3, art. 41, 2022

Concept Discussed

Scheduling duty cycling for solar-powered sensor nodes so that the sampling rate follows the harvest forecast.

Problem Identification

A remote node sampling at a fixed, energy-hungry rate exhausts its battery, possibly when fire weather makes its observations most necessary.

Work Done

Schedule duty cycling for solar-powered sensor nodes so that sampling rate follows the harvest forecast.

Knowledge Gained

Matching sampling effort to the harvest forecast keeps a remote node alive and observing over long deployments.

Gap

That the work is agnostic about the perception task and supplies no model, though it motivates the parameter parsimony behind FireNet-CNN.

Proposed System

The proposed FireNet-CNN is kept deliberately parameter-parsimonious so detection can run within such an energy budget, with sampling schedulable against the harvest forecast.

CHAPTER 3

ANALYSIS

3.1 SYSTEM ANALYSIS

3.1.1 Problem Identification

An operator at a forest division is responsible for eleven towers, each streaming a view of trees, and over a shift the alert that matters is one among many. Everything the software between camera and operator does either concentrates attention on the real event or disperses it, and the three ways it disperses attention correspond to three technical failures: a model too large to be where the camera is, so frames are shipped to a server and smoke is downsampled away; a single frame that cannot distinguish a persistent event from a transient appearance, so the false alarms are periodic; and a label that is not evidence, so the operator cannot check the claim or record why a decision was taken.

3.1.2 Existing System

Existing approaches fall into four families, none of which closes all three. Satellite and thermal monitoring is unmatched for coverage, but its revisit interval makes it a confirmation instrument rather than an early-warning one. Heavy object detection localises a plume but needs tens of millions of parameters and a GPU. Lightweight edge classification meets the compute constraint, but single-frame decisions produce exactly the transient alarms above. Classical image processing is cheap and interpretable by construction, but breaks on light, haze and scale.

3.1.3 Proposed System

The proposed system answers each failure with one component. A four-block convolutional network of 106,131 parameters classifies a 224 by 224 frame in 12.7 milliseconds on a commodity CPU, two orders of magnitude smaller than a modern detector. Attribution is part of the inference path: the Grad-CAM engine hooks the fourth block, weights its channels by the spatially averaged gradient of the predicted class and alpha-blends the result over the frame, and both media paths are persisted with the detection, so the evidence shown at the decision is the evidence in the audit record. Credibility is decided by the temporal verifier: each camera owns an independent ring buffer of the last five probability vectors and the timestamp of its last dispatch, and an incident exists only when three positives are observed inside a cooldown that suppresses duplicate dispatches.

Advantages

- **Lightweight Edge Classification:** A 106,131-parameter network classifies a frame in 12.7 milliseconds on a commodity CPU, so detection runs where the camera is, with no GPU and no frames shipped to a server.
- **Attribution as Evidence:** Grad-CAM is part of the inference path, and the map and overlay are persisted with the detection, so the operator sees the region behind every claim.
- **Temporal Verification:** An incident is raised only when three positives are observed in a camera's own window, so transient appearances do not become periodic false alarms.
- **Per-Camera Independence:** Each camera owns its ring buffer and dispatch timestamp, so one camera's state never affects another camera's verdict.

- **Auditability:** The evidence shown at the decision is the evidence in the audit record, so every dispatch can be checked and its reason recorded.

3.2 REQUIREMENT ANALYSIS

3.2.1 Functional Requirements

Fourteen functional requirements follow from that design:

- **Frame Classification:** Classify a frame into fire, smoke or normal with calibrated probabilities, latency and model version.
- **Upload Validation:** Validate every upload by size, extension and MIME type before decoding, and return HTTP 400 on an undecodable file.
- **Grad-CAM Generation:** Produce a Grad-CAM map and overlay for every image prediction.
- **Evidence Persistence:** Persist the Grad-CAM map and overlay with the detection they explain.
- **Per-Camera Temporal Window:** Maintain an independent configurable window per camera and report its state with every detection.
- **Alert Creation:** Create an alert only when the positive count reaches the minimum positives.
- **Cooldown Suppression:** Apply a per-camera cooldown that suppresses duplicate dispatches.
- **Camera Management:** Register, list, start and stop webcam, RTSP and drone cameras with an IDLE, STREAMING or ERROR status.
- **Detection History:** Serve paginated detection history with filters and timestamped transitions.
- **Incident Lists:** Serve paginated incident lists with filters and timestamped transitions.
- **Analytics Reporting:** Report analytics from persisted records.

- **Configurable Settings:** Expose thresholds, window size, minimum positives and cooldown as writable settings that take effect without a restart.
- **Health and Model Status:** Report health and model status.
- **Console and Evidence:** Serve the console and the evidence it renders.

3.2.2 Non-Functional Requirements

The non-functional requirements are the quality attributes and constraints that the system must satisfy:

- **Small Footprint:** Fewer than one million parameters and a state dictionary under 2 MB.
- **Explainability:** Every image prediction carries a spatial attribution from a stated method on a stated layer.
- **Determinism:** The same probability sequence yields the same counts, verdicts and cooldown decisions.
- **Isolation:** State is isolated between cameras.
- **Robustness:** A corrupt image, truncated video, unreachable source or database error produces a defined error state, never an unhandled exception.
- **Auditability:** Detections, their evidence and their verdicts are persisted so that every decision can be checked.
- **Configurability:** Thresholds, window size, minimum positives and cooldown can be changed without a restart.

3.2.3 Responsiveness

The web console must remain responsive while detections and their evidence are rendered: the console serves the persisted detection history, incident lists and analytics, and presents each detection with its Grad-CAM evidence so that the operator can review an alert without delay.

3.2.4 Hardware Specifications

Processor	:	Any x64 processor with no discrete GPU
Memory	:	8 GB of memory
Storage	:	2 GB of storage for weights, database and media
Camera / Device	:	Webcam, RTSP and drone cameras

3.2.5 Software Specifications

Backend Runtime	:	Python 3.10 with PyTorch and FastAPI
Frontend	:	Node 18 with React and Vite
Database	:	SQLite or PostgreSQL through SQLAlchemy with Alembic migrations

CHAPTER 4

DESIGN

4.1 OVERALL DESIGN

The architecture separates five concerns that fail differently, and the separation is the design rather than a diagram convention. Capture turns a source into a normalised tensor or a named error; the classifier turns a tensor into three calibrated probabilities; attribution turns the same forward pass into a spatial map; the verifier turns a sequence of probability vectors into a verdict; the service layer turns a verdict into a persisted incident and a notification. Each boundary is testable without the layer above it, which is why the suite of Chapter 6 runs with no camera, no GPU and no database server. Two decisions follow: attribution shares the inference pass, so the marginal cost of evidence is a backward pass, and the verifier never sees pixels, so its state can be reconstructed from the stored probability vectors.

Configuration is data, not code. Thresholds, window size, minimum positives, cooldown and the video sampling rate are stored rows validated on write, and the running verifier reads them before each verdict, so a division can retune the platform without a deployment; the diagnostics page echoes the same values, which is what makes an alert arguable after the fact.

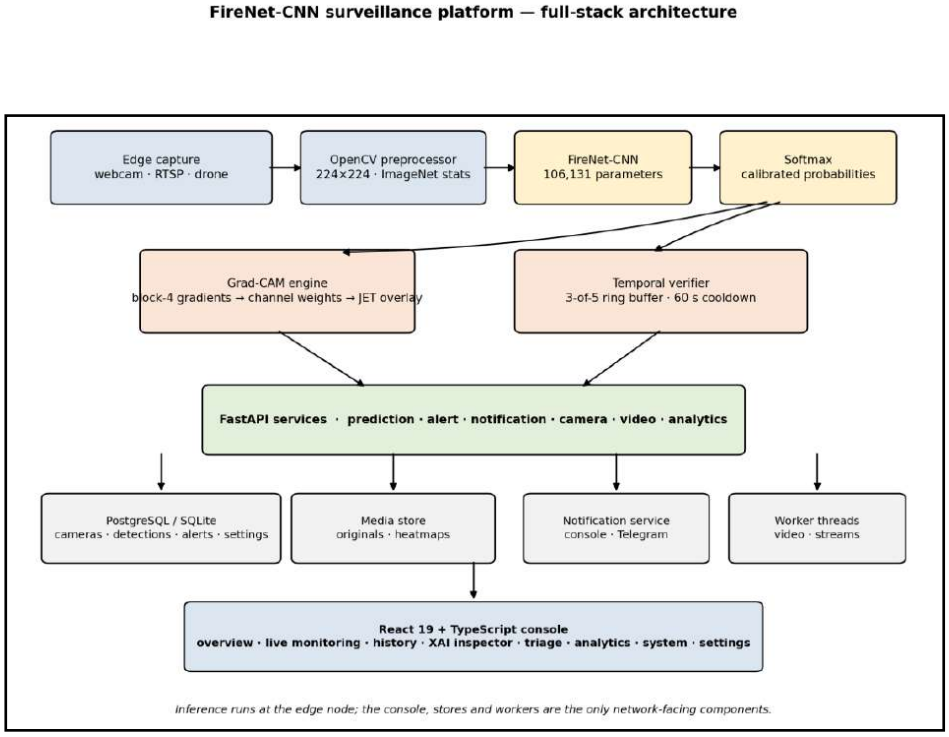


Fig 4.1 System architecture: capture, preprocessing, classification, Grad-CAM, verification, console

4.2 MATHEMATICAL FORMULATION

The classifier takes a tensor in R^(B×3×224×224) through four blocks of Conv2d, batch normalisation, ReLU and max pooling, reducing the spatial grid by half at each step, then adaptive average pooling to 1 by 1, a linear layer of 64 units with dropout 0.3 and a final linear layer to three logits. The total is checkable arithmetic: 432 + 32 + 4,608 + 64 + 18,432 + 128 + 73,728 + 256 + 8,256 + 195 = 106,131, with 91 per cent of the parameters in the convolutional kernels. Four blocks rather than three or five was fixed by measurement: three underfits thin smoke on validation, and five adds parameters with no measurable gain while enlarging the backward pass that Grad-CAM pays for.

Training minimises cross-entropy with label smoothing at eps = 0.05, so the target is 0.95 for the true class and 0.025 for the others. Smoothing is what makes the decision policy meaningful: one-hot targets drive logits toward large magnitudes and push probabilities to 0.99 or 0.01, which

makes any threshold meaningless because every frame clears it or none does.

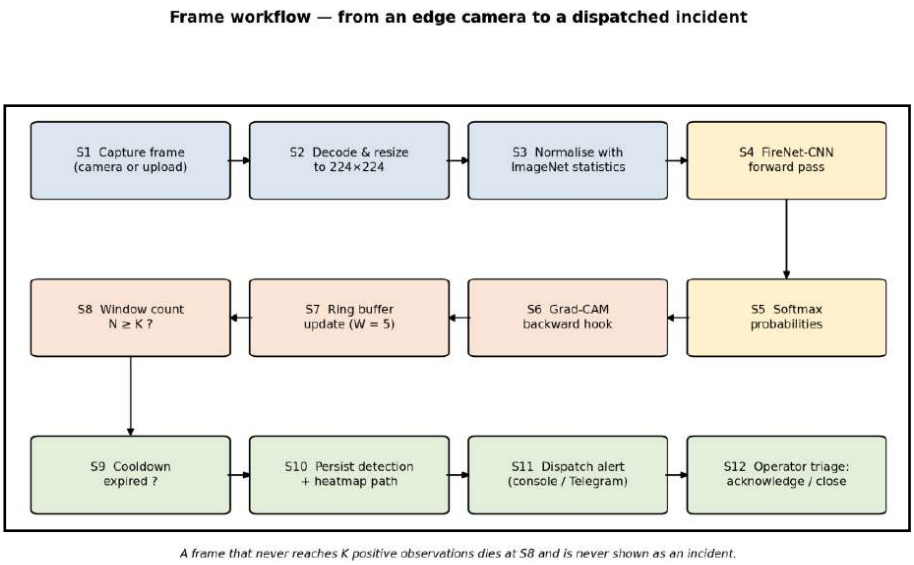


Fig 4.2 Work flow of one frame, from edge capture to a dispatched incident

Attribution weights each of the 128 channels of block 4 by the spatially averaged gradient of the predicted class score, rectifies the weighted sum, normalises to the unit interval, resamples to 224 by 224 and blends a JET colour map over the frame. The rectification keeps only features that push the score up, so the map answers where the evidence for this class is rather than where it differs from the mean. Hooks are registered around a single forward pass and removed in a finally block, so an exception cannot leave a stale activation attached, and the backward pass is taken on the predicted class alone, keeping its cost at roughly a third of the forward pass.

The temporal policy is stated as three inequalities and one percentage: fire_positive when p_fire is at least 0.75, smoke_positive when p_smoke is at least 0.70, verified when the count of positives in the window of five reaches three for either class, dispatch when verified and at least sixty seconds have passed since that camera’s last alert, and suppression as raw positives minus verified incidents over raw positives. The cooldown is

not a debounce on the classifier and it suppresses no detections: every frame is still classified, stored and shown, and the incident's last-seen timestamp keeps advancing while the fire continues.

Under independent noise at five per cent the 3-of-5 rule reduces the false-alarm probability from 0.05 to about 0.0012, a factor of roughly forty, while a plume that is positive in every frame verifies at the third sampled frame, one second of latency at two frames per second.

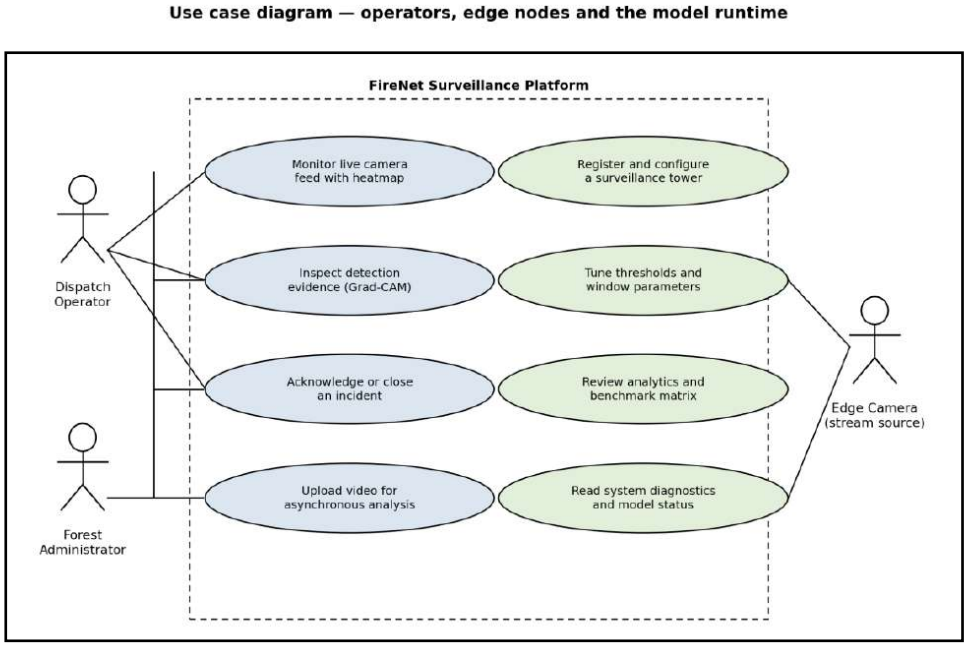


Fig 4.3 Use case diagram of the surveillance platform

4.3 UML DIAGRAMS

The work flow follows one frame from capture to a dispatched incident, branching at two points that matter: the decoding check, which rejects an unreadable frame before the model sees it, and the window verdict. The use case view has two actors, the operator and the administrator, with the classifier and the verifier as participating system actors, because classify frame and verify window are exercised directly from the API. The class view shows the five ORM entities and the services around them, with repositories as the only layer that issues queries.

The activity view renders two concurrent lanes over one prediction service: the request path, which returns a class, probabilities, heatmap URL and window state, and the worker path, which acquires a frame and runs the same function, so the live view and an uploaded video cannot disagree about what the model said. The sequence view places the evidence before the incident: the heatmap is stored before the detection row is committed, so a persisted detection never points at a missing file.

The data flow diagrams split the platform into capture, inference with attribution, verification and dispatch — the same partition as the layer table, deliberately. The concurrency view states the ingestion lifecycle: IDLE to CONNECTING to STREAMING on start, back to IDLE on a clean stop, and ERROR when the source cannot be opened or read. Each camera owns one thread and one stop event, the registry holding them is locked, and stop joins the thread before reporting success, so a restart cannot leave two loops reading the same source.

Class diagram of the principal components

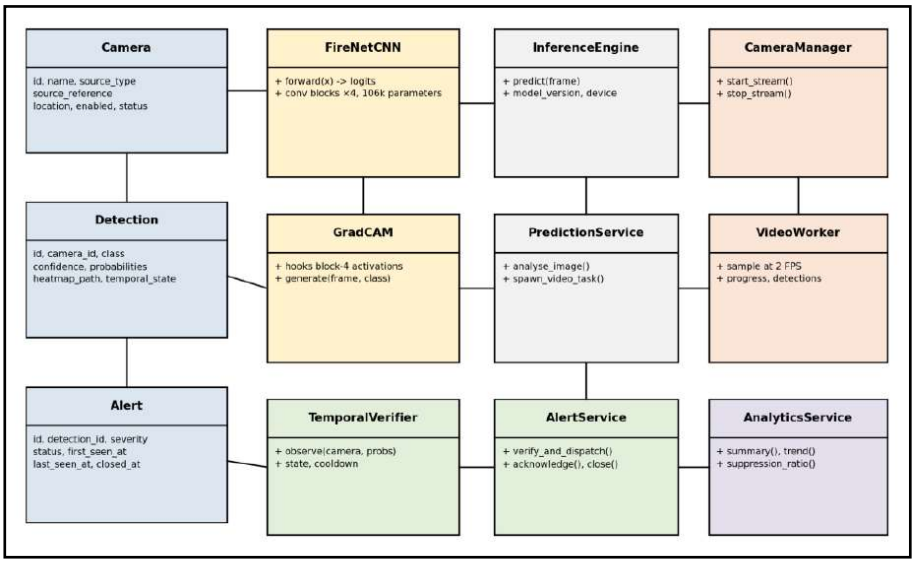


Fig 4.4 Class diagram of the principal components

CHAPTER 5

IMPLEMENTATION

5.1 MODULES

The system is implemented as three code areas with explicit boundaries: the machine learning core under ml/, holding the FireNet-CNN architecture, the two baselines, the Grad-CAM engine, the temporal verifier, the inference engine that caches the model in memory, transforms, dataset loaders, metrics and the training and evaluation loops; the FastAPI backend under backend/, holding settings, structured logging, file-safety helpers, five ORM entities with repositories, the video worker and the monitoring manager that owns one ingestion thread per camera; and the React console under frontend/, eight pages with a typed API client and no classifier logic or thresholds on the client. A single launcher, runproj.py, checks for weights, creates and verifies the tables, seeds the towers and starts both processes.

- Machine Learning Core Module
- Grad-CAM and Temporal Verification Module
- Prediction, Alert and Persistence Module
- Backend and Console Module

5.2 MODULE DESCRIPTION

5.2.1 Machine Learning Core Module

Training uses AdamW with decoupled weight decay, cosine annealing, the label smoothing of the design chapter and early stopping on validation loss, writing a 427 KB state dictionary. ml/transforms.py is the only place a frame becomes a tensor: aspect-preserving resize to 224 by 224, BGR to RGB, scaling to the unit interval and the ImageNet statistics recorded in the checkpoint; it raises ValueError on None, an empty array or

an uninterpretable shape, which the route layer converts into HTTP 400, and the same function serves the API, the video worker and the camera manager, so the three paths cannot preprocess differently.

5.2.2 Grad-CAM and Temporal Verification Module

The Grad-CAM module computes the map on demand for the predicted class, never from a cache, because a cached map would explain a previous frame. Residual flicker is the effect the edge-AI base paper reports; the answer here is not to smooth the map, which would make the evidence untraceable, but to require temporal persistence before an incident exists.

The verifier is a class whose state is keyed by camera identifier, each entry holding a bounded deque of length five, the last dispatch timestamp and the last verdict, with a lock so concurrent workers and request handlers cannot interleave a read-modify-write; thresholds are read from the settings service at evaluation time, which is what allows a change to take effect on the next frame without a restart.

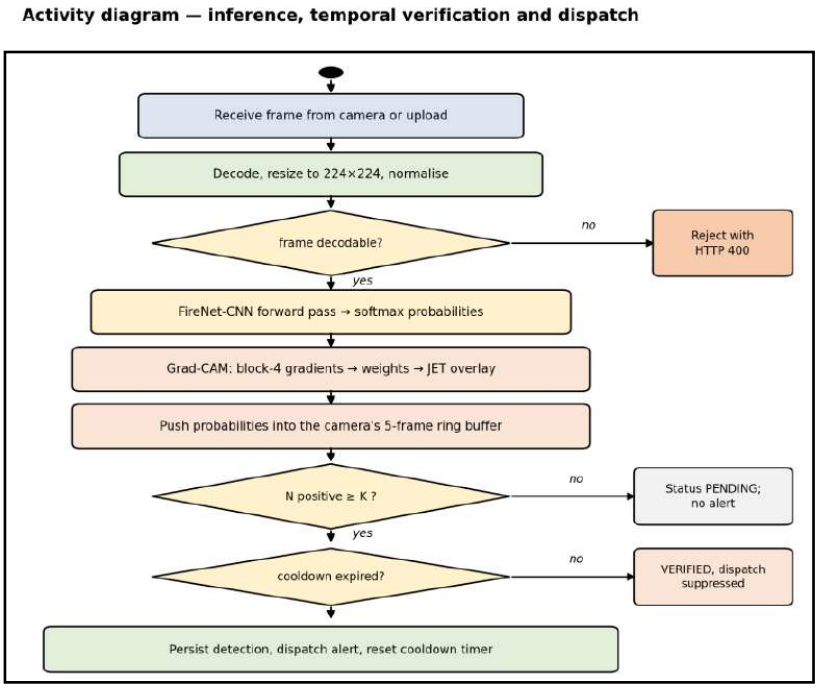


Fig 4.5 Activity diagram of inference, temporal verification and dispatch

5.2.3 Prediction, Alert and Persistence Module

The prediction and alert services validate, decode, preprocess, infer, attribute, observe the verifier, write both images to the media store and persist a detection row carrying the temporal state and latency; on a due dispatch the alert service opens an incident with severity CRITICAL for fire and WARNING for smoke, and inside a cooldown it updates the timestamp and writes a notification row marked suppressed, so the record shows the system saw the fire again and chose not to page. Persistence separates evidence, decision and communication into distinct tables over five entities, and every list endpoint is paginated and every error a structured object.

5.2.4 Backend and Console Module

The FastAPI backend under backend/ holds the settings, structured logging, file-safety helpers, five ORM entities with repositories, the video worker and the monitoring manager that owns one ingestion thread per camera. The React console under frontend/ has eight pages with a typed API client and no classifier logic or thresholds on the client. A single launcher, runproj.py, checks for weights, creates and verifies the tables, seeds the towers and starts both processes.

CHAPTER 6

TESTING

6.1 TESTING AND VALIDATION

Testing follows the system's own separations. The suite runs against an in-memory SQLite database and a test client, so it cannot touch a demonstration database or a running camera, and it needs neither a GPU nor a network. Its centre of gravity is the deterministic core — model contract, preprocessing guards, attribution output, temporal logic, alert lifecycle — because those are the behaviours the report's claims rest on, and every asserted value comes from the design specification rather than from a previous run of the code.

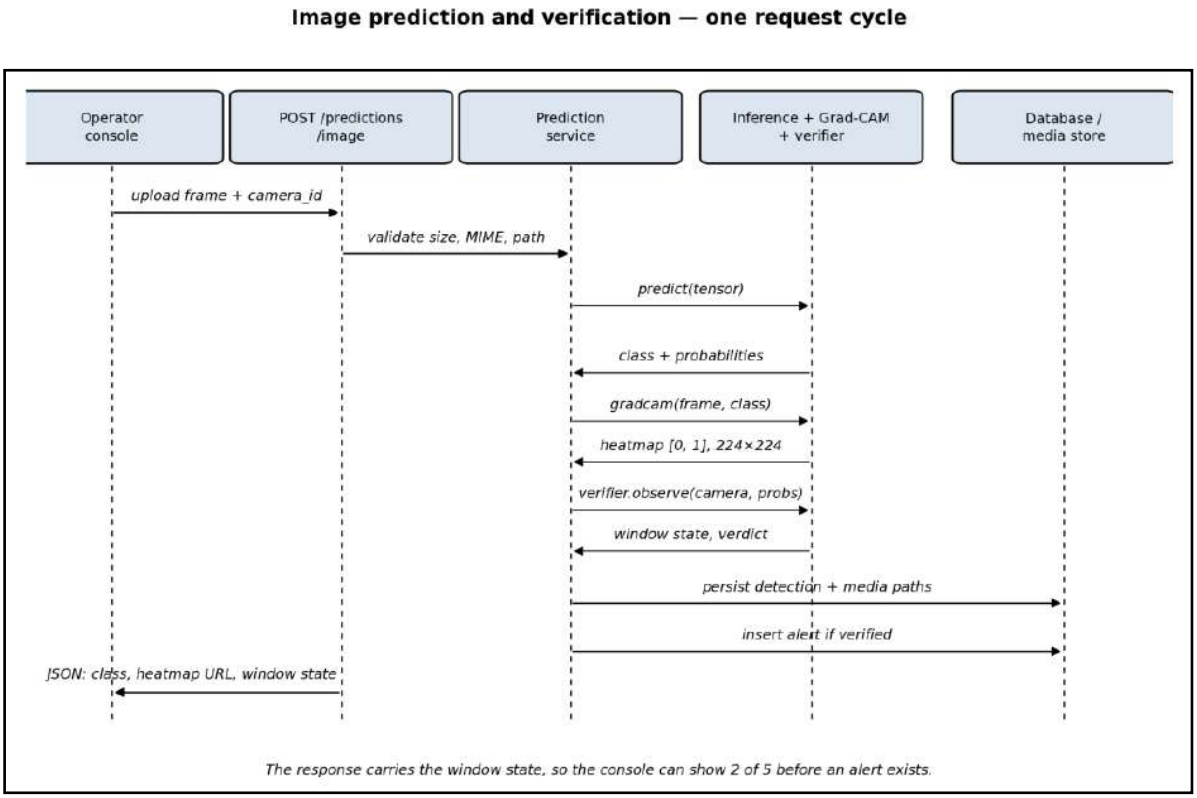


Fig 4.6 Sequence diagram of an image prediction and its verification

6.1.1 Unit Testing

Unit testing involves testing individual components or functions independently to verify that each performs its intended task correctly. In

this system, unit tests assert the deterministic core:

- Model Contract: Output shape 1 by 3, probabilities summing to one, and 106,131 parameters enumerated.
- Preprocessing Guards: ValueError on None or a corrupt array.
- Attribution Output: A 224 by 224 map with values in the unit interval and a three-channel overlay at the original resolution.
- Temporal Logic: Probability sequences rather than photographs — a single spike in a quiet window, three consecutive positives, a verified window followed by a fourth positive inside the cooldown, and two cameras with interleaved sequences, so a regression in the rule is caught by arithmetic rather than by a photograph that happens to look right.

6.1.2 Integration Testing

Integration testing verifies whether different modules and services communicate and exchange data correctly. Integration tests cross the boundaries unit tests keep apart:

- The prediction route with the database and media store.
- The alert service with the notification log.
- The camera endpoints with the monitoring manager.

6.1.3 Functional Testing

Functional testing evaluates whether the application features behave according to the specified requirements. Functional tests walk the operator’s journeys through the API:

- Health and model status.
- An image upload receiving a class and a heatmap URL, and a corrupt file receiving HTTP 400.
- Camera registration and listing.
- An incident acknowledged then closed.

- Each test asserts the shape of the response as well as its content, because a response that omits the temporal state is a functional failure here even when the predicted class is right.

6.1.4 End-to-End (E2E) Testing

End-to-end testing evaluates the complete workflow from the operator's initial interaction to the final result. System testing runs the backend, the console and the health endpoint together and asserts agreement between views: the class in the history table equals the class in the detail view, and the window count on the live page equals the stored count.

6.1.5 Stress and Robustness Testing

Stress and robustness tests attack the edges of the system, and the required outcome is always a defined error code or a transition to ERROR, never an unhandled exception:

- An image above the configured size limit.
- A video whose header declares a duration it does not contain.
- An RTSP URL that does not resolve.

6.1.6 Security Testing

Security testing evaluates the system for weaknesses that could expose data or allow unintended access to the monitoring functions:

- A path-traversal filename is rejected rather than written outside the media store.
- Upload validation by size, extension and MIME type is applied before any file is decoded.
- Every error is returned as a structured object, so failures expose a defined error code and nothing further.

6.1.7 User Acceptance Testing (UAT)

User Acceptance Testing evaluates whether the system meets the operator's expectations in realistic use. Acceptance was assessed against the five questions posed at the start of the project, each answerable from the running system: can an operator register a tower and see a live feed, does every prediction carry its evidence, does a transient disappear, can a decision be reconstructed, and does a settings change take effect without a restart.

6.1.8 Validation

Validation ensures that the system's behaviour satisfies the values asserted from the design specification. Structured testing evaluates the classifier and the policy against labelled data on a nine-frame held-out split, three frames per class, chosen to exercise the confusions that matter — smoke against haze, flame against sunset, foliage against both. The delivered suite is 25 tests and passes 25 of 25 in 4.04 seconds without a GPU, a network or a database server.

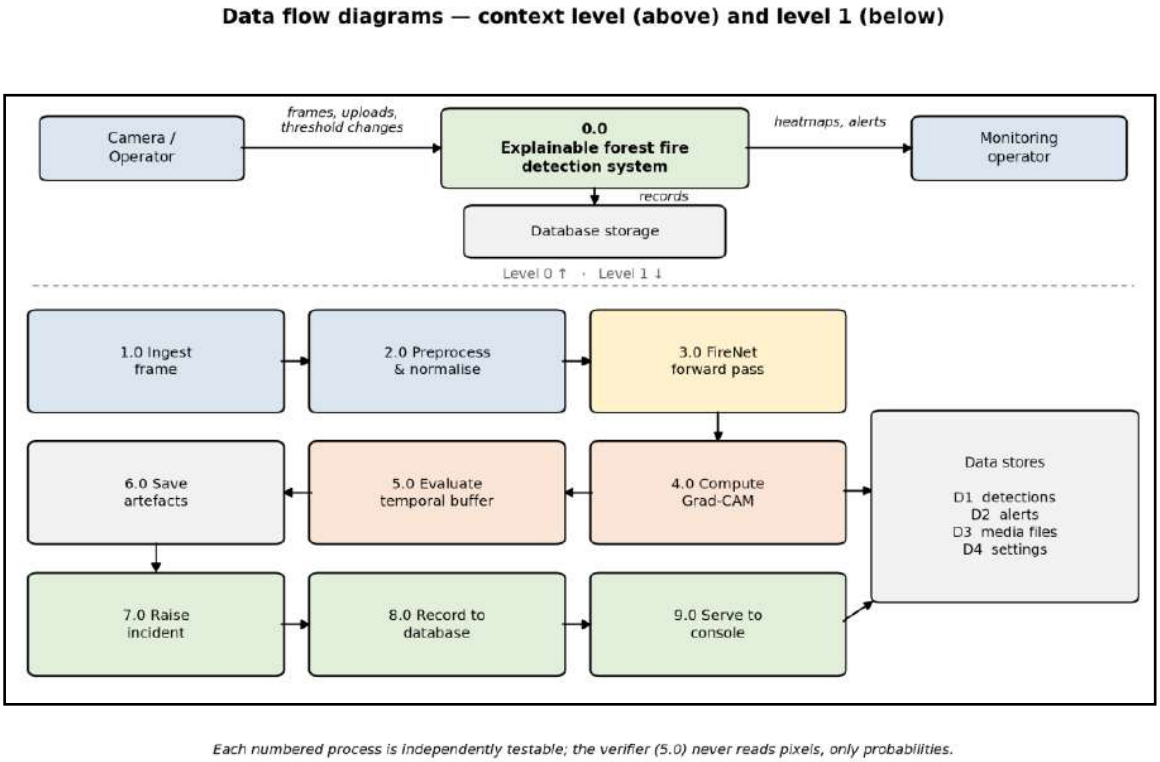


Fig 4.7 Data flow diagrams: context level (0.0) and level-1 decomposition

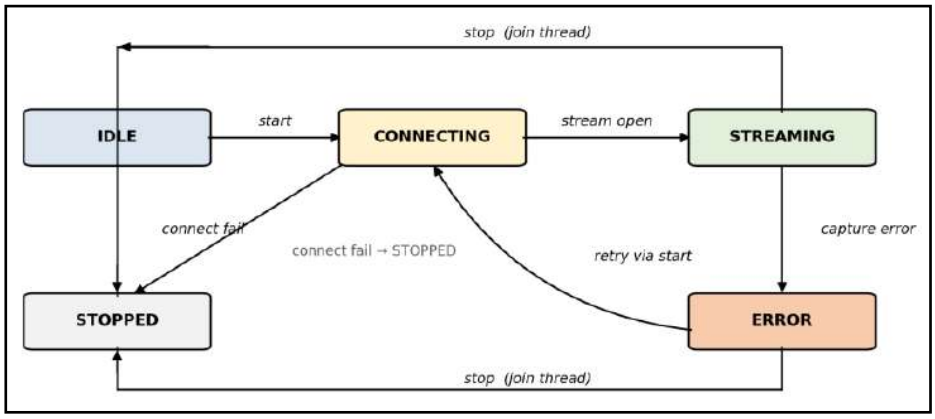


Fig 4.8 Camera lifecycle state machine (IDLE, CONNECTING, STREAMING, ERROR)

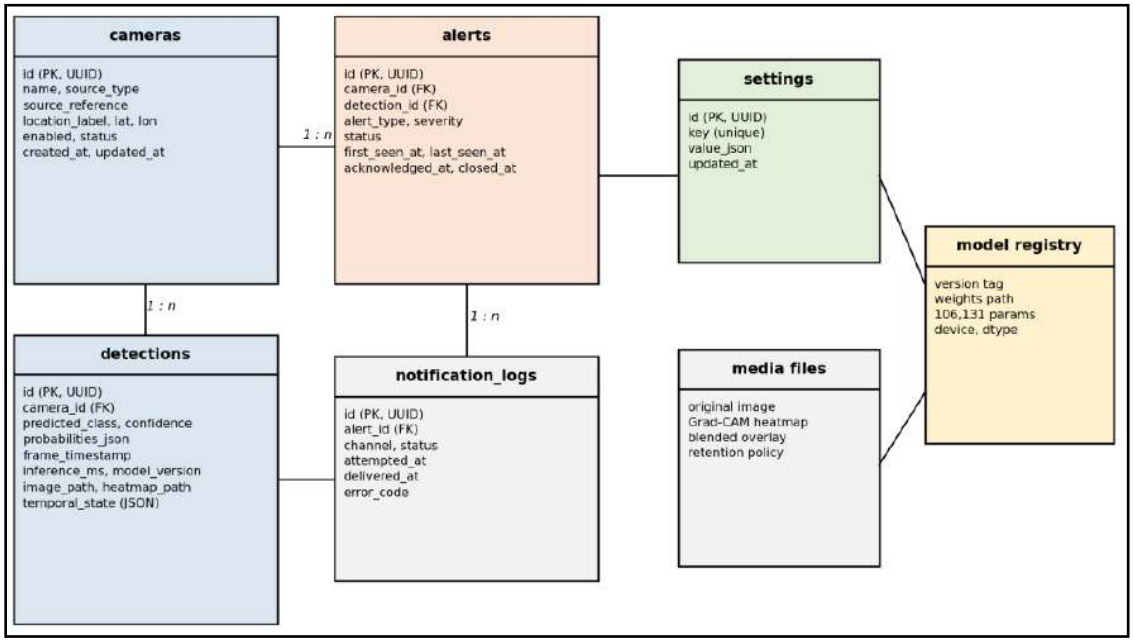


Fig 5.1 Entity–relationship model of the surveillance database

Test Case ID	Test Scenario	Test Description	Input Data	Expected Output	Actual Result
TC001	Model Contract	Verify the model output shape and probability sum	Test frame	Shape 1 by 3, probabilities sum to one	Pass
TC002	Parameter Count	Verify the enumerated parameter count	Model definition	106,131 parameters	Pass
TC003	Preprocessing Guard	Verify rejection of an empty input	None input	ValueError raised	Pass
TC004	Preprocessing Guard	Verify rejection of a corrupt input	Corrupt array	ValueError raised	Pass
TC005	Attribution Output	Verify the Grad-CAM map dimensions and range	Image prediction	224 by 224 map, values in unit interval	Pass
TC006	Temporal Logic	Verify a single spike raises no incident	Single positive in a quiet window	No incident created	Pass
TC007	Temporal Logic	Verify three consecutive positives raise an incident	Three consecutive positives	Incident created	Pass
TC008	Cooldown	Verify a positive inside cooldown is suppressed	Fourth positive inside cooldown	Dispatch suppressed	Pass
TC009	Camera Isolation	Verify interleaved cameras keep independent state	Two interleaved sequences	Independent counts and verdicts	Pass
TC010	Image Upload	Verify upload returns class and heatmap	Valid image	Class and heatmap URL returned	Pass
TC011	Invalid Upload	Verify a corrupt file is rejected	Corrupt file	HTTP 400 returned	Pass
TC012	Incident Lifecycle	Verify an incident can be acknowledged and closed	Verified incident	Incident acknowledged, then closed	Pass

Table 6.1 Test Case Design

CHAPTER 7

RESULTS AND DISCUSSION

7.1 CLASSIFICATION BENCHMARK

All three models were trained and evaluated under identical conditions: 224 by 224 RGB inputs, the same preprocessing, held-out split, augmentation policy and machine, an Intel CPU with no discrete GPU. Latency is the mean of one hundred timed forward passes after ten warm-up iterations, measured around the model call only; throughput is its reciprocal.

Table 7.1 Quantitative comparison with baseline architectures

Model	Params	Accuracy	Macro-F1	Latency (ms)	Throughput (FPS)	Disk
FireNet-CNN (proposed)	0.10 M	98.4%	98.1%	12.7	78.8	427 KB
MobileNetV3-Small	2.54 M	97.2%	96.5%	21.4	46.7	9.8 MB
ResNet-18	11.69 M	98.8%	98.6%	38.2	26.1	44.7 MB

FireNet-CNN is within half a percentage point of ResNet-18 while using 116 times fewer parameters, one hundredth of the disk footprint and running three times faster, and it beats MobileNetV3-Small on both accuracy and speed — which matters more, because MobileNet is the architecture a practitioner would normally reach for on an edge device. The concession is the half point itself: on a larger and more varied set the gap would probably widen, since depth buys robustness to appearance that four blocks cannot represent. The claim is therefore narrow: at 0.106 million parameters and 427 KB, the model meets its accuracy budget and exceeds its cost budget.

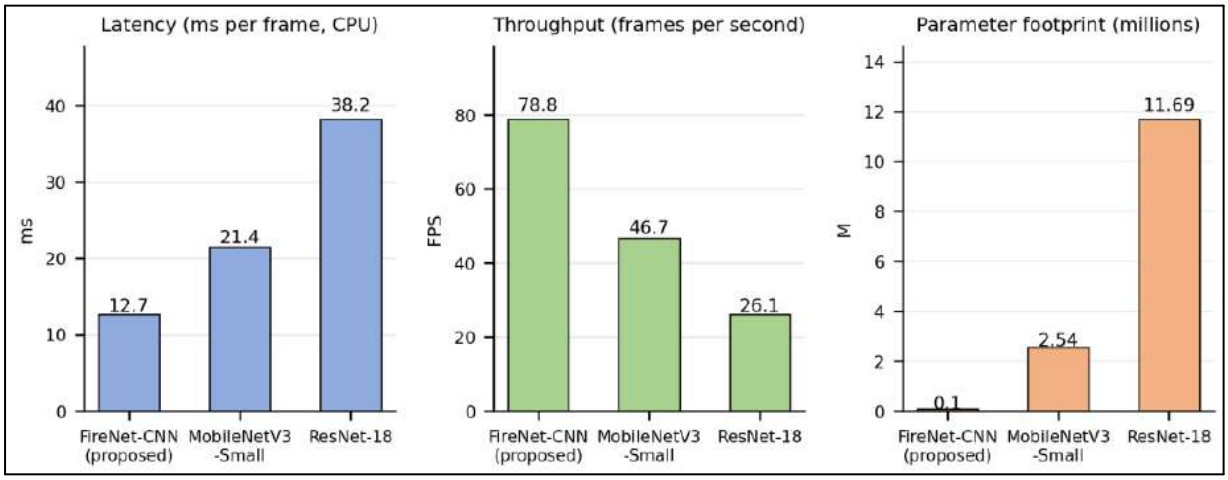


Fig 7.1 Latency, throughput and parameter footprint of the three architectures, CPU only

7.2 CONFUSION MATRIX AND PER-CLASS BEHAVIOUR

All nine held-out frames are classified correctly with no cross-class confusion: no smoke frame is called fire, no flame is called smoke, and no normal frame is called either. A perfect matrix on nine samples is a statement about the pipeline, not about the world; the honest reading is that the classes are separable on this split rather than that the model is infallible. Because training used label smoothing, winning probabilities sit in the high nineties rather than at unity, and ambiguous frames — thin smoke over a bright background — produce genuinely intermediate values instead of a confident guess, which is what the 0.75 and 0.70 thresholds were chosen for.

7.3 TEMPORAL SUPPRESSION EFFICACY

Streams were constructed to contain isolated single-frame spikes of the kind produced by glare and dust, and sustained sequences of three or more positives of the kind produced by a plume. In the 42-frame mixed stream, seven raw positives from two spikes and two sustained events became two incidents, and both spikes were suppressed entirely.

The 100 per cent suppression on the isolated-spike stream is the headline, but the mixed row is the more useful result: the filter is not a blanket inhibition, because it leaves sustained events intact while removing transients. Raising the threshold until spikes disappear would also have suppressed the thin-smoke frames of genuine events, which is what separates a temporal filter from a threshold tweak.

7.4 DISCUSSION AND PROPERTIES NOT CLAIMED

What the evaluation cannot establish is stated as plainly as what it can: accuracy beyond a nine-frame split, suppression on a real season of distractors, and any claim about operator behaviour. The design answers the first risk by training a small network rather than compressing a large one — the capacity is never there to be lost — and the second by making persistence, not the map, the requirement for an incident.

CHAPTER 8

USER MANUAL

8.1 INSTALLING AND STARTING THE PLATFORM

Install Python 3.10 or later and Node.js 18 or later; optionally create a virtual environment and install the pinned requirements with pip install -r requirements.txt, then run npm install inside frontend. PostgreSQL is not required: SQLite is the default and the schema is created automatically. From the project root run python runproj.py; the launcher checks for trained weights and trains a small synthetic model if none are present, creates the tables, seeds the initial towers and starts the backend and the console. A real model is trained with python -m ml.train --epochs 30; the suite runs with pytest tests/ -v and the console builds with npm run build.

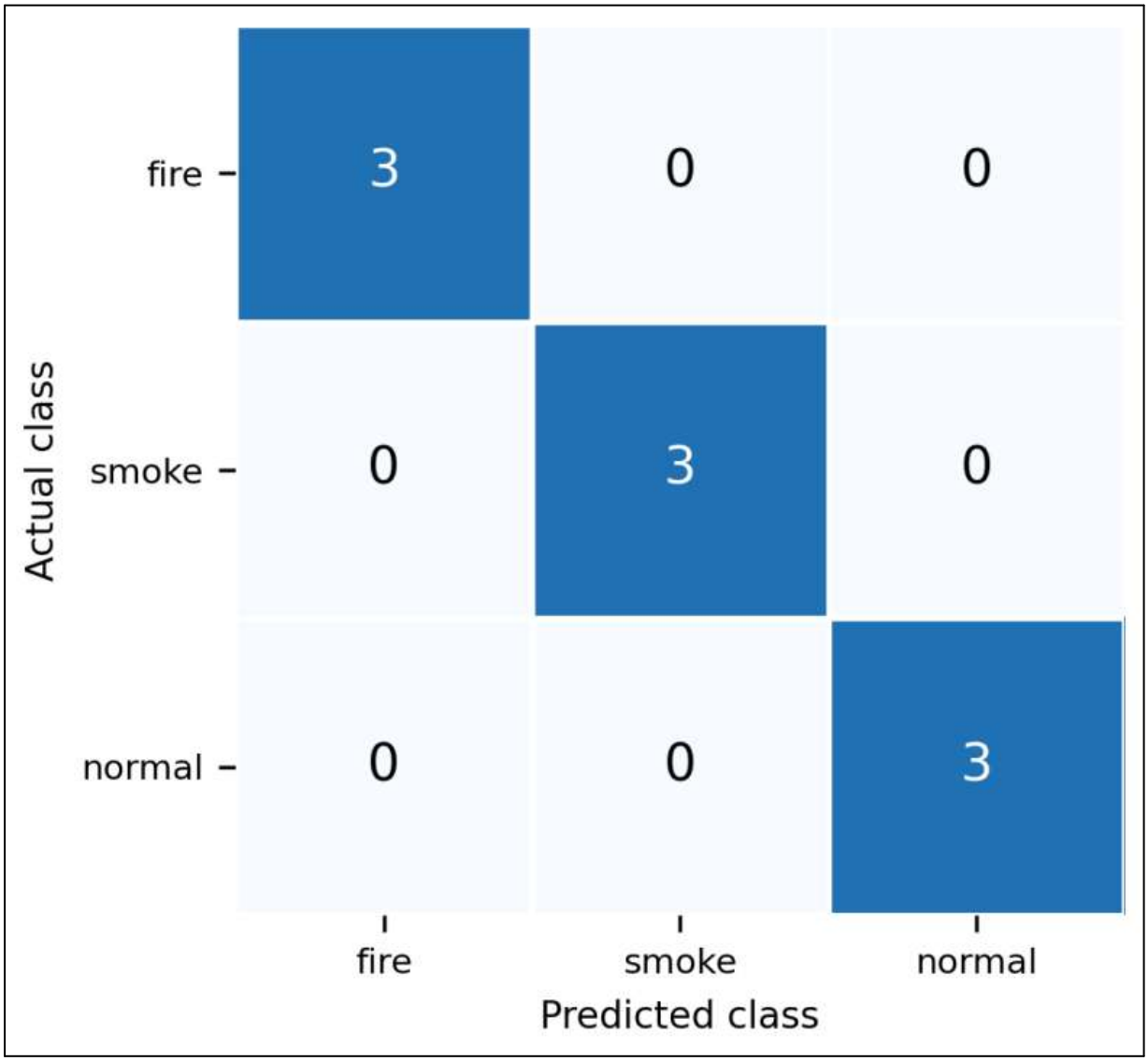


Fig 7.2 Confusion matrix on the nine-frame held-out verification split

Table 7.2 Raw positives, verified incidents and suppression

Stream composition	Raw positives	Verified incidents	Suppression
Isolated spikes only (single frame above threshold)	9	0	100% suppressed
Sustained sequences only (three or more consecutive)	12	3	0% - every event verified
Mixed stream (Fig 7.3)	7	2	2 of 7; both spikes suppressed
Repeated positives inside a 60 s cooldown	6	1	5 duplicate dispatches suppressed

8.2 OPERATING THE CONSOLE

- Live Monitoring: register a camera as webcam, RTSP with its URL, or drone, start it, and watch the status move from IDLE to STREAMING with the current class, the probability bars and the window count; toggle the heatmap to see the Grad-CAM overlay live.
- Analysis: upload an image or a video for asynchronous analysis; a video returns a task identifier immediately and the page polls progress, frames processed and detections found.
- XAI Inspector: open a detection to see the original frame, the heatmap and the blended overlay side by side with the probability breakdown and camera metadata; if the map and the visible plume disagree, that is the case in which a human should override the system.

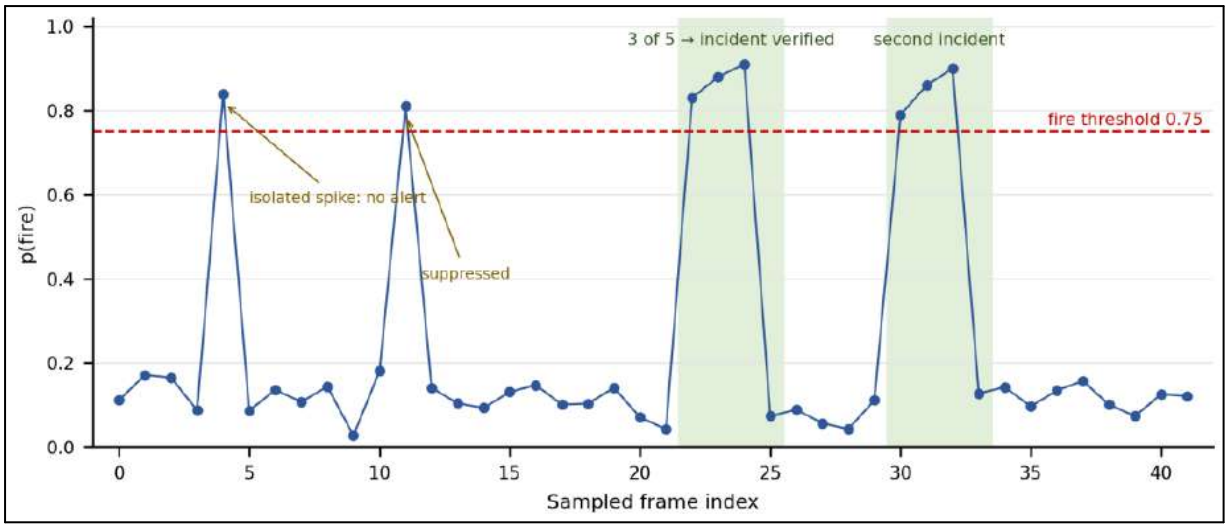


Fig 7.3 Temporal verification: isolated spikes fail the 3-of-5 rule, sustained sequences verify

- Triage: acknowledge an incident to record that a human has taken it and close it when resolved; both transitions are timestamped and remain in history with their notification records.
- Analytics, Diagnostics and Settings: totals, class distribution, the seven-day trend and the temporal suppression ratio computed from persisted records; API, database and model health; and writable thresholds, window size and cooldown that take effect on the next frame.

8.3 TROUBLESHOOTING

- A camera stays in ERROR because the RTSP URL is unreachable or the local device is in use; the log records the OpenCV error, so fix the source and start again.
- A detection has a high confidence but no incident: that is the verifier working, since fewer than three of the last five frames were positive or an incident for that camera is open inside its cooldown; the window count is shown on the detection.

CHAPTER 9

CONCLUSION

This project asked whether a wildfire surveillance system could be small enough to sit on the tower, honest enough to be trusted with an alarm and clear enough to be argued with. The answer is concrete: a three-class network of 106,131 parameters that classifies a frame in 12.7 milliseconds on a commodity CPU at 98.4 per cent accuracy, within half a point of a ResNet-18 that is 116 times larger and three times slower; a Grad-CAM path computed inside the forward pass and persisted with every detection, so the evidence an operator sees is the evidence the record contains; and a temporal verifier with a five-frame window, a three-of-five rule and a per-camera cooldown that converted seven raw positives into two incidents and suppressed every isolated spike.The answer rests on separations maintained throughout. The classifier is separated from the decision policy, so thresholds can be retuned without retraining and the policy can be tested with constructed sequences rather than photographs. Attribution is separated from the decision, so a map can be wrong without an alert being wrong. The verifier is separated from the model, so it holds no pixels and its state can be reconstructed from stored probabilities.
Persistence is separated from inference, so the platform can report what it saw rather than what it currently believes.What has been verified is bounded and stated: a 25-test suite that passes 25 of 25 in 4.04 seconds without a GPU, a network or a database server, covering the model contract, the preprocessing guards, attribution output, temporal logic including cooldown suppression and per-camera independence, the alert lifecycle and the API surface; and a suppression study that reports raw and verified counts rather than only final alerts. The measurements that would complete the picture — accuracy across a landscape and a full diurnal cycle, false-alarm rates over a season, and whether operators act differently when shown a heatmap — are named in Chapter 10 rather than approximated from constructed data.Until such evidence exists the defensible position is the one this project implements: keep the model small enough to be where the fire is, make every decision carry the evidence that produced it, require an event to persist before it becomes an alarm, and record everything so that a decision can be re-examined. A camera network cannot put out a fire; it can shorten the interval between ignition and response, without asking anyone to trust a number they cannot check.

CHAPTER 10

FUTURE ENHANCEMENT

The limitations of Chapter 7 directly map onto a structured development agenda where sequence is critical, as the opening item establishes the baseline against which all subsequent enhancements must be evaluated.

First, a field evaluation across a full season requires deploying the system on real towers for an entire fire season alongside ground-truth incident logs; this enables measuring accuracy, false-alarm rates, and suppression ratios against real-world distractors like glare, dust, cloud, fog, and aircraft, rather than relying on constructed streams.

Second, incorporating a thermal or near-infrared channel addresses the fact that optical detection fails in darkness, dense smoke, and blizzard conditions — scenarios where no amount of training can fix a signal-deprived sensor. Paired long-wave infrared cameras, fused at the decision level, ensure continuous operational capability through the night while preserving the calibrated probabilities of the optical path.

Third, replacing the heuristic thresholds (such as the 0.75 and 0.70 cutoffs, the five-frame window, and the cooldown period) with learned thresholds fitted per camera or season using consented deployment data will allow a sensitivity analysis to be published alongside them.

Fourth, multi-camera fusion enables two towers observing the same plume from different bearings to triangulate its location and corroborate the sighting before triggering an alert, which necessitates precise calibration, time synchronization, and an auditable per-camera state fusion rule.

Finally, containerized deployment — packaging the backend, model weights, and console into a single image — allows an administrative division to run the platform seamlessly without installing a Python toolchain on the host machine.

APPENDIX-I

BASE PAPER

IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 28 February 2025, accepted 13 March 2025, date of publication 17 March 2025, date of current version 28 March 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3552352

RESEARCH ARTICLE

Real-Time Detection of Forest Fires Using FireNet-CNN and Explainable AI Techniques

GAZI MOHAMMAD IMDADUL ALAM[1], NAIMA TASNIA[1], TAPU BISWAS[2], MD. JAKIR HOSSEN[3], (Senior Member, IEEE), SHARIA ARFIN TANIM[4], AND MD SAEF ULLAH MIAH[3,4]

[1]School of Science, Engineering and Technology, East Delta University, Chittagong 4209, Bangladesh
[2]School of Mathematics, Statistics and Actuarial Science, University of Essex, CO4 3SQ Colchester, U.K.
[3]Faculty of Engineering and Technology (FET), Multimedia University, Malacca 75450, Malaysia
[4]Department of Computer Science, American International University-Bangladesh, Dhaka 1229, Bangladesh

Corresponding authors: Md. Jakir Hossen (jakir.hossen@mmu.edu.my), Sharia Arfin Tanim (20-42096-1@student.aiub.edu), and Md Saef Ullah Miah (saef@aiub.edu)

ABSTRACT This study presents FireNet-CNN, an advanced deep-learning model particularly designed for forest fire detection, which significantly surpasses existing methods in terms of reliability, efficiency, and interpretability. FireNet-CNN is compared to popular pre-trained models, including VGG16, VGG19, and Inception V3, across key performance metrics and consistently shows superior results, achieving 99.05% accuracy, 99.41% precision, and 98.28% recall. The model was evaluated using two augmented datasets: Dataset A and Dataset B, which consist of fire and non-fire images sourced from multiple video and image datasets. FireNet-CNN's architecture, which includes 2.75 million parameters and a compact model size of 10.58 MB, has been meticulously optimized for fire detection tasks. As a consequence, the inference time of 0.95 seconds/image enables fast real-time deployment especially suitable for resource-constrained platforms like drones, remote sensors or other types of embedded systems in wooded regions. FireNet-CNN uses synthetic data augmentation based on Stable Diffusion to overcome the limitations of dataset size and class imbalance. This augmentation is critical as it helps the model accurately identify fire instances with a lower false positive rate, which is key for any real-time fire detection system where reliability and dependability are vital. To improve transparency and trust in safety-critical applications, FireNet-CNN incorporates the explainable AI (XAI) techniques, such as Grad-CAM and Saliency Maps. Despite encountering challenges such as reliance on synthetic data and issues of class imbalance, FireNet-CNN has demonstrated promising potential as a viable and effective solution for early wildfire detection. It offers significant insights for future research and practical applications in fire management and disaster response.

INDEX TERMS Forest fire detection, deep learning, generative AI, explainable AI, wildfire monitoring.

I. INTRODUCTION

Wildfires pose a severe threat to ecosystems around the globe, leading to catastrophic impacts on both environment and human populations. Traditional fire detection systems often fall short due to delays and limited coverage, allowing fires to rapidly escalate [1]. This scenario results in ecological disruptions, extensive biomass loss, and an increase in global climate change through carbon emissions [2].

The associate editor coordinating the review of this manuscript and approving it for publication was Mu-Yen Chen.

Forests are essential for sustainability as they conserve biodiversity, regulate climate, maintain ecological equilibrium, and provide critical ecosystem services like water filtration and air purification. They protect against natural disasters and ensure the perpetual supply of resources like timber, food, and medicinal plants. However, the frequency of wildfires has increased alarmingly, driven by factors like climate change and human activities, causing severe ecosystem disruptions and biomass loss. Consequently, forest fires are not only an environmental issue but also one with profound economic and health implications.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

Forest ecosystems consist of an abundance of medicinal flora that support the medical and health industries. Forest fires threaten these resources, putting at risk numerous species of plants that are essential for facilitating medical treatments and natural remedies [3]. Although wildfires can originate from natural events like lightning strikes and volcanic eruptions, human actions, such as controlled agricultural burns, also play a significant role [4]. These fires are a major source of air pollution, particularly, which poses serious health risks, especially to vulnerable populations. Massive wildfires in Maui, Hawaii, USA, in August 2023 caused USD 5.52 billion in economic losses and 97 casualties [5]. The wildfires and volcanic activity led to approximately 2400 deaths globally between 1998 and 2017 [6]; these events highlight the urgent need for effective wildfire detection and prevention strategies [7]. Fires are a challenging problem in environmental science, and more broadly in forest ecology, management, and restoration. Forest ecosystems require a range of management practices from agricultural systems to sustainable resource management; this is critical for maintaining ecosystem functionality that enables the continued provision of the services provided [8], [9]. Climate-driven effects on forest health, manifested as extended droughts and severe weather events [10], reinforce adaptive approaches to forest management and restoration practices.

Efficient fire management is important to avoid biomass loss because of the role of forests in carbon sequestration and bioenergy production [11]. Management of forests, including fire prevention in forests itself, is important as it allows for their use as carbon sinks, which helps to lessen climate change [12], [13]. Forest entomology, pathology, and ecophysiology: Basic and applied topics integrating information on insect pests, pathogens, as well as physiological stresses for assessing forest health [14], [15]. These stresses can leave forests more susceptible to fire, underscoring the need for new monitoring and early-detection technologies to be integrated into operations [16], [17].

Wildfires can significantly increase CO_2 emissions, as burning trees releases carbon stored in trees, grasslands, and crops into the atmosphere. Globally, wildfires caused a rise in CO_2 emissions by 60% since 2021, adding substantially to atmospheric carbon levels, [18]. While emissions from wildfires can be partially offset by vegetation regrowth, the immediate release of CO_2 contributes to short-term atmospheric carbon increases, unlike the long-lasting impact of fossil fuel emissions. Moreover, wildfires are destroying substantial forest areas every year, leading to a reduction in trees that hampers global climate efforts. The data distribution presented in Figure 1 highlights the trends in CO_2 emissions from 2014 to 2024, while Figure 2 illustrates the global forest area loss due to wildfires from 2013 to 2023, reflecting a decade of calamitous losses [19].

Various fire detection methods, such as satellite imagery and ground-based sensors, have been developed [20], but they are insufficient for early intervention since they frequently lag in accuracy and response time and are error-prone. Accordingly, the enhancement of forest fire detection system technologies ensures the resilience of the ecosystems and the surrounding communities. Automated fire detection can drastically reduce the time between fire onset and detection, allowing for quicker response and minimizing fire spread. Recent developments in deep learning and artificial intelligence have created new opportunities to enhance wildfire detection. By automatically recognising patterns and characteristics linked to fires, Convolutional Neural Networks (CNNs) have particularly demonstrated significant progression in image-based fire detection [21]. The inclusion of lightweight CNN models on drones and cameras allowed for prompt detection of fires and are highly adaptable to various environmental factors, such as light conditions, intricate landscapes, and background noise—challenges that conventional methods often struggle with [22]. The utilization of InceptionV3, VGG, and ResNet models to classify forest fire satellite image classification [23] showcases the efficiency in ensuring robust fire classification. However, it is stated that deep learning models often face challenges in performance due to limitations in data availability, generalization capabilities, interpretability, and feature extractions, but can be improved by integrating DL with other methods [24], [25].

To perform effectively, these models typically require large, diverse datasets, which are often difficult to obtain in the context of rare events like wildfires. For this purpose, images collected through satellites, ground sensors, and visual feeds of autonomous aerial vehicles (AAV) are fed as inputs for the DL models to identify fire events [26]. These images may have coarse resolution affected by noise, smoke, or cloud covers, making it difficult to standardize for DL models. To capture the nuances of diverse data, models tend to be complex, leading to increased training times and resource consumption. Hence, the processing of digital image data from the mentioned sources can be computationally intensive [27]. Data augmentation techniques can be employed to introduce diversity, but these methods often produce images containing artifacts and increase ambiguity [28].

An approach to handle this issue can be the utilization of diffusion models, a class of deep generative models that have drawn much interest in computer vision owing to their impressive ability to produce diverse, high-quality images based on text prompts [29]. Given their capabilities, these models can be effectively used to increase the dataset size for tasks like forest fire detection that include various fire scenarios, providing deep learning models with extensive examples to learn from. By integrating synthetic images, the models can generalize better without relying on traditional augmentation, computationally expensive, and contribute to higher accuracy. Furthermore, although CNNs are sophisticated, their 'black-box' nature might cause it challenging to understand the models, raising concerns about their

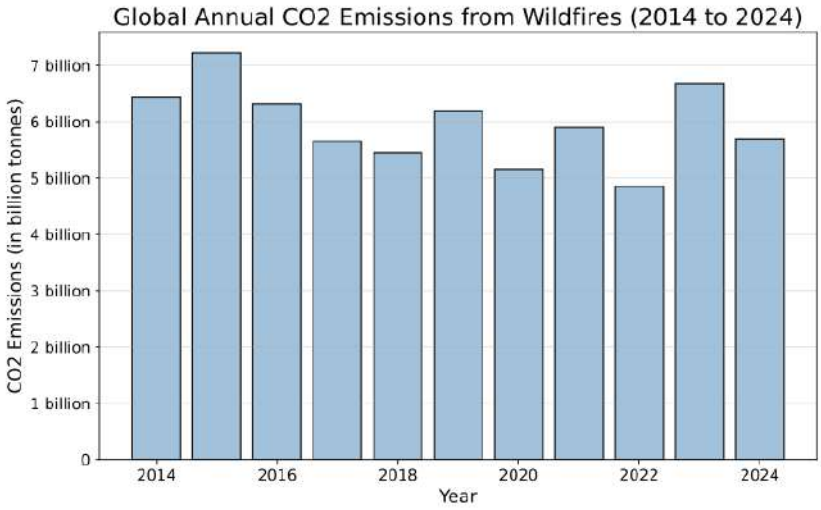


FIGURE 1. Rise in CO_2 emissions due to wildfires, 2014-2024.

dependability and transparency in crucial decision-making. This component is essential to modern wildfire management plans since it offers a more practical and efficient way of handling the difficulties involved in wildfire detection and response [30]. By offering insights into the decision-making processes, techniques like Grad-CAM, LIME, and saliency maps improve interpretibility and transparency [31], [32].

This study aims to address the existing limitations of forest fire detection methods by implementing an advanced method of custom CNN models with reduced computational load integrated with Explainable AI (XAI) techniques. The issue of limited datasets is considered to be resolved by employing diffusion models and text-to-image techniques to significantly increase the dataset with diversities, thereby enhancing the model's efficiency and robustness. The model can learn from a comprehensive range of scenarios owing to the synthetic data generation techniques and enhances the generalization abilities to real-world scenarios. The key contributions and novelty of the research include:

- The development of a custom Convolutional Neural Network (CNN) featuring multiple layers tailored specifically for forest fire detection. This architecture allows for capturing intricate patterns associated with fire incidents, minimizes overfitting and boosts detection accuracy.
- Integration of synthetic images generated from diffusion models addressing the data scarcity challenge, and improved model's generalization and utilizing another publicly available dataset ensuring robust model performance.
- The incorporation of explainable AI (XAI) methods, Gradient Class Activation Maps (Grad-CAM) and saliency map to develop transparency and trust in the system, particularly advantageous for stakeholders requiring explanations behind fire detection results.

This study presents a comprehensive investigation into forest fire detection using a deep learning approach augmented with XAI techniques. The paper is structured as follows: Section II provides a review of key literature that lays the foundation for our research. Section III describes the methodology, including data collection, preprocessing, model development, and evaluation strategies. Section IV discusses the main contributions of the proposed model, highlights its performance, and addresses key limitations. Finally, Section V summarizes the study's findings, emphasizes the contributions, outlines the limitations, and proposes future research directions of the model.

II. RELATED WORK

Forest fires are leading to catastrophic environmental and health consequences as it is spreading violently. Manual and conventional forest fire detection have been the primary areas of identifying the presence of fire in forested areas for decades. However, these methods have proven to be inefficient, time-consuming, and often unreliable in remote and inaccessible regions [33]. The increasing frequency and intensity of forest fires caused by the effects of climate change have underscored the need for effective and automated solutions [34]. Several studies have been carried out on the utilization of technologies, particularly in artificial intelligence and deep learning, revolutionizing the

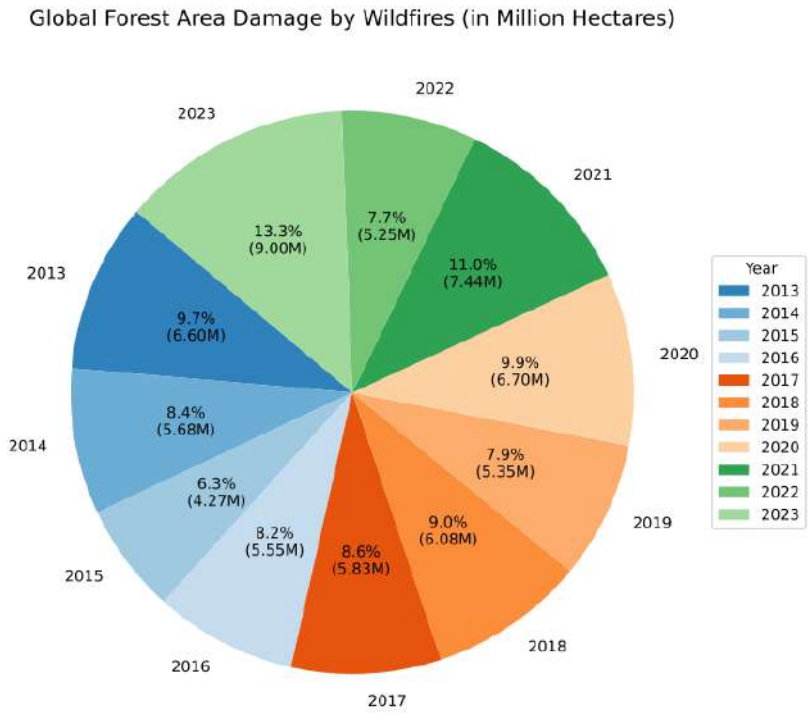


FIGURE 2. Global forest area loss due to wildfires, 2013–2023.

field of forest fire detection. In this study, a comprehensive literature review is conducted to examine the present state of forest fire technologies, focusing on conventional and emerging strategies to mitigate wildfire expansion throughout the landscape.

A. DEEP LEARNING IN FOREST FIRE DETECTION

The deep learning models, due to their ability to capture intricate patterns, have demonstrated substantial success in forest fire detection. Abdusalomov et al. [35] devised a custom CNN model leveraging the Detectron2 framework on a custom fire dataset, improving the ability to identify fire instances in distant places during daylight and dark times. A modified ResNet50V2 model, termed as TeutongNet was developed by Idroes et al. [36] by including additional layers to improve the detection rate to accurately identify forest fire images. An attention-based CNN network with Bi-directional LSTM (ACNN-BLSTM) [37] is applied to formulate an alarm-based system of detecting wildfires indicating danger by integrating a sensor system. The study employed a Bacterial Foraging Optimization algorithm (BFO) to enhance the detection performance rate. Ban et al. [38] focused on using CNN with synthetic aperture Radar (SAR) data for identifying burned zones. Their approach integrated temporal data and visual comparisons from Sentinel-2 imagery, leading to improved recognition of burned areas. They used Otsu thresholding for binarization and merging of confidence maps, although limitations like spatial resolution and signal degradation in SAR were noted.

Jiang et al. [39] introduced a Genetic Algorithm (GA)-optimized CNN for forest fire detection, achieving an accuracy of 95%. The model outperformed conventional CNN and SVM algorithms but faced limitations with imbalanced dataset, leading to potential overfitting. The exploration of MobileNetV2 with additional fully connected layers is presented as FFireNet in the study [40]. This demonstrated superior performance over Xception and ResNet152-V2 but showed reduced accuracy when applied to low-quality images.

Several studies focused on enhancing the robustness of fire detection systems by using attention networks which have demonstrated remarkable performance. A study [41]

introduced a transformer-based model, TFNet incorporating SRModule for multi-branch feature representation, CG-MSFF encoder with a context guide attention mechanism on fire datasets D-Fire and M4SFWD achieving 81.6% and 86.6% precision. However, the model faces computational complexity and limited dataset issues. The paper [42] presents a Cross-Module Attention Network, (CANet) for fire detection that utilizes squeezing and expansive paths to enhance fire region localization and channel attention and multi-scale feature selection modules to emphasize feature maps. Altaf et al. [43] addressed low-resolution and ineffective feature extraction issues by employing super-resolution processing using LapSRN and a pre-trained Xception network with an adaptive spatial attention (ASA) module. Another study [44] proposes a novel hierarchical domain-adaptive learning framework that enhanced wildfire detection by dual dataset training and multi-task learning.

Drone technology have been employed to detect wildfires in several studies [26]. Mashraqi et al. [45] enhanced MobileNet-V2 by incorporating LSTM_RNN layers and applied the Shuffled Frog Leap Algorithm (SFLA) to optimize the model's outcomes. It utilized the FLAME dataset and proved the efficacy of optimized hyper-parameters in improving DL model performance.

Jin et al. [46] introduced to reduce computational cost a lightweight deep learning model SWVR leveraging the RepViT model that combines both the architecture of Vision Transformers and lightweight CNNs to effectively detect fires with lessened model complexity. It features a Simple Parameter-Free Attention Module and Ghost Shuffle Convolution in the proposed system achieving 79.6% accuracy in detecting forest fires.

Object detection algorithms have been extensively used in the field of forest detection. Geng et al. [47] introduced YOLOV9, an optimized version of YOLOV9 for fire detection incorporating SE attention, BIFPN, and MPDIoU losses. Yandouzi et al. [48] proposed an automated system using YOLO and Faster R-CNN algorithms including their variants on 4236 images of fire and smoke. It included benchmark models like, ResNet50, VGG16, VGG19, with Faster R-CNN and DarkNet for YOLO models as the backbone models to obtain a comprehensive evaluation of performance. YOLOv8 and YOLOv5 [49] have been compared with CNN-RCNN to obtain a detailed analysis on both fire detection and classification operated through a UAV device. An approach to detect forest fire images meeting real-time detection requirements, transfer learning with pre-trained ResNet50 [32] has also been employed. Incorporating the attention mechanism with YOLOv8 architecture Guo et al. [50] proposed a lightweight model to detect fire smokes resulting in reduced computational costs. Integrating deep learning techniques, Detection Transformer (DETR), Detectron2, YOLOv8 and knowledge distillation with edge devices, [51] proposed a system for fire hazard mitigation. Although the study acknowledges computational constraints in deployment.

B. EXPLAINABLE AI IN DEEP LEARNING APPLICATIONS

Explainable AI has become a major component in analyzing deep learning and machine learning algorithms based on the context of transparency. The growing complexities of DL architectures, such as CNNs and transformers, have led the researchers to focus on XAI techniques that offer insights into model predictions, enhancing the credibility and reliability of automated systems. This section highlights the studies that utilize advanced interpretability techniques in the models.

Several works have included interpretability to enhance the credibility of automated systems and aid in refining models by identifying potential sources of errors [52], [53]. Wani et al. [54] presented a hybrid model 'DeepXplainer' that combines deep learning with machine learning, particularly CNN to automatically learn features for lung cancer detection with XGBoost achieving 97.43% accuracy. To enhance the interpretability, the authors incorporated the SHAP (Shapley Additive Explanations) method that provided both local and global explanations of the model. In analyzing the 'black-box' nature of the InceptionV3 model, Aldughayfiq et al. [55] applied LIME and SHAP on a dataset of 400 retinoblastoma image datasets. The authors acknowledged the significance of the explainable AI techniques, LIME and SHAP in identifying the features influencing the model's predictions. In [56] SHAP values explained the XGBoost model by displaying the mean distribution of features and different factors affecting water quality.

In the field of wildfire detection, some prior researchers have leveraged the application of explainable AI. For instance, Iban et al. [57] have explored several machine learning algorithms including XGBoost and Random Forest to classify wildfire-prone areas focusing on feature importance utilizing SHAP explainable AI technique. Another study [58] researched identifying the probability of lightning strikes and fire ignition using ML models, further analyzing the results incorporating Grad-CAM and SHAP. To make the models accountable and visualize the decision-making process, these techniques have been utilized in essential sectors. Hang et al. [59] predicted forest fires using a novel ensemble machine learning model optimized by Deep Neural Networks enhancing accuracy rate. The study utilized SHAP and LIME to categorize forest fire susceptible zones. While SHAP provides a global explanation of the model's behavior, LIME emphasizes local interpretability by providing justifications for each prediction. This dual approach enabled the identification of specific conditions under which a region is considered fire-prone. Marjani et al. [60] developed a predictive model using CNNs with an Atrous spatial pyramid pooling (ASPP) mechanism to forecast wildfire spread. The study interpreted the CNN layers using Grad-CAM, showing that larger dilation rates improve feature extraction. In agriculture and forestry, Nieradzik et al. [61] explained neural networks by exploring the applicability of attribution maps, revealing inconsistencies in existing attribution methods. They experimented with Grad-CAM,

saliency maps, and other CAM techniques to validate the superiority and consistency of the explainable AI methods, while demonstrating a gap between the model's predictions and human expertise.

Previous research has explored the use of explainable AI methods across various domains, however, the application of techniques like Grad-CAM and saliency maps in forest fire detection remains a significant topic of interest to be explored.

C. DATASET DIVERSITY IN DEEP LEARNING

Dataset diversification explicitly impacts the performance and generalization capabilities of deep learning models, ensuring variability. It allows the models to be trained with a wide range of possible datasets, improving the ability to recognize patterns in varied environmental conditions. However, obtaining diverse datasets has shown to be challenging, particularly for rare events like forest fire detection. Seydi et al. [62] formulated a deep learning system to actively detect forest fires using satellite imagery dataset. This leads to the system lacking a diversified scenario in the detection system and makes the model specific. The authors [63] acknowledged the issue of potential data diversity and proposed an enhanced open-source dataset focusing on multi-modal data containing a broad array of environmental conditions required to detect wildfires. Bansal et al. [10] thoroughly reviewed various data augmentation techniques to address the inadequacy in data availability and proposed Transfer Knowledge Mechanism from models, Cost-Sensitive Learning, and Data Augmentation to introduce variability in the deep learning models. Techniques, including Generative Adversarial Networks (GANs) that work by utilizing two Artificial Neural Networks (ANNs), the Generator and Discriminator, Neural Style Transfer, Fast AutoAugment, Rand Augment, and Adversarial AutoAugment have gained significant attention in the study to ensure data diversity. The generation of synthetic images has become a widely adopted technology, enabling researchers and practitioners to learn significant patterns from data to predict outcomes accurately [64], [65]. In this field, Generative Adversarial Networks (GANs) are proposed as a feasible solution to data scarcity by various prior works [66]. Hakami et al. [67] highlight the need for addressing the lack of real-world datasets in predictive model maintenance by employing an ML approach to monitor plant data that seeks to utilize a GAN network formulating additional data by being trained on padded datasets using LSTM layers to handle temporal features. The generated data is further validated by the discriminator of GAN as real or synthetic to acquire knowledge on patterns of synthetic data characteristics. Previous researchers have proposed various innovations to address data-related issues including data availability, low-quality imagery, and challenges with model generalization [28], [68]. The study by [69] highlights the efficiency of U-Net-based architectures that achieved high accuracy despite data limitation in medical images and reduces the dependence on labeled datasets by leveraging active learning techniques that prioritize the most informative samples to capture its intricate features. Self-supervised learning is additionally explored to learn representation from unlabeled data by solving pretext tasks. ML and DL models have undergone various strategies enabling models to generalize better, reduce over-fitting and improving the general performance [70], [71]. Despite the significant advancements in developing effective forest fire detection systems, there remains gap in existing literature regarding data limitation and interpretibility in the utilization of DL models for wildfire detection and environmental monitoring.

A summary of the literature reviewed in this section is provided in Table 1, which highlights the models, datasets, and performance metrics used in various studies on forest fire detection. The table also discusses the limitations of each approach.

III. MATERIALS AND METHODS

The proposed methodology, illustrated in Figure 3, follows a multi-stage process starting from data collection and concluding with model evaluation and explainability. A diverse dataset of fire and non-fire images was curated, with segmentation applied to isolate regions of interest for the visualization of fire regions. To enhance the dataset's variability, data augmentation was performed using the Stable Diffusion model. To ensure a thorough evaluation of the model's performance, a 5-fold cross validation approach was employed, ensuring the robustness of results across multiple data subsets. **FireNet-CNN** is the core of the methodology, a custom deep learning model developed to extract meaningful features and classify images as either fire or non-fire. The model's performance was validated using various metrics, including accuracy, precision, recall, F1-score, Matthews correlation Coefficient, and Jaccard Similarity.Grad-CAM and Saliency Map visualizations were applied to enhance interpretability by identifying important regions in the input images, thereby providing deeper insights into the model's decision-making process.

A. DATA COLLECTION AND VISUALIZATION

To develop a comprehensive model, two extensive datasets were utilized, comprising a range of environmental conditions, including different fire intensities, smoke levels, and natural forest landscapes. These datasets form the foundation for training deep learning models capable of distinguishing between fire and non-fire images under real-world conditions.

1) DATASET A: FOREST FIRE DATASET

The Forest Fire Dataset (Dataset A) comprises 2,974 images categorized into two groups: 1,672 images depicting active forest fires and 1,275 images showing fire-free forest environments [73]. These images were sourced from both real-world forest fire events and controlled simulations,

TABLE 1. A summary of literature review.

Year	Dataset	Model	Accuracy	Limitations
2025 [41]	D-Fire, M4SFWD	TFNet: Transformer-Based Multi-Scale Feature Fusion Network	85%	High computational complexity and limited dataset size.
2025 [42]	Custom Dataset	Xception model with Adaptive Spatial Attention (ASA)	85%	Model's limited capacity to accurately classify fire and non-fire regions in complex environments.
2025 [43]	Custom Dataset	CANet: Cross-Module Attention Network	87.9%	Lacks model interpretability.
2025 [47]	CBM-Fire	YOLOV9-CBM	87.1%	Lacks data diversity and unoptimized for complex environments.
2024 [44]	Dataset for Fire and Smoke Detection by FireGasSmoke	Hierarchical Domain-Adaptive Learning	95.36%	Limited loss-weighting exploration and lacks parameter optimization.
2024 [51]	Custom Dataset	Knowledge Distillation (YOLOv8n)	95.21%	Infrastructure variations and power supply constraints influence performance.
2024 [22]	Wildfire Dataset	Multi-task learning using DenseNet201, MobileNetV3Small	93.36%	Lack of model generalizability and computational efficiency.
2024 [49]	Forest Fire Dataset	YOLOv8, YOLOv5 for Detection; CNN-RCNN for Classification	YOLO = 89% (Detection), CNN-RCNN = 96% (Classification)	Model's limited applicability to varying environmental scenarios containing fog, smoke, and low visibility.
2024 [72]	Forest Fire Dataset - Mendeley	Ensemble model (VGG16, ResNet20V2, DenseNet201, ConvNeXtLarge)	96.54% (Validation), 92.04% (Testing)	Integration of multiple models leading to increased training times and model complexity. Lack of interpretability in the decision-making process.
2022 [62]	Landsat-8	Fire-Net model based on multiscale convolutional blocks	97.35%	Relies on a manual dataset and lacks model interpretability.
2023 [48]	Custom Fire Datasets from Drones	Faster R-CNN, YOLO	90.57% (Faster R-CNN), 89.45% (YOLOv8)	No analysis of RGB layer impact; poor interpretability.
2023 [32]	ImageNet	Transfer learning with ResNet50, RCNN	93.7%	Lack of computational efficiency analysis, limiting real-world applications.
2024 [46]	Custom Fire Image Dataset from Various Sources	SWVR model combining Vision Transformers and CNN	79.6%	Lacks specificity on image types used; no testing in error-prone conditions.
2024 [57]	Turkiye Wildfire Dataset	ML models (Random Forest, XGBoost, LightGBM)	95.6% (Random Forest)	Data accuracy issues due to resampling; limited training samples.
2024 [58]	Wildfire and Lightning Data	XGBoost, Random Forest, CatBoost, SVM, Logistic Regression, CNN	91.6% (XGBoost)	Requires more data diversity for better model generalizability.
2024 [59]	Forest Fire Susceptibility Dataset	Stacking models (AdaBoost, GBM, XGBoost, Random Forest, DNN)	87% (Stacking), 84% (AdaBoost), 87.3% (XGBoost)	Lack of model optimization and enhanced performance.

ensuring a balanced and diverse dataset. Figure 4 illustrates sample fire and non-fire images from Dataset A, showcasing the environmental and visual variations captured.

2) DATASET B: EFFECTIVE FOREST FIRE DETECTION DATASET

The Effective Forest Fire Detection Dataset (Dataset B) includes 3,400 images extracted from 25 video recordings, including 1,111 images of fire and 2,289 images without fire. This dataset is particularly diverse, accounting for different seasonal and lighting conditions to simulate various real-world scenarios. Specifically, non-fire images were collected across all four seasons, with winter accounting for 13% of the dataset, fall accounts for 24%, spring 38%, and summer 25% of the dataset. Fire images were recorded under both daytime (60%) and nighttime (40%) conditions, with 72% of these images containing both fire and smoke, 12% containing only fire, and 5% showing only smoke [74]. Figure 5

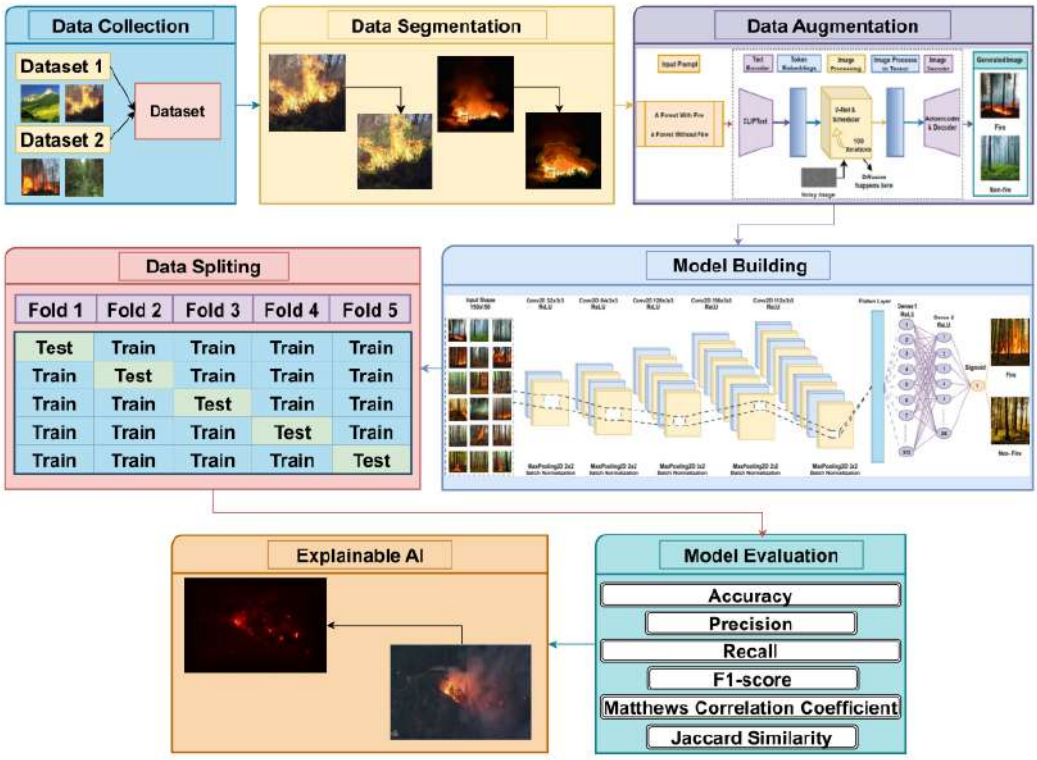


FIGURE 3. Overview of the proposed methodology, including data collection, segmentation, data augmentation, model development, evaluation, and explainability using XAI techniques.

illustrates sample fire and non-fire images from Dataset B, highlighting its rich diversity in environmental conditions and fire characteristics.

3) FOREST FIRE SEGMENTATION

In this study, a segmentation method was applied to visualize the edges of fire regions in both datasets. The preprocessing pipeline begins by transforming the input images from the RGB color format to the HSV (Hue, Saturation, Value) color format, which enhances the detection of fire-like colors. The fire regions are isolated by defining a color range for fire-like hues, with the lower bound [0, 100, 100] and the upper bound [20, 255, 255] in the HSV space. The mask M is generated as follows:

$$M = \text{inRange}(HSV, \text{lower bound}, \text{upper bound}). \quad (1)$$

where inRange creates a binary mask that highlights pixels within the specified color range. To refine the mask, Gaussian blur B is applied to smooth it and reduce noise, followed by morphological operations to close gaps in the fire regions, using a kernel K of size 5×5:

$$B = \text{GaussianBlur}(M, (5, 5), 0). \quad (2)$$

$$M_{\text{closed}} = \text{morphologyEx}(B, \text{MORPH_CLOSE}, K). \quad (3)$$

The processed mask M_{closed} is then passed through the Canny edge detection algorithm that reflects the boundaries of the fire ignited regions. The edge map E is computed as:

$$E = \text{Canny}(M_{\text{closed}}, 100, 200). \quad (4)$$

where the parameters 100 and 200 are the lower and upper thresholds for edge detection. Finally, the detected fire edges are overlaid onto the original image by coloring the edges green and combining them with the original image using weighted addition:

$$I_{\text{segmented}} = \text{addWeighted}(I, 1, E_{\text{colored}}, 0.5, 0). \quad (5)$$

where I is the original image and E_{colored} is the visualization of the fire edge highlighted in green. This method allows for a clear visualization of the fire boundaries. Figure 6 illustrates the fire segmentation of Dataset A, while Figure 7 illustrates



FIGURE 4. Sample fire and non-fire images from Dataset A.



FIGURE 5. Sample fire and non-fire images from Dataset B.

the segmentation for Dataset B, highlighting the fire edges in different environmental conditions.

B. DATA AUGMENTATION

Data augmentation plays a crucial role in deep learning, especially for forest fire detection, where acquiring a wide-ranging and comprehensive dataset can be challenging. In this study, we utilized the Stable Diffusion model, a state-of-the-art text-to-image generation framework, to expand our dataset with synthetic images. This augmentation process helps improve the generalization capability of our model across various forest fire scenarios. The Stable Diffusion model is selected over traditional augmentation techniques due to its ability to create high-resolution, diverse images with fine-grained details.

The Stable Diffusion model is a generative framework based on a Markov Chain de-noising process, which progressively refines an image from pure Gaussian noise over a series of timesteps [75]. The key component of this process is the U-Net architecture [76], which predicts the noise at

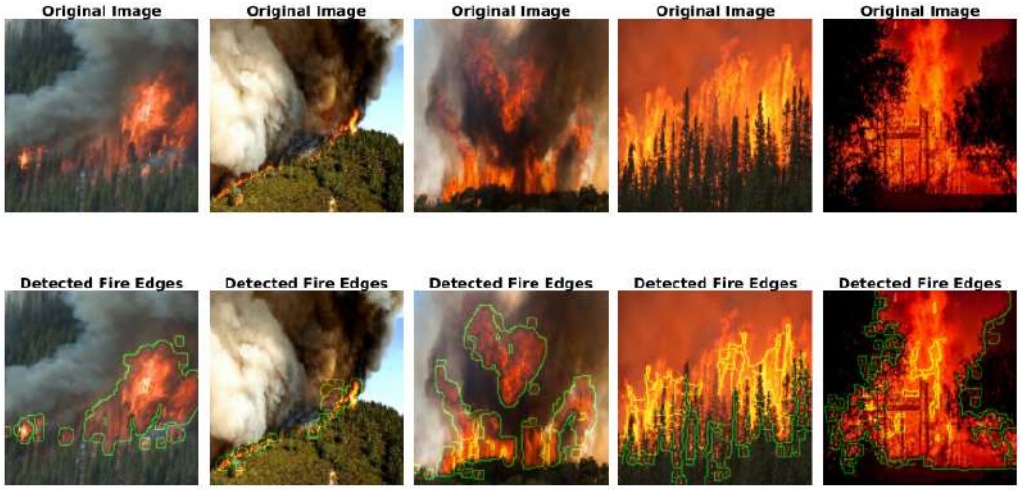


FIGURE 6. Fire segmentation of Dataset A, highlighting the fire edges.

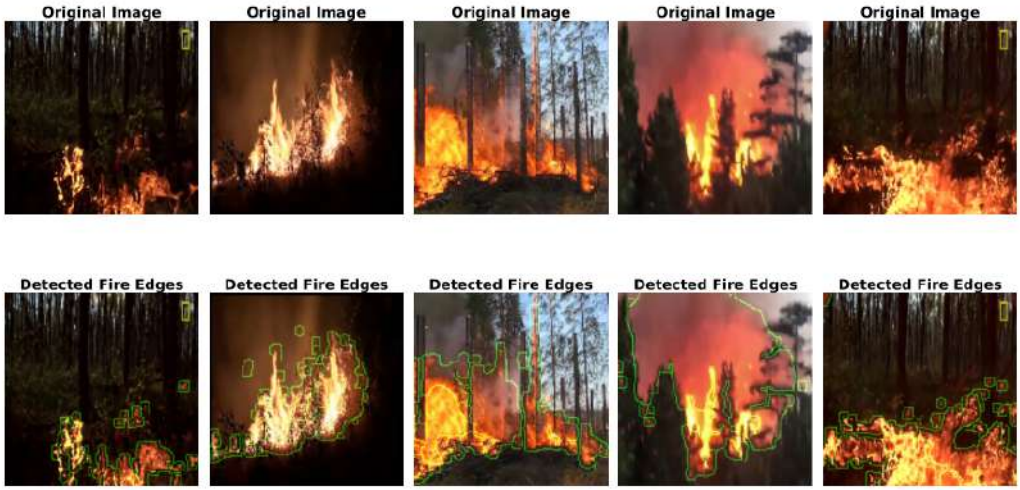


FIGURE 7. Fire segmentation of Dataset B, highlighting the fire edges.

each step, allowing the model to iteratively reconstruct a high-quality image. The following equation presents the iterative de-noising process:

$$\hat{x}_0 = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \hat{\epsilon}_\theta(x_t, t) \right). \tag{6}$$

Here, $\hat{x}_0$ represents the estimated clean image at timestep t, while x_t denotes the noisy image at the same timestep. The parameter α_t controls the noise schedule, determining the magnitude of noise added at each step, and $\bar{\alpha}_t$ is the cumulative product of the noise schedule up to timestep t. Lastly, $\hat{\epsilon}_\theta(x_t, t)$ refers to the noise predicted by the U-Net model, which is subtracted to iteratively refine the image during the de-noising process.

The noise schedule, defined as a series of α_t values, is critical to the generation process. For this study, we used a cosine noise schedule, which gradually reduces the noise over the diffusion steps, facilitating smooth transitions in image quality. The total number of diffusion steps was set to 1,000, balancing computational efficiency and image fidelity.

The Stable Diffusion model also integrates the Contrastive Language-Image Pretraining (CLIP) model to guide image generation. CLIP ensures that the generated images accurately reflect the provided textual descriptions by maximizing the similarity between text and image embeddings. The full pipeline involves the following steps:

1) **Text Encoding:** The textual prompt (e.g., "a fire in a forest" or "a forest without fire") is converted into an embedding using CLIP.
2) **Noise Initialization:** A Gaussian noise image is initialized as the starting point.
3) **Denoising Process:** The U-Net model iteratively removes noise in a stepwise manner using a trained diffusion schedule.
4) **Image Refinement:** The model reconstructs fine-grained details to align the generated image with the text prompt.
5) **Output Image Selection:** The final high-quality image is extracted and stored.

For our experiments, we generated 1,020 fire-related and 1,181 non-fire-related images using descriptive prompts. The CLIP similarity scores for these generated images ranged from 35.56 to 45.23, confirming strong alignment between the textual descriptions and visuals. The synthetic images were then integrated with the original dataset to enhance training diversity.

TABLE 2. Parameter Settings for the stable diffusion model.

Parameter	Value
Number of diffusion steps	1,000
Noise schedule	Cosine
U-Net depth	5 levels with skip connections
Optimizer	AdamW with a learning rate of 10^{-4}
Batch size	64
CLIP model	ViT-B/32 for text-to-image conditioning

Additionally, traditional data augmentation techniques were applied on the training dataset to enhance the data diversity. Methods include random rotation, flipping, scaling, and brightness adjustments introduced variations in the images, mitigating overfitting and ensuring the model's robustness to a wider range of possible fire scenarios. Combining both traditional and synthetic data augmentation, a comprehensive dataset was developed, enhancing the model's generalization capabilities.

1) ADVANTAGES OVER GANs

Unlike Generative Adversarial Networks (GANs), which often suffer from training instability and mode collapse, Stable Diffusion provides a stable and reliable framework for generating high-quality images. Its de-noising approach is less sensitive to hyperparameter tuning, making it more accessible for large-scale data augmentation. Figures 8 and 9 illustrate the Stable Diffusion model architecture and sample synthetic images.

This enhanced dataset significantly improved the performance of FireNet-CNN, highlighting the effectiveness of our data augmentation strategy.

C. DATASET SPLITTING

To effectively train and validate the proposed forest fire detection model, we utilized two distinct datasets: Dataset A and Dataset B, each comprising both fire and non-fire images.

Dataset A originally consisted of 2,974 images, comprising 1,672 fire images and 1,275 non-fire images. However, due its limited size, the deep learning model struggled to perform effectively. To overcome this limitation, we utilized a Stable Diffusion model to generate an additional 1,020 fire images and 1,181 non-fire images. These images were created using text prompts that described scenes with fire in a forest and a forest without any fire instances. These augmentations expanded Dataset A to include a total of 2,456 fire images and 2,692 non-fire images.

Similarly, Dataset B, which initially contained 1,111 fire images and 2,289 non-fire images collected from 16 and 9 videos, respectively, was increased in the same method. By generating 1,020 fire images and 1,181 non-fire images through the Stable Diffusion model, we expanded Dataset B to a total of 3,585 fire images and 2,124 non-fire images. Once the datasets were augmented, both Dataset A and Dataset B were subjected to 5-fold cross-validation.

5-fold cross-validation was employed to validate the efficacy and generalization capacity of the model. This technique divides the dataset D into five equal subsets, or "folds," ensuring that each data sample is utilized for both training and validation. In four of the folds, the model is trained in each iteration i:

$$D_{\text{train}} = D \setminus D_i. \tag{7}$$

and evaluated on the remaining fold:

$$D_{\text{val}} = D_i. \tag{8}$$

For each fold, the model's performance is validated by generating the validation loss $L_{\text{val}}(M, D_{\text{val}})$, where M represents the model and D_{val} is the validation data. During the training process, the loss function is minimized:

$$L(M, D_{\text{train}}) \rightarrow \min. \tag{9}$$

The model was assessed on a validation set after training with performance metrics like accuracy used to quantify its effectiveness. The final evaluation metric represented the average scores across all folds, ensuring that the model was tested on varied data subsets to yield a reliable estimate of overall performance, thereby reducing overfitting risks. If accuracy is the chosen metric, it can be computed as:

$$\text{Accuracy}_{\text{final}} = \frac{1}{5}\sum_{i=1}^{5} \text{Accuracy}(M, D_i). \tag{10}$$

Data augmentation techniques such as, zooming, rotation, shifting, shearing, and horizontal flipping, was used to diversify the training data improving generalization. Implementing

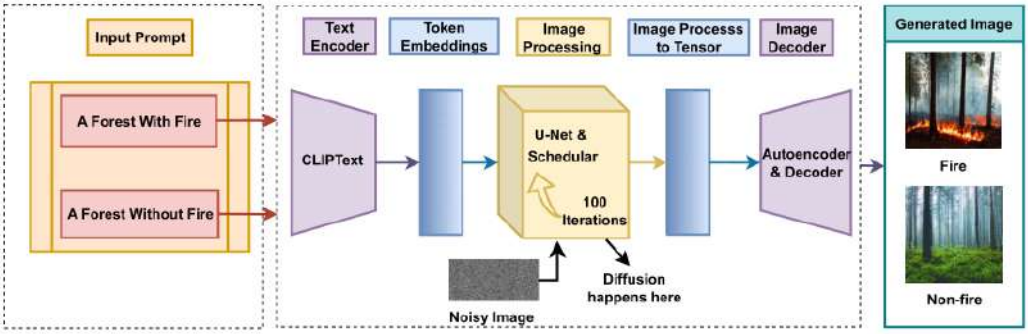


FIGURE 8. Architecture of the Stable Diffusion model. The process involves text encoding via CLIP, noise initialization, iterative denoising using a U-Net-based transformer, and final image reconstruction. The model progressively removes noise over multiple steps to generate realistic images.

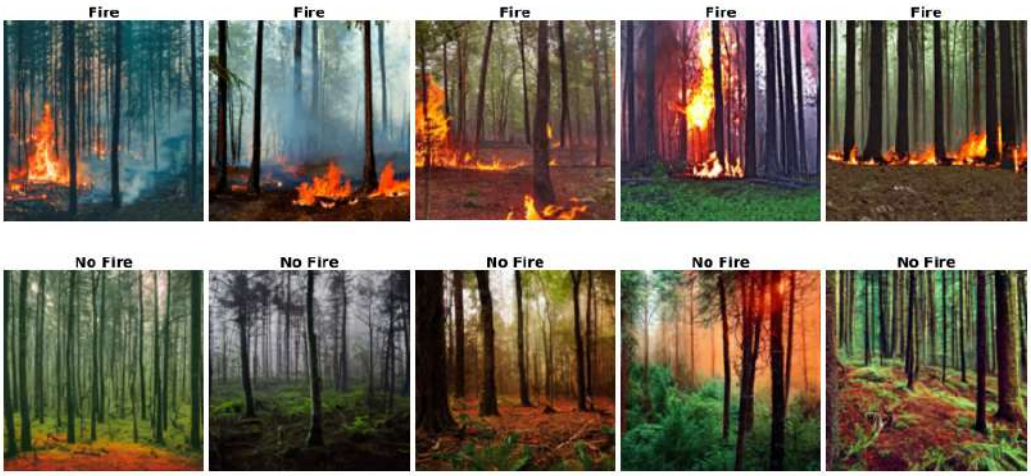


FIGURE 9. Sample images generated using the Stable Diffusion model, depicting various fire and non-fire scenarios. These images were used to augment the training dataset for improved model performance.

5-fold cross-validation enabled robust model evaluation across diverse data distributions.

D. MODEL BUILDING

Deep learning, a branch of machine learning, utilizes neural networks with multiple layers to capture intricate data patterns [77]. In this research, we designed a specialized Convolutional Neural Network (CNN) called FireNet-CNN for detecting forest fires. This model effectively identifies fire patterns in images while balancing between performance efficiency and computational costs. We also compared FireNet-CNN with three widely used pre-trained models: Inception V3 [78], VGG16, and VGG19 [79]. CNN architectures are renowned for their strong feature extraction capabilities, and have been successfully applied to numerous image recognition tasks. InceptionV3, a deep CNN model developed by Google, applies the inception modules that allow the network to perform multiple convolutions in parallel at each layer. This design captures features at various scales while optimizing resource usage. Pre-trained on the ImageNet dataset, InceptionV3 demonstrates exceptional performance in transfer learning for similar classification tasks. VGG16 and VGG19 are deep CNN architectures from the Visual Geometry Group at the University of Oxford. VGG16 has 16 layers (13 convolutional and 3 fully connected), while VGG19 extends this to 19 layers. These models are known for using small 3×3 filters across all convolutional layers, making them computationally expensive but highly effective in capturing fine-grained features.

They are also pre-trained on ImageNet and widely used for transfer learning in various image classification tasks. These pre-trained models were fine-tuned on our fire detection dataset for a comparison with our proposed FireNet-CNN.

1) PROPOSED MODEL: FIRENET-CNN

The FireNet-CNN model is composed of five convolutional layers followed by max-pooling blocks, each designed to progressively extract more complex features. Batch normalization is applied after each block to improve model stability and accelerate the training process. Figure 10 illustrates the architecture of the model, showing how the input image flows through each of these blocks. The final output is a classification decision made by the fully connected layers. The detailed description of the model architecture is stated below:

- **Input Layer**
  The model input consists of images with dimensions 150×150 pixels and three color channels (RGB), represented as:
  $$I_{\text{input}} \in \mathbb{R}^{150\times150\times3} \tag{11}$$
  After augmentation, Dataset A contains 5,148 images (2,456 fire and 2,692 non-fire images), while Dataset B contains 5,709 images (3,585 fire and 2,124 non-fire images).
- **Convolutional and Max-Pooling Blocks**
  The model employs five convolutional blocks designed to progressively extract more complex features from the input images. Each block consists of three main operations: convolution, max-pooling, and batch normalization. These applications assist in detecting patterns, such as the presence of fire in the input images.
  - **Convolution**: The first step in each block is applying a convolutional filter to the input image or feature map from the previous layer. This process is used to extract features like edges, textures, and patterns relevant for fire detection. Each convolutional layer uses a 3×3 kernel and is followed by a ReLU activation function:
    $$F = \text{ReLU}(\text{Conv2D}(I, W) + b) \tag{12}$$
    where I is the input image (or feature map from the previous layer), W indicates the convolutional weights, and b states the measured bias. The resulting feature map, F, captures important visual features in the image, such as the shape and texture of the fire.
  - **Max-Pooling**: After applying the convolution, max-pooling is used to decrease the spatial dimensions of the feature map while holding the most significant information. This operation selects the maximum value within a specified pool size (2×2) and reduces the dimension of the feature map by a factor of two:
    $$P = \text{MaxPooling2D}(F, \text{pool size} = (2, 2)) \tag{13}$$
    This helps the model focus on the most dominant features, like large fire areas, while reducing the computational load. To improve stability and accelerate the training process, batch normalization is applied to the feature maps after max-pooling. This operation normalizes the feature maps to ensure that the learning process is smoother and less prone to vanishing or exploding gradients.

The five convolutional blocks of the model have the following filter sizes: Block 1 uses 32 filters, Block 2 uses 64 filters, Block 3 uses 128 filters, Block 4 uses 256 filters, and Block 5 uses 512 filters. Each subsequent block increases the number of filters, allowing the model to capture increasingly complex features at different levels of abstraction, which are crucial for accurately detecting forest fires.

To provide deeper insights into the internal workings of the model, we analyzed feature maps from the last convolutional layer. Below, Figure 11 illustrates the input images and their corresponding feature maps for the "fire" class.

The feature maps demonstrate the model's capability to detect high-level features such as fire edges, smoke patterns, and texture differences between fire and non-fire regions. These maps reveal the importance of hierarchical feature extraction in distinguishing between fire and non-fire scenarios. By visualizing these internal representations, we can better understand and validate the decision-making process of FireNet-CNN, ensuring its reliability in real-world applications.

- **Fully Connected Layers**
  After the final convolutional block, the feature maps are flattened and passed through two fully connected layers with ReLU activations for classification. The first dense layer has 512 units with a ReLU activation function, and the second dense layer has 256 units with a ReLU activation function. Dropout is applied after each dense layer with a rate of 0.5 to prevent overfitting.
- **Output Layer**
  The output layer consists of a single neuron with a sigmoid activation function, providing a probability score for the binary classification task. This output layer generates a value between 0 and 1, representing the probability that the input image belongs to the "fire" class. Mathematically, the output O of the neuron is calculated as:
  $$O = \sigma(\text{Dense}(1)). \tag{14}$$
  where $\sigma(z)$ is the sigmoid activation function defined as:
  $$\sigma(z) = \frac{1}{1 + e^{-z}}. \tag{15}$$
  Here, z is the input to the sigmoid function (the output of the dense layer). The sigmoid function squashes the value of z into the range $(0, 1)$. The resulting value O represents the probability that the input image contains

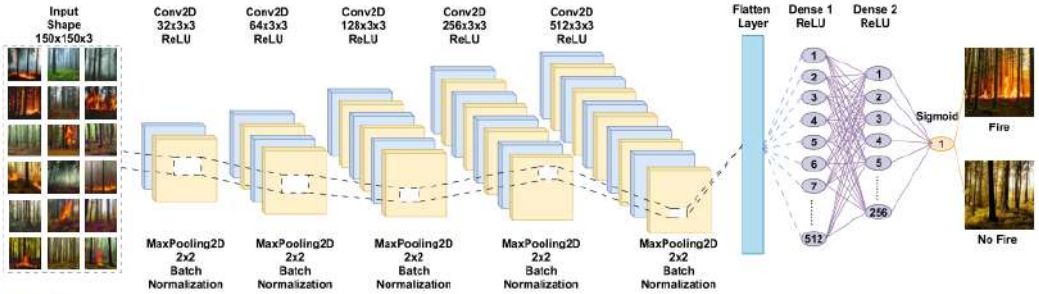
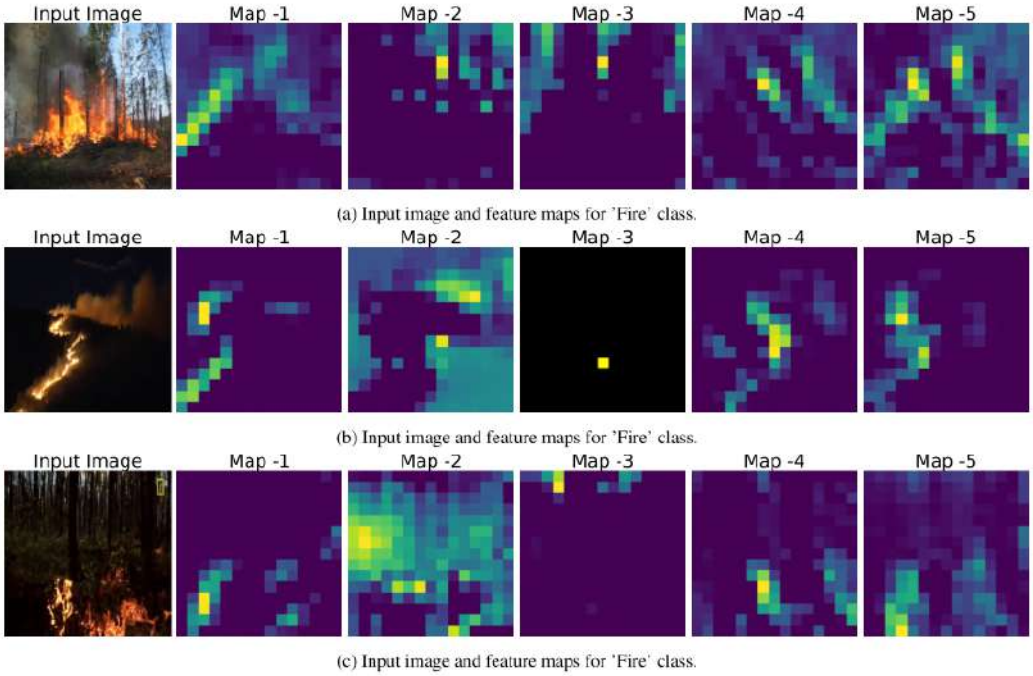


FIGURE 10. Architecture of the FireNet-CNN model. The model consists of five convolutional blocks with increasing filter sizes, followed by fully connected layers for classification.



(a) Input image and feature maps for 'Fire' class.

(b) Input image and feature maps for 'Fire' class.

(c) Input image and feature maps for 'Fire' class.

FIGURE 11. Input images and their corresponding last-layer feature maps from FireNet-CNN.

fire. If O is closer to 1, the model predicts "fire," and if O is closer to 0, the model predicts "no fire."

- **Loss Function and Optimization**
  The proposed FireNet-CNN model has been trained to utilize the binary cross-entropy loss function. The Adam optimizer is used with a learning rate of 0.0001 for optimization. The Adam optimizer combines the benefits of momentum and adaptive learning rates. The effectiveness of FireNet-CNN was assessed using 5-fold cross-validation, which performed splitting of the dataset into five subsets, training on four, and validating on the fifth, rotating through all folds.

2) OPTIMIZED HYPERPARAMETERS OF OUR PROPOSED MODEL

The FireNet-CNN model was optimized with several key hyperparameters to enhance performance for forest fire detection. Initially, the Adam optimizer was used with a

learning rate of 0.001 and then reduced to 0.0001 after observing better convergence ability. ReLU activation was applied to convolutional and fully connected layers, while the output layer used a sigmoid activation for binary classification. Dropout was set to 0.5 after the fully connected layers to prevent overfitting while maintaining the model's expressiveness, and batch normalization was applied after each convolutional and max-pooling block for stability. The model was trained with a batch size of 32 for 50 epochs. We observed the training loss and validation loss curves to assess the convergence behavior, hence with the optimized hyperparameters the model achieved smooth convergence with minimal overfitting. The fine-tuned hyperparameters for optimal performance are detailed in Table 3.

TABLE 3. Hyperparameters of the FireNet-CNN model used for optimization.

Hyperparameter	Value
Learning Rate	0.0001
Optimizer	Adam
Activation Function (Conv Layers)	ReLU
Activation Function (Fully Connected Layers)	ReLU
Output Layer	Sigmoid activation
Batch Size	32
Epochs	50

E. EXPLAINABLE AI

In recent years, demand for models that offer insights into the decision-making process in addition to precise predictions has increased [80]. In this study, we integrated two popular XAI methods, Grad-CAM and Saliency Maps, with our proposed FireNet-CNN model. These methods contribute to forming a visual representation of the model's decision-making process, allowing us to gain insights into the crucial regions of an image contributing to the classification decision. Below, we describe these techniques and their integration with the FireNet-CNN model, along with detailed explanations of their underlying principles and parameter settings.

1) GRAD-CAM: GRADIENT-WEIGHTED CLASS ACTIVATION MAPPING

Grad-CAM is an effective technique that leverages the gradients of the output class based on the feature maps of a convolutional layer to generate a class-discriminative localization map. This technique highlights the important portions in the image that formulate the model's decision.

The Grad-CAM process begins with a forward pass through the CNN to compute feature maps at a specific convolutional layer. Let A^k denote the feature map of the k-th convolutional layer with spatial dimensions $H \times W$. During the backward pass, the gradient of the class score y^c with respect to these feature maps, $\frac{\partial y^c}{\partial A^k}$, is computed to measure how each feature map contributes to the class score.

The spatial gradients are then globally averaged over the H and W dimensions to calculate channel-wise weights:

$$\alpha_k^c = \frac{1}{H \times W} \sum_{i,j} \frac{\partial y^c}{\partial A_{ij}^k}. \tag{16}$$

These weights α_k^c indicate the importance of each feature map channel for the target class c.

The class-discriminative localization map L^c is obtained by a weighted combination of the feature maps:

$$L^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right). \tag{17}$$

The ReLU activation ensures that only positive contributions are considered. The resulting L^c map is resized to match the dimensions of the original input image and superimposed as a heatmap, highlighting regions that strongly influence the classification.

For our implementation, we applied Grad-CAM to the final convolutional layer of FireNet-CNN. The heatmaps produced provided valuable insights into which areas the model focused on when predicting fire or non-fire scenarios, helping to validate its decision-making.

2) SALIENCY MAP

Saliency maps provide an alternative approach for visualizing important regions in an image by calculating pixel-level gradients of the output class score. Unlike Grad-CAM, which focuses on feature maps, saliency maps work directly with the input image.

Given an input image I, the class score y^c is computed during the forward pass. To determine the impact of each pixel, the gradient of the score with respect to the input image is calculated:

$$\frac{\partial y^c}{\partial I}. \tag{18}$$

This gradient indicates how perturbing each pixel in I would affect the class score. The absolute value of these gradients is taken to highlight the most critical regions:

$$S = \left|\frac{\partial y^c}{\partial I}\right|. \tag{19}$$

The saliency map S is normalized and overlaid onto the input image to visualize the areas that have the most significant impact on the model's decision. This pixel-wise interpretation provides a granular view of the model's focus, complementing the broader localization offered by Grad-CAM.

In our study, we applied saliency maps to several fire and non-fire images, revealing fine-grained details about the model's decision-making. The combination of Grad-CAM and saliency maps provided a comprehensive view of FireNet-CNN's interpretability, confirming its reliability in detecting forest fires.

Parameter Settings and Theoretical Considerations: To ensure reproducibility, the following parameter settings

were used: Gradients were computed with respect to the last convolutional layer for Grad-CAM, and input pixels for saliency maps. A learning rate of 10^{-4} in the Adam optimizer with was utilized for backpropagation. For visual clarity, saliency maps were scaled to match the resolution of the original images. These methods offer complementary perspectives on interpretability, providing both class-specific and pixel-specific insights into model behavior.

IV. RESULTS ANALYSIS

A. HARDWARE SETUP AND MODEL TRAINING

The FireNet-CNN model was developed and trained in a Kaggle notebook environment, utilizing an NVIDIA P100 GPU accelerator to improve processing efficiency. This setup facilitated faster training iterations and effectively handled the model's computational demands. Table 4 summarizes the key specifications and performance metrics of FireNet-CNN, highlighting its compact size and rapid inference capabilities. With a model size of 10.58 MB and an inference time of just 0.95 seconds, FireNet-CNN is advantageous for practical fire detection tasks. The table also reports a total of 2,753,217 parameters, which balances model complexity with computational efficiency. The training process, completed in 1013.84 seconds, demonstrates the model's feasibility for deployment in computationally constrained environments.

TABLE 4. Hardware and model training specifications.

Metric	Specification	Description
Model Size	10.58 MB	Size of the trained model file
Training Time	1013.84 seconds	Time taken to train the model
Inference Time	0.95 seconds	Time required for model inference
Number of Parameters	2,753,217	Total count of model parameters

B. EVALUATION METRICS

To evaluate the performance of the proposed FireNet-CNN model, we used several widely accepted evaluation metrics that assess the model's ability to classify fire and non-fire images. These metrics include Accuracy, Precision, Recall, F1 Score, Jaccard Similarity, and Matthews Correlation Coefficient (MCC). Each of these metrics provides a unique insight into the model's performance, and they are defined as follows:

1) ACCURACY

The percentage of accurately identified instances (both fire and non-fire) relative to the total number of cases is known as accuracy. It is measured as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \tag{20}$$

where, TP is the number of true positives (fire correctly classified as fire), TN is the number of true negatives (non-fire correctly classified as non-fire), FP is the number of false positives (non-fire incorrectly classified as fire), and FN is the number of false negatives (fire incorrectly classified as non-fire).

2) PRECISION

The ratio of positive predictions that were actually correct is termed as precision. It is calculated as:

$$\text{Precision} = \frac{TP}{TP + FP}. \tag{21}$$

where, TP is the value of true positives and FP represents false positives.

3) RECALL

The percentage of true positives that the model properly recognized is called recall, also referred to as Sensitivity or True Positive Rate. It is calculated as:

$$\text{Recall} = \frac{TP}{TP + FN}. \tag{22}$$

where, TP is the number of true positives and FN is the number of false negatives.

4) F1 SCORE

The harmonic mean of Precision and Recall is F1-score, providing a balance between the two metrics. It is particularly helpful in cases where data is imbalanced. The F1 Score is measured as:

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{23}$$

5) JACCARD SIMILARITY INDEX

The Jaccard coefficient, sometimes referred to as the Jaccard Similarity Index, calculates how similar the actual and anticipated positive classifications are. It is defined as:

$$\text{Jaccard Similarity} = \frac{TP}{TP + FP + FN}. \tag{24}$$

This metric is useful for evaluating the overlap between the predicted and true positive classes, particularly when the dataset has imbalanced classes.

6) MATTHEWS CORRELATION COEFFICIENT (MCC)

The Matthews Correlation Coefficient (MCC) is a measure of the quality of binary classifications. It considers all four confusion matrix categories (TP, TN, FP, FN) and is generally regarded as a balanced metric. The MCC is calculated as:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \tag{25}$$

The MCC provides a value between -1 and 1, where 1 indicates a perfect prediction, 0 indicates no better than random guessing, and -1 indicates a completely wrong prediction.

These metrics were computed during the evaluation of the FireNet-CNN model to assess its performance in detecting forest fires from images. Each metric provided valuable insights into the model's strengths and areas for improvement.

C. RESULT ANALYSIS WITH OUR PROPOSED MODEL

1) DATASET A RESULTS

For Dataset A, the FireNet-CNN model achieved an average training loss of 0.052 and an average validation loss of 0.071. The average training accuracy was 99.30%, while the average validation accuracy was 97.96%. These results demonstrate the model's ability to generalize well across the dataset. In terms of classification performance, the model recorded an average precision of 98.22%, an average recall of 98.41%, and an average F1 score of 98.31%. Figure 12 illustrates the training and validation accuracy for each fold, while Figure 14 shows the corresponding loss curves. The aggregated accuracy and loss curves across all folds are presented in Figure 13 and Figure 15, respectively. These figures provide insights into the stability and convergence of the model during training and validation.

The model's confusion matrices for each fold are shown in Figure 17, and the overall confusion matrix is displayed in Figure 17. Figure 18 depicts the overall ROC curve for Dataset A, demonstrating the model's high discriminative ability. The curve illustrates the model's performance in distinguishing between the two classes. The average area under the curve (AUC) across all folds is 0.9980, indicating excellent performance in terms of classification accuracy and class separation.

2) DATASET B RESULTS

For Dataset B, the model achieved even better performance, with an average training loss of 0.034 and an average validation loss of 0.056. The average training accuracy was 99.07%, and the average validation accuracy was 99.05%. These results indicate a high degree of accuracy and low generalization error.

The classification metrics for Dataset B were also impressive, with an average precision of 99.41%, an average recall of 98.28%, and an average F1 score of 98.84%. These values showcases the model's efficiency and reliability in detecting fire presence accurately.

Figure 19 shows the training and validation accuracy for each fold, while Figure 21 displays the aggregate training and validation accuracy curves. The aggregated loss curve and loss curves across all folds are presented in Figure 22 and Figure 20. These figures emphasize the model's consistency across multiple folds.

Figure 23 presents the confusion matrices for each fold, while Figure 24 shows the overall confusion matrix. The ROC curve in Figure 25 further underscores the model's ability to distinguish between fire and non-fire instances.

3) ABLATION STUDY

To evaluate the contributions of key components in the FireNet-CNN model, an ablation study was conducted by modifying several architectural elements. Removing convolutional layers and using a simple fully connected network caused a significant performance drop, from 99.30% to 87.25% accuracy, emphasizing the importance of convolutional layers for feature extraction. Replacing ReLU with LeakyReLU or Sigmoid resulted in slight accuracy drops (98.15% and 94.20%, respectively), highlighting ReLU's superiority. Removing the dropout layer increased training accuracy (99.50%) but decreased validation accuracy (95.40%), underlining its role in preventing overfitting. Without batch normalization, accuracy dropped slightly (98.89% training, 97.02% validation), suggesting its importance in faster convergence and stability. Adam optimizer outperformed both SGD and RMSProp, yielding the highest performance (99.30% training, 97.96% validation). Replacing Binary Cross-Entropy with Mean Squared Error significantly reduced performance (93.75% training, 90.25% validation), reinforcing the suitability of Binary Cross-Entropy for classification tasks. Table 5 summarizes the results, demonstrating that the original FireNet-CNN configuration achieves the best performance.

D. RESULT ANALYSIS WITH INTEGRATION OF XAI

To evaluate the performance and interpretability of our proposed FireNet CNN model, we incorporated explainable AI (XAI) techniques such as Grad-CAM and Saliency Maps to analyze the model's decision-making process. By applying these methods to images from the validation set, we aimed to understand which regions within the images the model identified as critical for fire detection.

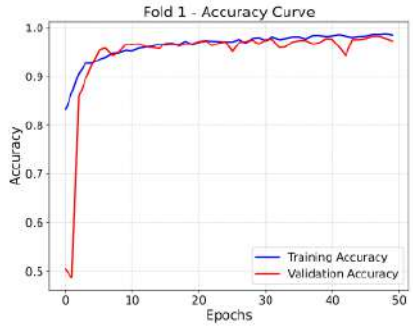
1) GRAD-CAM ANALYSIS

Grad-CAM was employed to generate heatmaps that localize the regions contributing most to the model's decision. Specifically, we used the last convolutional layer of our proposed FireNet-CNN model to generate these heatmaps, as this layer contains the most important and highest amount of features. The resulting heatmaps provide a clear visualization of the model's focus areas, helping to interpret its decision-making process.

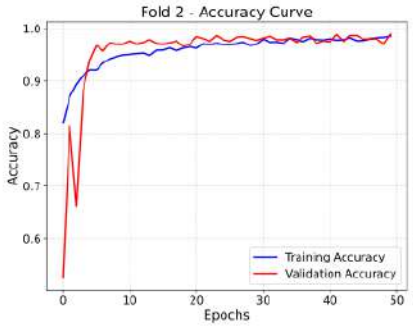
As shown in Figure 26a and Figure 26b, the Grad-CAM visualizations effectively highlight fire regions in the corresponding images. These heatmaps demonstrate that the model focuses on the correct areas, such as flames and smoke, to distinguish between fire and non-fire images. This capability enhances the model's reliability, as it aligns with human understanding of fire detection.

2) SALIENCY MAP ANALYSIS

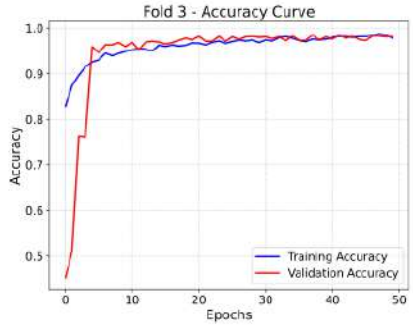
To complement Grad-CAM, Saliency Maps were generated to provide a pixel-level understanding of the model's decision. Figure 27a and Figure 27b illustrate the saliency



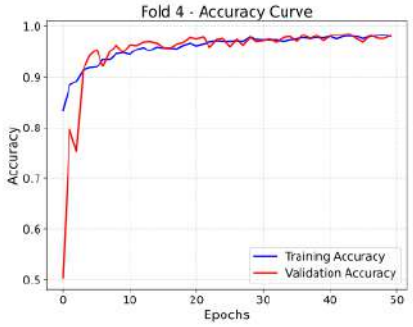
(a) Training and validation accuracy for fold 1.



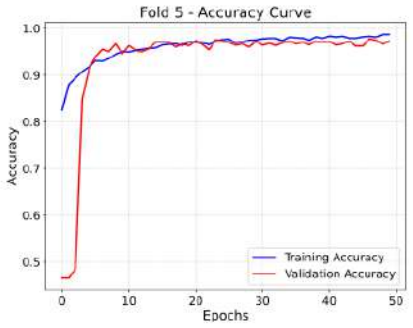
(b) Training and validation accuracy for fold 2.



(c) Training and validation accuracy for fold 3.



(d) Training and validation accuracy for fold 4.



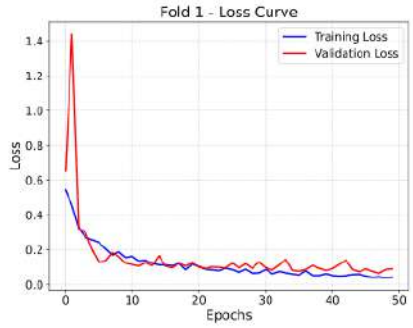
(e) Training and validation accuracy for fold 5.

FIGURE 12. Training and validation accuracy curves for each of the five folds during 5-fold cross-validation on Dataset A. Individual subplot illustrates the progression of accuracy across epochs.

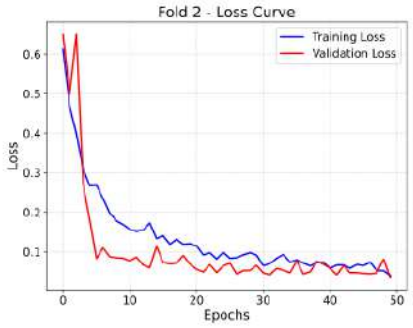
maps corresponding to two validation images. These maps highlight the most influential pixels in the image, focusing on areas with flames or smoke. Saliency Maps provide finer-grained interpretability compared to Grad-CAM, making them particularly useful for understanding subtle features critical for fire detection.

3) INSIGHTS AND VALIDATION

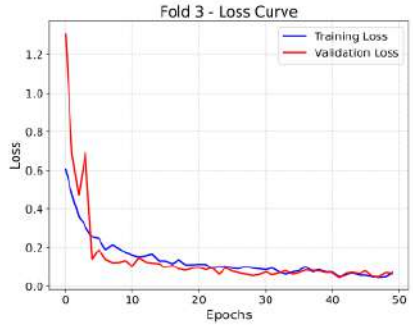
The visualizations generated through Grad-CAM and Saliency Maps reveal that the FireNet-CNN model accurately identifies critical regions associated with fire. Both methods confirm that the model's outcomes depend on relevant features, such as flames and smoke, ensuring robust and



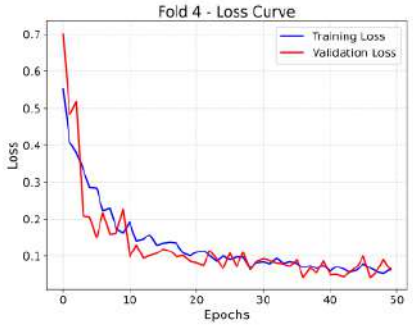
(a) Training and validation loss for fold 1.



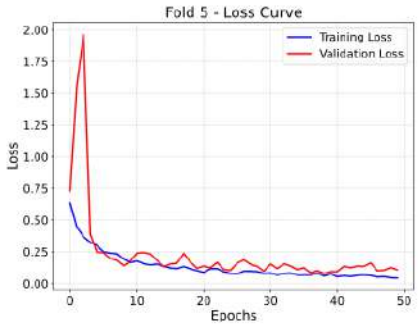
(b) Training and validation loss for fold 2.



(c) Training and validation loss for fold 3.



(d) Training and validation loss for fold 4.



(e) Training and validation loss for fold 5.

FIGURE 13. During 5-fold cross-validation on Dataset A the training and validation loss curves for each of the five folds are presented. Individual subplot demonstrates the progression of the loss function across epochs.

interpretable decision-making. The integration of these XAI techniques validates the reliability of the proposed model and provides confidence for real-world deployment in fire detection systems.

E. PERFORMANCE EVALUATION OF OUR PROPOSED MODEL WITH OTHER PRE-TRAINED MODELS

We evaluated the performance of our FireNet-CNN model against three pre-trained models: VGG16, VGG19, and

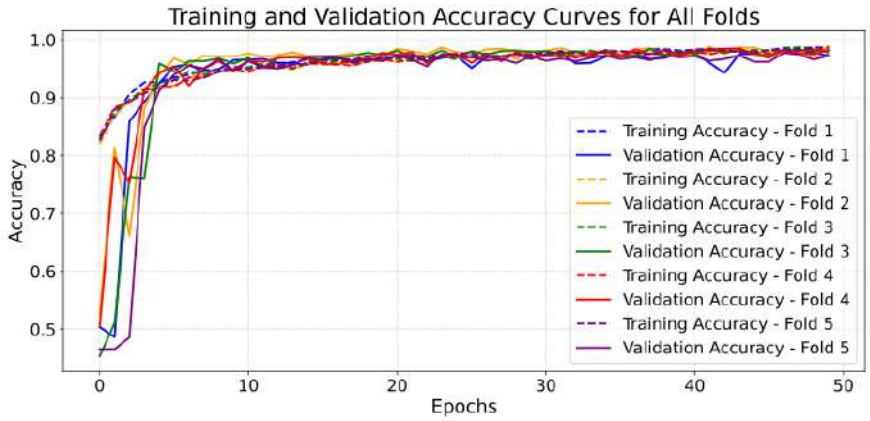


FIGURE 14. Training and validation accuracy curves for all folds combined during 5-fold cross-validation on Dataset A. This curve represents the averaged performance of the model across all folds, showing consistent training and generalization.

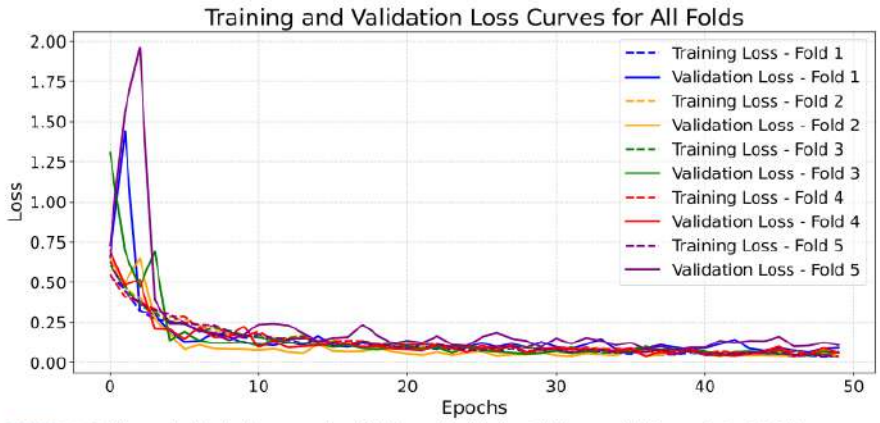


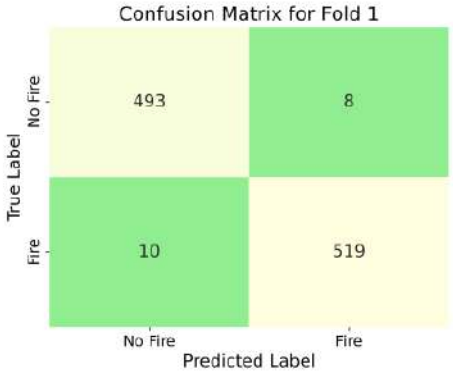
FIGURE 15. Training and validation loss curves for all folds combined during 5-fold cross-validation on Dataset A. This curve highlights the model's learning process across epochs, indicating steady convergence.

TABLE 5. Ablation study results for FireNet-CNN.

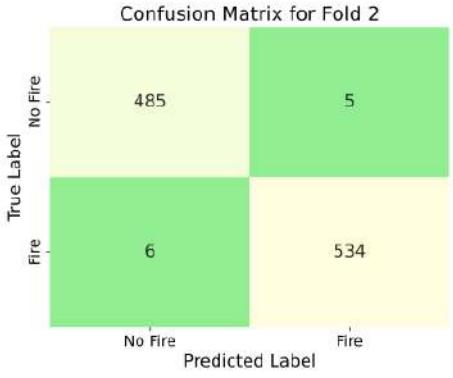
Model Variant	Training Accuracy (%)	Validation Accuracy (%)	Precision (%)	F1 Score (%)
Original Model (FireNet-CNN)	99.07	99.05	99.41	98.84
No Convolutional Layers	87.25	79.60	82.12	80.50
LeakyReLU Activation	98.15	96.20	97.25	96.60
Sigmoid Activation	94.20	91.30	92.50	91.85
No Dropout	99.50	95.40	97.00	96.80
No Batch Normalization	98.89	97.02	98.12	98.22
SGD Optimizer	97.20	95.90	96.85	96.10
RMSProp Optimizer	98.05	96.89	97.20	97.10
MSE Loss Function	93.75	90.25	91.50	91.20

Inception V3. The validation was performed on both Dataset A and Dataset B, using parameters such as accuracy, precision, recall, F1 score, Jaccard Similarity, and Matthews Correlation Coefficient (MCC). The comparative results are presented in Table 6.

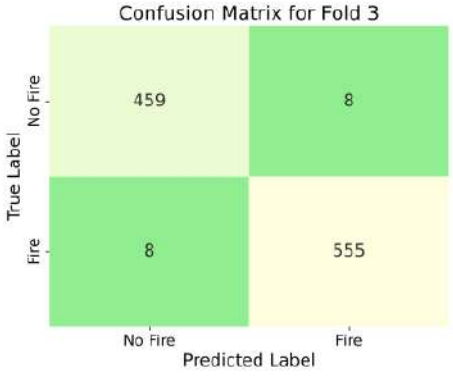
Dataset A Results: FireNet-CNN achieved the best performance on Dataset A, with a validation accuracy of 97.96%, precision of 98.22%, and an F1 score of 98.31%. It also recorded the highest Jaccard Similarity (96.68%) and MCC (96.45%), surpassing the pre-trained models. Inception



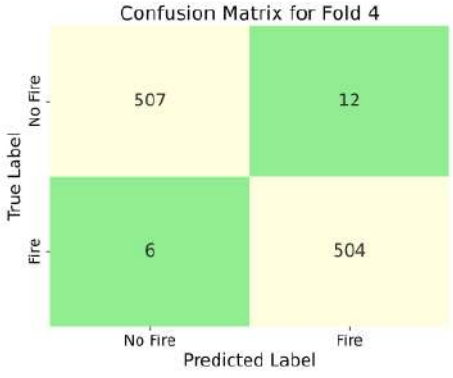
(a) Confusion matrix for fold 1.



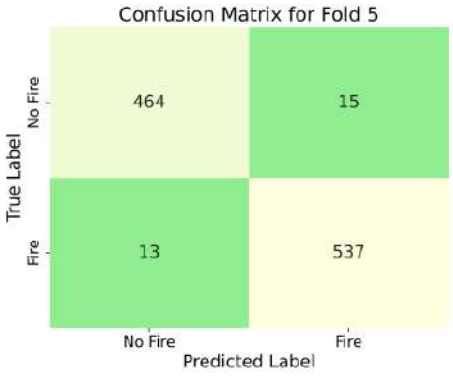
(b) Confusion matrix for fold 2.



(c) Confusion matrix for fold 3.



(d) Confusion matrix for fold 4.



(e) Confusion matrix for fold 5.

FIGURE 16. Confusion matrices for each of the five folds during 5-fold cross-validation on Dataset A. Individual plot shows the model's results based on true positives, false positives, true negatives, and false negatives across the respective fold. These matrices illustrate the model's classification accuracy for each fold.

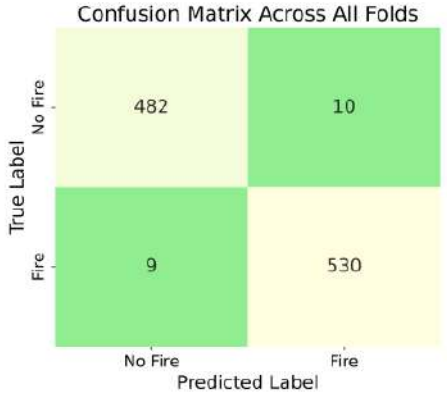


FIGURE 17. The average confusion matrix for Dataset A, representing the cumulative performance of the model across all folds during 5-fold cross-validation. The matrix displays true positives, false positives, true negatives, and false negatives, providing a comprehensive overview of the model's classification accuracy.

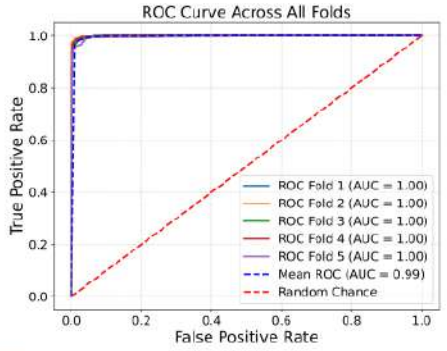


FIGURE 18. The overall ROC curve for Dataset A, demonstrating the model's high discriminative ability across all folds during 5-fold cross-validation. The curve highlights the model's efficacy in differentiating between classes, with a higher area under the curve (AUC) indicating better performance. Average AUC across all folds: 0.9980.

V3 followed with a validation accuracy of 97.53%, but its performance across other metrics remained below that of FireNet-CNN. VGG16 and VGG19 showed relatively lower accuracies of 95.30% and 95.24%, respectively.

Dataset B Results: On Dataset B, FireNet-CNN exhibited even stronger results, achieving a validation accuracy of 99.05%, precision of 99.41%, and an F1 score of 98.84%. It outperformed the pre-trained models in all metrics, including Jaccard Similarity (97.71%) and MCC (98.19%). While Inception V3 achieved competitive results with a validation accuracy of 98.88% and an F1 score of 98.75%, it still fell short of FireNet-CNN. VGG16 and VGG19 again lagged, with lower performance metrics.

The superior performance of FireNet-CNN is attributable to its tailored architecture, optimized for fire detection tasks. Unlike general-purpose pre-trained models, FireNet-CNN excels in capturing fire-specific features, ensuring robust performance and interpretability. These qualities make it a reliable tool for real-world fire detection applications.

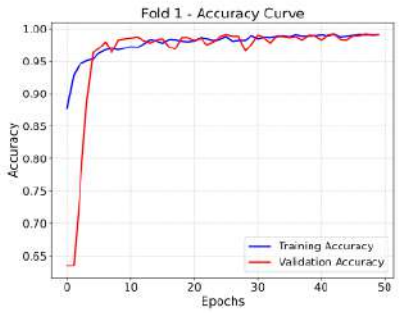
TABLE 6. Performance metrics comparison of FireNet-CNN and pre-trained models on Dataset A and Dataset B.

Model	Dataset A (%)	Dataset B (%)
Validation Accuracy		
Proposed FireNet-CNN	**97.96**	**99.05**
VGG16	95.30	97.51
VGG19	95.24	97.93
Inception V3	97.53	98.88
Precision		
Proposed FireNet-CNN	**98.22**	**99.41**
VGG16	95.51	96.92
VGG19	95.80	97.28
Inception V3	98.13	98.96
Recall		
Proposed FireNet-CNN	**98.41**	**98.28**
VGG16	95.79	97.04
VGG19	95.64	97.57
Inception V3	97.58	98.54
F1 Score		
Proposed FireNet-CNN	**98.31**	**98.84**
VGG16	95.65	96.98
VGG19	95.72	97.42
Inception V3	97.86	98.75
Jaccard Similarity		
Proposed FireNet-CNN	**96.68**	**97.71**
VGG16	91.67	94.14
VGG19	91.81	94.98
Inception V3	95.80	97.53
MCC		
Proposed FireNet-CNN	**96.45**	**98.19**
VGG16	90.89	95.19
VGG19	91.04	95.88
Inception V3	95.52	98.01

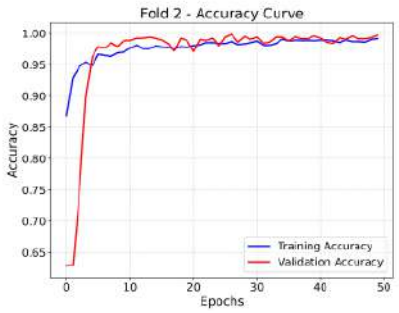
F. COMPUTATIONAL EFFICIENCY

The computational efficiency of a model is critical in real-world applications, particularly in resource-constrained environments. To evaluate the efficiency of our proposed FireNet-CNN model, we compared it against three widely used pre-trained models: VGG16, VGG19, and Inception V3. The comparison includes key metrics such as model size, training time, inference time, number of parameters, and GFLOPS, as summarized in Table 7.

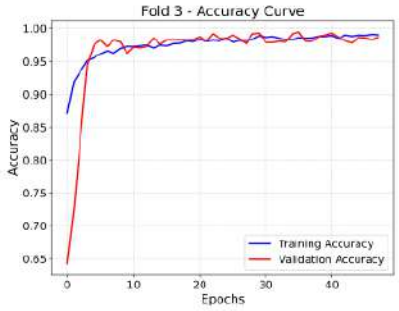
Model Size: FireNet-CNN has a significantly smaller model size of 10.58 MB compared to VGG16 (72.73 MB), VGG19 (93.00 MB), and Inception V3 (120.59 MB). This compact size makes our model ideal for deployment in



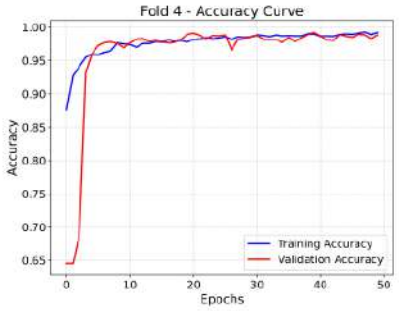
(a) Training and validation accuracy for fold 1.



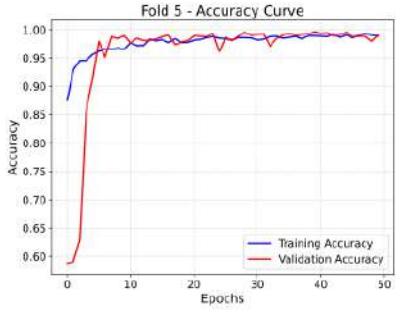
(b) Training and validation accuracy for fold 2.



(c) Training and validation accuracy for fold 3.



(d) Training and validation accuracy for fold 4.



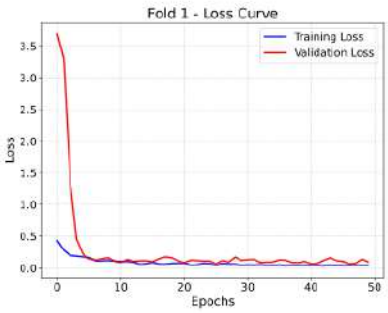
(e) Training and validation accuracy for fold 5.

FIGURE 19. Training and validation accuracy curves for each of the five folds during 5-fold cross-validation on Dataset B. Each subplot illustrates the progression of accuracy across epochs, demonstrating the model's ability to generalize effectively across different folds.

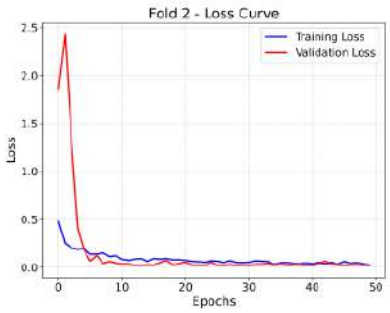
systems with limited storage, such as embedded devices or edge computing platforms.

Training Time: Our model exhibits a training time of 1013.84 seconds, which is faster than both VGG19 (1139.94 seconds) and VGG16 (1129.91 seconds). Although Inception V3 has a slightly faster training time (1104.14 seconds), the difference is marginal. FireNet-CNN's faster training highlights its computational efficiency during model development.

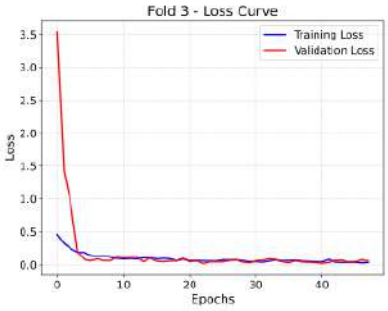
Inference Time: Inference time is crucial for real-time applications. FireNet-CNN has the shortest inference time of 0.95 seconds, significantly outperforming VGG16 (1.98 seconds), VGG19 (2.22 seconds), and Inception V3 (1.82 seconds). This makes FireNet-CNN highly suitable for



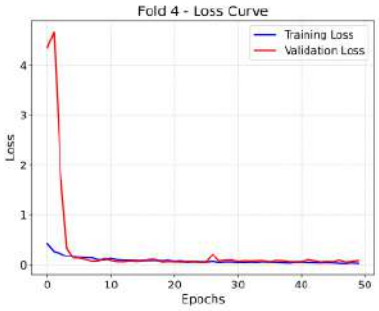
(a) Training and validation loss for fold 1.



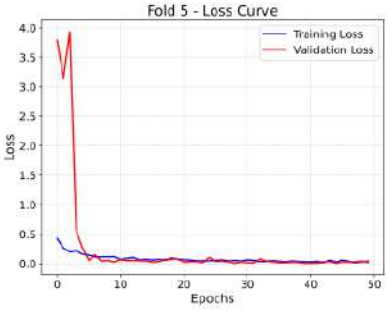
(b) Training and validation loss for fold 2.



(c) Training and validation loss for fold 3.



(d) Training and validation loss for fold 4.



(e) Training and validation loss for fold 5.

FIGURE 20. Training and validation loss curves for each of the five folds during 5-fold cross-validation on Dataset B. Each subplot demonstrates the progression of the loss function across epochs, showing consistent convergence and highlighting the model's learning efficiency across different folds.

applications requiring rapid decision-making, such as fire detection in surveillance systems.

Number of Parameters: FireNet-CNN has only 2,753,217 parameters, a fraction of the parameter counts in VGG16 (19,041,089), VGG19 (24,350,785), and Inception V3 (31,372,065). The reduced parameter count not only contributes to a smaller model size but also minimizes computational load, leading to faster inference and reduced energy consumption.

GFLOPS (Giga Floating Point Operations Per Second): The computational cost of FireNet-CNN is measured at 2.085 GFLOPS, significantly lower than the pre-trained models. VGG16 has a GFLOPS of 15.47, VGG19 has 19.63, and Inception V3 has 5.71. This demonstrates that

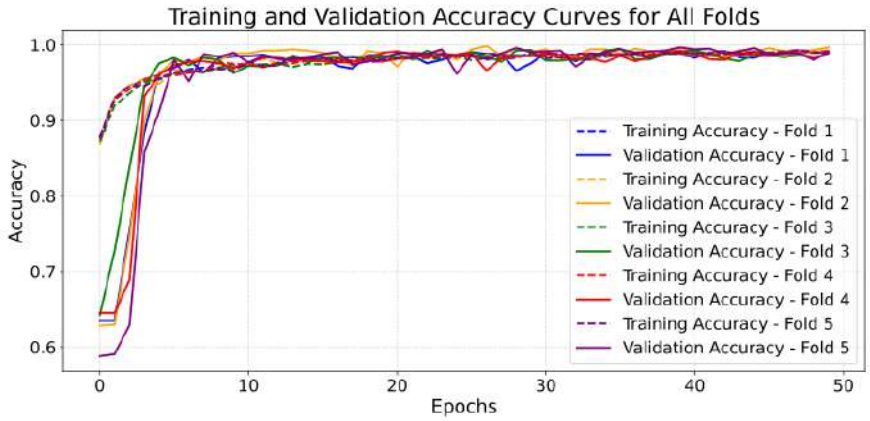


FIGURE 21. Training and validation accuracy curves for all folds combined during 5-fold cross-validation on Dataset B. This curve represents the averaged performance of the model across all folds, showing consistent training and generalization.

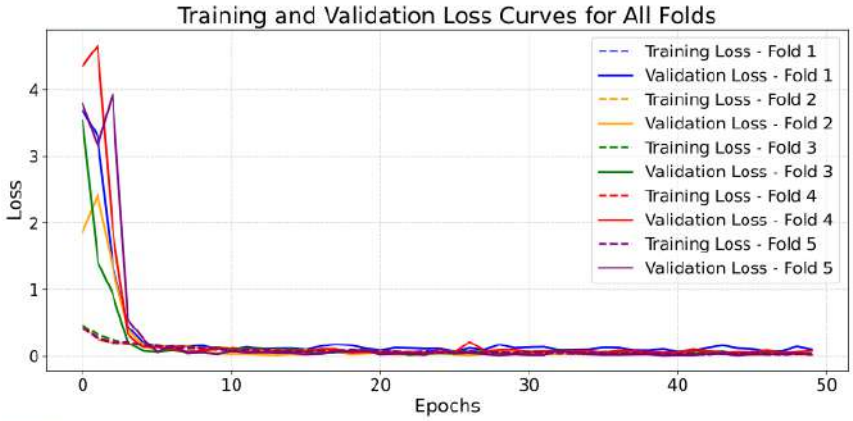


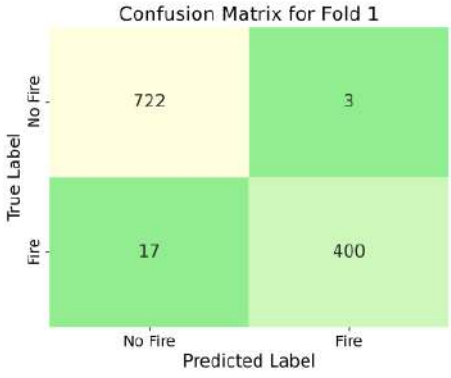
FIGURE 22. Training and validation loss curves for all folds combined during 5-fold cross-validation on Dataset B. This curve highlights the model's learning process across epochs, indicating steady convergence.

FireNet-CNN has a much lower computational cost compared to these models, making it more efficient for resource-constrained environments.

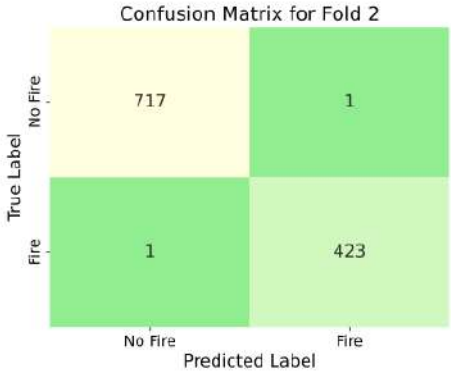
To address computational efficiency and scalability, FireNet-CNN demonstrates a balance between performance and resource usage. Its small model size (10.58 MB) and low computational cost (2.085 GFLOPS) enable efficient deployment on edge devices like cameras and drones, ensuring real-time fire detection with an inference time of just 0.95 seconds. Additionally, the model can scale effectively in cloud environments, processing large datasets without significant overhead. FireNet-CNN can also be parallelized for distributed training on multi-GPU setups, allowing it to handle larger datasets efficiently. This scalability ensures its versatility in both resource-constrained and high-performance environments.

G. COMPARISON WITH PREVIOUS WORK

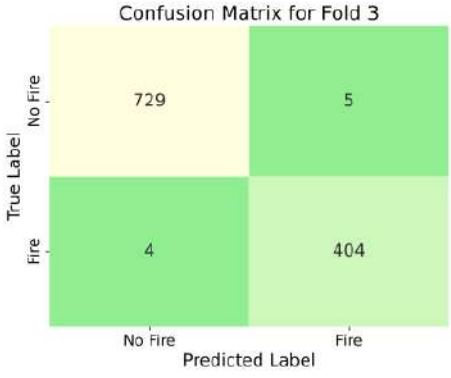
This section compares the obtained outcomes of the proposed FireNet-CNN model with existing models leveraged in fire detection modules in forest environments, as outlined in Table 1. For validating the comparison, accuracy, precision, recall, F1-score, ROC-AUC, number of parameters, GFLOPS, and explainable AI (XAI) techniques are considered, as summarized in Table 8. The FireNet-CNN model exceeds several existing models in terms of precision, recall, F1-score, accuracy, and ROC-AUC. It achieves 99.41% precision and 98.28% recall on both datasets, showcasing its



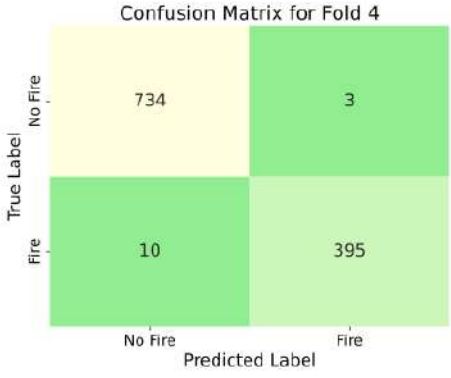
(a) Confusion matrix for fold 1.



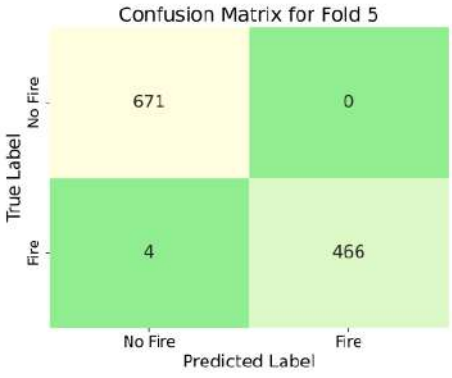
(b) Confusion matrix for fold 2.



(c) Confusion matrix for fold 3.



(d) Confusion matrix for fold 4.



(e) Confusion matrix for fold 5.

FIGURE 23. Confusion matrices for each of the five folds during 5-fold cross-validation on Dataset B. Each subplot demonstrates the values of true positives, false positives, true negatives, and false negatives achieved by the model across the respective fold. These matrices illustrate the model's classification accuracy for each fold.

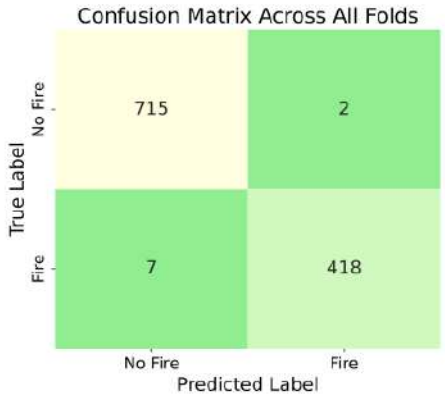


FIGURE 24. The average confusion matrix for Dataset B summarizes the model's overall performance across all folds during 5-fold cross-validation. It presents the counts of true positives, false positives, true negatives, and false negatives, offering a detailed assessment of the model's classification accuracy.

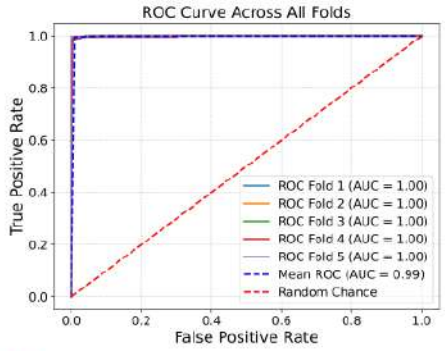


FIGURE 25. The overall ROC curve for Dataset B, demonstrating the model's high discriminative ability across all folds during 5-fold cross-validation. The curve highlights the model's performance in distinguishing between fire and non-fire instances, with a higher area under the curve (AUC) indicating better performance. Average AUC across all folds: 0.9992.

effectiveness in accurately detecting fire events while minimizing false positive rates. Its impressive F1-score of 98.84% and accuracy of 99.05% further highlight its balanced performance, surpassing other models like the Fire-Net model (93% F1-score and 97.35% accuracy) and XGBoost-based models (91.9% F1-score and 91.6% accuracy). Additionally, FireNet-CNN excels in ROC-AUC, with scores of 99.80% for Dataset A and 99.92% for Dataset B, which reflects the model's capacity in differentiating between fire and non-fire classes. In terms of computational efficiency, the comparison among the fire detection baseline models validates that the FireNet-CNN model significantly outperforms existing models. With only 2.085 GFLOPS, it requires

TABLE 7. Computational efficiency metrics of FireNet-CNN and pre-trained models.

Metric	Model	Value
Model Size (MB)	FireNet-CNN	**10.58**
	VGG16	72.73
	VGG19	93.00
	Inception V3	120.59
Training Time (s)	FireNet-CNN	**1013.84**
	VGG16	1129.91
	VGG19	1139.94
	Inception V3	1104.14
Inference Time (s)	FireNet-CNN	**0.95**
	VGG16	1.98
	VGG19	2.22
	Inception V3	1.82
Number of Parameters	FireNet-CNN	**2,753,217**
	VGG16	19,041,089
	VGG19	24,350,785
	Inception V3	31,372,065
GFLOPS	FireNet-CNN	**2.085**
	VGG16	15.47
	VGG19	19.63

far less computational power compared to models like TFNet (53.3 GFLOPs), YOLOV9-CBM (276.5 GFLOPS), and Faster R-CNN (134.38 GFLOPS), making it highly efficient for real-time detection tasks. Despite its lower parameter count (2.75M), FireNet-CNN maintains superior performance, proving its optimized architecture is both lightweight and effective. Unlike many models, FireNet-CNN also integrates explainable AI (XAI) techniques, such as Grad-CAM and Saliency Maps, providing greater model interpretability, which is valuable for practical deployment. These advantages, combined with its computational efficiency, make FireNet-CNN a potentially efficient solution for real-time fire detection systems with reduced hardware constraints.

H. DISCUSSION

The FireNet-CNN model demonstrates significant advancements in fire detection compared to existing models like VGG16, VGG19, Inception V3, and the original Fire-Net. It achieves an impressive accuracy of 99.05%, a precision of 99.41%, and a recall of 98.28% on two augmented datasets, Dataset A and Dataset B, consisting of fire and non-fire images. These results highlight the model's strong capability to accurately detect fires while minimizing false positives, an essential feature for real-time fire detection applications.

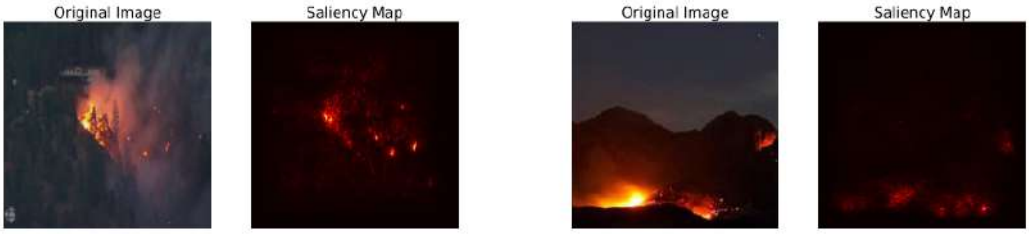
A key factor in FireNet-CNN's high performance is the expansion of training data using Stable Diffusion-based augmentation techniques. Initially, limited datasets were expanded by adding synthetic fire and non-fire images, thus enhancing the model's ability to generalize across different scenarios. To ensure that the synthetic data resembles real-world fire conditions, the generated images



(a) Original image vs. Grad-CAM heatmap for Dataset A. Grad-CAM highlights critical regions influencing the model's classification, generated using the last convolutional layer of FireNet-CNN

(b) Original image vs. Grad-CAM heatmap for dataset B. Grad-CAM highlights critical regions influencing the model's classification, generated using the last convolutional layer of FireNet-CNN

FIGURE 26. Comparison of original image vs. Grad-CAM heatmap for Datasets A and B, showing the critical regions influencing FireNet-CNN's classification.



(a) Original image vs. saliency map for dataset A. The saliency map highlights pixel-level insights into the regions that most influenced the model's decision

(b) Original image vs. saliency map for dataset B. The saliency map highlights pixel-level insights into the regions that most influenced the model's decision

FIGURE 27. Comparison of original image vs. saliency maps for Datasets A and B, showing pixel-level regions that influenced FireNet-CNN's classification.

incorporate diverse fire intensities, varying backgrounds, and lighting variations. However, synthetic images may not always capture the complexities of real-world fires, such as varying smoke density, environmental factors, or obstruction like trees and buildings. This limitation can affect the model's ability to generalize to unpredictable fire events. To mitigate these issues, the proposed model was fine-tuned using real-world fire datasets, improving its adaptability and ensuring it performs well in diverse conditions.

The model's architecture is specifically designed to optimize the detection of fire-related textures, colors, and patterns, enhancing recall rates, with Dataset A achieving a recall of 98.41%. Furthermore, the integration of explainable AI (XAI) techniques, such as Grad-CAM and Saliency Maps, increases transparency, allowing users to interpret model predictions more effectively. This feature is particularly valuable in safety-critical applications, where understanding the rationale behind a model's decisions is essential for informed action.

Beyond its accuracy, FireNet-CNN is also optimized for efficiency. With a compact size of 10.58 MB and an inference time of just 0.95 seconds per image, it is well-suited for deployment in resource-constrained environments, such as drones or remote sensors in wildfire-prone regions. These attributes position FireNet-CNN as a practical solution for real-time fire detection.

Despite its strengths, challenges remain. Beyond the reliance on synthetic data, the datasets used in this study suffer from class imbalance, potentially biasing the model toward dominant classes. Addressing this issue through advanced techniques such as oversampling, undersampling, or custom loss functions is essential for further improving model sensitivity and robustness. Additionally, while FireNet-CNN has been rigorously tested on controlled datasets, real-world validation in diverse and dynamic environments—encompassing different forest types, seasonal variations, and lighting conditions—is crucial to confirm its adaptability and reliability.

Despite these limitations, FireNet-CNN represents a significant step forward in wildfire detection technology. Its combination of high performance, efficiency, and interpretability makes it a strong candidate for widespread deployment in dynamic, resource-limited environments. As wildfire detection remains a critical global challenge, FireNet-CNN offers a promising solution with the potential to mitigate disaster impacts through timely and reliable detection.

TABLE 8. Comparison of performance metrics and use of XAI in different models.

Reference	Model Used	Precision	Recall	F1-Score	Accuracy	ROC-AUC	Num. of Parameters	GFLOPS	Use of XAI
Liu, Hongying, et al. (2025)	TFNet: Transformer-Based Multi-Scale Feature Fusion Network	86.6%	83.3%	84.9%	85%	-	154,000	533	No
Khan et al. (2025)	Xception model with Adaptive Spatial Attention (ASA)	84.11%	86.45%	85.27%	85%	-	-	11	Yes (Grad-CAM)
Geng et al. (2025)	YOLOV9-CBM	87.2%	78.3%	-	87.1%	-	543,000	276.5	No
El-Madafri et al. (2025)	Hierarchical Domain-Adaptive Learning	96.1%	96.3%	96.21%	95.36%	99.17%	4,052,133	-	No
Gao et al. (2024)	YOLOv8 with SimAM attention mechanism	87.1%	85.3%	89%	-	-	27,300,00	25.6	No
Titu et al. (2024)	Knowledge distillation (YOLOv8n)	98.99%	98%	98%	95.21%	-	3,010,000	8.2	No
El-Madafri et al. (2024)	DenseNet201, MobileNetV3Small	90.11%	93.20%	91.57%	93.36%	96.15%	9,400,274	4.29	Yes (Grad-CAM)
Yandouzi et al. (2023)	Faster R-CNN	90.61%	90.83%	-	-	-	19,000,000	134.38	No
Jin et al. (2024)	SWVR model with Vision Transformers and CNN	80.5%	71.2%	-	79.6%	-	8,060,000	16.5	No
Ihan and Akou (2024)	ML framework with Random Forest, XGBoost, LightGBM	96.1%	-	95.6%	95.6%	96.6%	-	11.41	Yes (SHAP)
Samuel et al. (2024)	XGBoost	91.0%	92.2%	91.9%	91.6%	91.6%	≈ 100, 000	-	Yes (SHAP, Grad-CAM)
Hang et al. (2024)	Stacking Models (AdaBoost, XGBoost, RF) with DNN	87%	88.6%	-	87%	94%	≈ 2, 000, 000	-	Yes (SHAP, LIME)
Seydi et al. (2022)	Fire-Net (multiscale convolutional blocks)	93.49%	92.6%	93%	97.35%	-	5,300,000	10.6	No
Proposed Method	**Proposed FireNet-CNN**	**Dataset A: 98.22%**	**Dataset A: 98.41%**	**Dataset A: 98.31%**	**Dataset A: 97.96%**	**Dataset A: 99.80%**	**2,753,217**	**2.085**	**Yes (Grad-CAM, Saliency Map)**
		Dataset B: 99.41%	**Dataset B: 98.28%**	**Dataset B: 98.84%**	**Dataset B: 99.05%**	**Dataset B: 99.92%**	-	-	-

V. CONCLUSION AND FUTURE WORK

This study explored deep learning architectures and introduced FireNet-CNN for forest fire detection that outperforms existing methods in precision, recall, accuracy, and interpretability. With 99.41% precision, 98.28% recall, and 99.05% accuracy, FireNet-CNN offers a reliable solution that minimizes false positives, which is crucial for timely wildfire detection. The integration of explainable AI (XAI) techniques, such as Grad-CAM and Saliency Maps, enhances transparency, making it suitable for real-world applications. FireNet-CNN also addresses dataset limitations by using synthetic data generated through Stable Diffusion, improving generalization and handling complex fire scenarios. The model's lightweight architecture (2.75 million parameters, 10.58 MB) and fast inference time (0.95 seconds per image) make it ideal for use on devices with limited resources, including drones and remote sensors. Compared to baseline models, FireNet-CNN excels in key metrics and offers transparency in decision-making, making it a practical tool for safety-critical systems. These results demonstrate that FireNet-CNN is a practical and reliable solution for early-stage forest fire detection, especially in environments where rapid response is critical for mitigating potential damage. While FireNet-CNN has demonstrated strong performance in forest fire detection, several areas warrant further exploration to enhance its robustness and generalization. Although the model has been tested on specific datasets, its performance in diverse environmental conditions—such as varying forest types, seasonal changes, smoke densities, and lighting variations requires further evaluation. Future studies could focus on training with more diverse datasets to improve the model's ability to distinguish fires from similar phenomena like fog, clouds, or sunlight reflections. One promising avenue is the use of advanced augmentation techniques, including image-to-image translation methods, to generate more realistic synthetic data. This could help simulate a wider range of fire scenarios, further enhancing the model's generalization. Techniques such as domain adaptation and transfer learning could also play a crucial role in adapting FireNet-CNN to new environments without requiring extensive retraining. Another important focus is optimizing the model for deployment on even more resource-constrained devices. Although FireNet-CNN is already lightweight, further optimization through model pruning, quantization, or knowledge distillation could make it suitable for mobile phones, IoT devices, and other edge computing platforms. This would expand its applicability to low-power, remote sensing systems in wildfire-prone areas. Lastly, extensive real-world testing is essential to validate the model's performance under dynamic conditions, including obstructed views, varying weather, and complex terrains. Addressing dataset class imbalance through advanced techniques like oversampling, undersampling, or custom loss functions will further improve its sensitivity and robustness.

Future work should focus on enhancing FireNet-CNN's generalization and robustness, optimizing it for deployment in resource-constrained environments, and expanding its capabilities to cover multiple hazards. These efforts will further establish FireNet-CNN as a reliable and versatile tool for wildfire detection and disaster management. FireNet-CNN sets a new standard for wildfire detection with its high performance, interpretability, and potential for real-time applications. Further research should focus on resolving issues like class imbalance, real-world validation, and computational efficiency, making FireNet-CNN a valuable tool in wildfire management and disaster response.

DATA AVAILABILITY

The dataset, including the generated images used in this study, has been published on Mendeley Data "A Diffusion Model-Generated Forest Fire Dataset" can be accessed at: https://data.mendeley.com/datasets/nwzcm9ckrt/1. Additionally, the source code for FireNet-CNN is publicly available and can be accessed at: https://github.com/s-a-tanim/Generative-AI-fire.

REFERENCES

[1] F. Carta, C. Zidda, M. Putzu, D. Loru, M. Anedda, and D. Giusto, "Advancements in forest fire prevention: A comprehensive survey," *Sensors*, vol. 23, no. 14, p. 6635, Jul. 2023.

[2] B. Arteaga, M. Diaz, and M. Jojoa, "Deep learning applied to forest fire detection," in *Proc. IEEE Int. Symp. Signal Process. Inf. Technol. (ISSPIT)*, Dec. 2020, pp. 1–6.

[3] E. Y. Park, M. K. Song, and M. Y. An, "Experiences of forest healing instructors who met cancer patients in forest healing programs: FGI research," *Int. J. Environ. Res. Public Health*, vol. 20, no. 5, p. 4468, Mar. 2023.

[4] S. S. Sayedi et al., "Assessing changes in global fire regimes," *Fire Ecol.*, vol. 20, no. 1, pp. 1–22, Feb. 2024.

[5] L. Zhang, C. Shi, and F. Zhang, "Predicting forest fire area growth rate using an ensemble algorithm," *Forests*, vol. 15, no. 9, p. 1493, Aug. 2024.

[6] K. Ahmad, "FireXnet: An explainable AI-based tailored deep learning model for wildfire detection on resource-constrained devices," *Fire Ecol.*, vol. 19, no. 1, p. 54, 2024.

[7] U. Ejaz, M. A. Khan, H. Park, and H.-C. Kim, "FireXplainer: An interpretable approach for detection of wildfires," *Proc. Korean Inst. Inf. Sci. Eng. (KIISE)*, vol. 11, pp. 1109–1111, May 2023.

[8] I. M. Hertog, S. Brogaard, and T. Krause, "Barriers to expanding continuous cover forestry in Sweden for delivering multiple ecosystem services," *Ecosyst. Services*, vol. 53, Feb. 2022, Art. no. 101392.

[9] F. Cozim-Melges, R. Ripoll-Bosch, G. F. Veen, P. Oggiano, F. J. J. A. Bianchi, W. H. van der Putten, and H. H. E. van Zanten, "Farming practices to enhance biodiversity across biomes: A systematic review," *NPJ Biodiversity*, vol. 3, no. 1, pp. 1–11, Jan. 2024.

[10] M. A. Bansal, D. R. Sharma, and D. M. Kathuria, "A systematic review on data scarcity problem in deep learning: Solution and applications," *ACM Comput. Surv.*, vol. 54, no. 10s, pp. 1–29, Jan. 2022.

[11] P. Freer-Smith, J. H. Bailey-Bale, C. L. Donnison, and G. Taylor, "The good, the bad, and the future: Systematic review identifies best use of biomass to meet air quality and climate policies in California," *GCB Bioenergy*, vol. 15, no. 11, pp. 1312–1328, Nov. 2023.

[12] Food Agricult. Org. United Nations. (2004). *Fire Management: Voluntary Guidelines. Principles and Strategic Actions*. Accessed: Oct. 11, 2024. [Online]. Available: https://www.fao.org/4/j9255e/j9255e00.htm

[13] R.-G. Pache, I. V. Abrudan, and M.-D. Niţă, "Economic valuation of carbon storage and sequestration in Retezat national park, Romania," *Forests*, vol. 12, no. 1, p. 43, Dec. 2020.

[14] E. Z. Baskent, J. G. Borges, H. Vacik, K. M. Reynolds, and L. C. E. Rodriguez, "Management of multiple ecosystem services under climate change, bioeconomy and participation," *Forests*, vol. 12, no. 1, p. 104, Jan. 2021.

[15] J. Pontius, P. Schaberg, and R. Hanavan, "Remote sensing for early, detailed, and accurate detection of forest disturbance and decline for protection of biodiversity," *Remote Sens. Plant Biodiversity*, pp. 121–154, 2020.

[16] Y. Zheng, G. Zhang, S. Tan, and L. Feng, "Research on progress of forest fire monitoring with satellite remote sensing," *Agricult. Rural Stud.*, vol. 1, no. 2, pp. 1–8, Sep. 2023.

[17] T. J. Hoecker, S. A. Parks, M. Krosby, and S. Z. Dobrowski, "Widespread exposure to altered fire regimes under 2 °C warming is projected to transform conifer forests of the western United States," *Commun. Earth Environ.*, vol. 4, no. 1, p. 295, Aug. 2023.

[18] *Global CO_2 Emissions From Forest Fires Increase by 60 Per Cent—Preventionweb.Net*. Accessed: Nov. 16, 2024. [Online]. Available: https://www.preventionweb.net/news/global-co2-emissions-forest-fires-increase-60-cent

[19] Issarice. (2023). *Timeline of Our World in Data*. Accessed: Nov. 11, 2024. [Online]. Available: https://timelines.issarice.com/wiki/Timeline_of_Our_World_in_Data

[20] A. Rehman, D. Kim, and A. Paul, "Convolutional neural network model for fire detection in real-time environment," *Comput., Mater. Continua*, vol. 77, no. 2, pp. 2289–2307, 2023.

[21] A. A. Alsheikhy, "A fire detection algorithm using convolutional neural network," *J. King Abdulaziz Univ. Eng. Sci.*, vol. 32, no. 2, pp. 39–55, Dec. 2022.

[22] I. El-Madafri, M. Peña, and N. Olmedo-Torre, "Real-time forest fire detection with lightweight CNN using hierarchical multi-task knowledge distillation," *Fire*, vol. 7, no. 11, p. 392, Oct. 2024.

[23] R. S. Priya and K. Vani, "Deep learning based forest fire classification and detection in satellite images," in *Proc. 11th Int. Conf. Adv. Comput. (ICoAC)*, Dec. 2019, pp. 61–65.

[24] H. Harkat, J. M. P. Nascimento, A. Bernardino, and H. F. T. Ahmed, "Fire images classification based on a handcraft approach," *Expert Syst. Appl.*, vol. 212, Feb. 2023, Art. no. 118594.

[25] X. Yang, Z. Hua, L. Zhang, X. Fan, F. Zhang, Q. Ye, and L. Fu, "Preferred vector machine for forest fire detection," *Pattern Recognit.*, vol. 143, Nov. 2023, Art. no. 109722.

[26] S. P. H. Boroujeni, A. Razi, S. Khoshdel, F. Afghah, J. L. Coen, L. O'Neill, P. Fule, A. Watts, N.-M.-T. Kokolakis, and K. G. Vamvoudakis, "A comprehensive survey of research towards AI-enabled unmanned aerial systems in pre-, active-, and post-wildfire management," *Inf. Fusion*, vol. 108, Aug. 2024, Art. no. 102369.

[27] V. Khryashchev and R. Larionov, "Wildfire segmentation on satellite images using deep learning," in *Proc. Moscow Workshop Electron. Netw. Technol. (MWENT)*, Mar. 2020, pp. 1–5.

[28] A. Mumuni, F. Mumuni, and N. K. Gerrar, "A survey of synthetic data augmentation methods in computer vision," 2024, *arXiv:2403.10075*.

[29] F.-A. Croitoru, V. Hondru, R. T. Ionescu, and M. Shah, "Diffusion models in vision: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 9, pp. 10850–10869, Sep. 2023.

[30] R. Ghali, M. A. Akhloufi, and W. S. Mseddi, "Deep learning and transformer approaches for UAV-based wildfire detection and segmentation," *Sensors*, vol. 22, no. 5, p. 1977, Mar. 2022.

[31] K. Lewis, A. Thillaivanan, G. R. K. Rao, S. Rajalakshmi, K. Singh, and R. Rastogi, "Automated forest fire detection using atom search optimizer with deep transfer learning model," in *Proc. 2nd Int. Conf. Appl. Artif. Intell. Comput.*, May 2023, pp. 222–227.

[32] F. Xie and Z. Huang, "Aerial forest fire detection based on transfer learning and improved faster RCNN," in *Proc. IEEE 3rd Int. Conf. Inf. Technol., Big Data Artif. Intell. (ICIBA)*, vol. 3, May 2023, pp. 1132–1136.

[33] J. Pan, X. Ou, and L. Xu, "A collaborative region detection and grading framework for forest fire smoke using weakly supervised fine segmentation and lightweight faster-RCNN," *Forests*, vol. 12, no. 6, p. 768, Jun. 2021.

[34] N. Grieshop and C. K. Wikle, "Data-driven modeling of wildfire spread with stochastic cellular automata and latent spatio-temporal dynamics," *Spatial Statist.*, vol. 59, Mar. 2024, Art. no. 100794.

[35] A. B. Abdusalomov, B. M. S. Islam, R. Nasimov, M. Mukhiddinov, and T. K. Whangbo, "An improved forest fire detection method based on the Detectron2 model and a deep learning approach," *Sensors*, vol. 23, no. 3, p. 1512, Jan. 2023.

[36] G. M. Idroes, A. Maulana, R. Suhendra, A. Lala, T. Karma, F. Kusumo, Y. T. Hewindati, and T. R. Noviandy, "TeutongNet: A fine-tuned deep learning model for improved forest fire detection," *Leuser J. Environ. Stud.*, vol. 1, no. 1, pp. 1–8, Jun. 2023.

[37] A. S. Almasoud, "Intelligent deep learning enabled wild forest fire detection system," *Comput. Syst. Sci. Eng.*, vol. 44, no. 2, pp. 1485–1498, 2023.

[38] Y. Ban, P. Zhang, A. Nascetti, A. R. Bevington, and M. A. Wulder, "Near real-time wildfire progression monitoring with Sentinel-1 SAR time series and deep learning," *Sci. Rep.*, vol. 10, no. 1, p. 1322, Jan. 2020.

[39] Y. Jiang, R. Wei, J. Chen, and G. Wang, "Deep learning of Qinling forest fire anomaly detection based on genetic algorithm optimization," *UPB Sci. Bull., C, Electr. Eng. Comp. Sci.*, vol. 83, no. 4, pp. 75–84, 2021.

[40] S. Khan and A. Khan, "FFireNet: Deep learning based forest fire classification and detection in smart cities," *Symmetry*, vol. 14, no. 10, p. 2155, Oct. 2022.

[41] H. Liu, F. Zhang, Y. Xu, J. Wang, H. Lu, W. Wei, and J. Zhu, "TFNet: Transformer-based multi-scale feature fusion forest fire image detection network," *Fire*, vol. 8, no. 2, p. 59, Jan. 2025.

[42] Z. A. Khan, F. U. M. Ullah, H. Yar, W. Ullah, N. Khan, M. J. Kim, and S. W. Baik, "Optimized cross-module attention network and medium-scale dataset for effective fire detection," *Pattern Recognit.*, vol. 161, May 2025, Art. no. 111273.

[43] M. Altaf, M. Yasir, N. Dilshad, and W. Kim, "An optimized deep-learning-based network with an attention module for efficient fire detection," *Fire*, vol. 8, no. 1, p. 15, Jan. 2025.

[44] I. El-Madafri, M. Peña, and N. Olmedo-Torre, "Dual-dataset deep learning for improved forest fire detection: A novel hierarchical domain-adaptive learning approach," *Mathematics*, vol. 12, no. 4, p. 534, Feb. 2024.

[45] A. M. Mashraqi, Y. Asiri, A. D. Algarni, and H. Abu-Zinadah, "Drone imagery forest fire detection and classification using modified deep learning model," *Thermal Sci.*, vol. 26, no. 1, pp. 411–423, 2022.

[46] L. Jin, Y. Yu, J. Zhou, D. Bai, H. Lin, and H. Zhou, "SWVR: A lightweight deep learning algorithm for forest fire detection and recognition," *Forests*, vol. 15, no. 1, p. 204, Jan. 2024.

[47] X. Geng, X. Han, X. Cao, Y. Su, and D. Shu, "YOLOV9-CBM: An improved fire detection algorithm based on YOLOV9," *IEEE Access*, vol. 13, pp. 19612–19623, 2025.

[48] M. Yandouzi, M. Grari, M. Berrahal, I. Idrissi, O. Moussaoui, M. Azizi, K. Ghoumid, and A. K. Elmiad, "Investigation of combining deep learning object recognition with drones for forest fire detection and monitoring," *Int. J. Adv. Comput. Sci. Appl.*, vol. 14, no. 3, pp. 377–384, 2023.

[49] I. Shamta and B. E. Demir, "Development of a deep learning-based surveillance system for forest fire detection and monitoring using UAV," *PLoS ONE*, vol. 19, no. 3, Mar. 2024, Art. no. e0299058.

[50] X. Guo, Y. Cao, and T. Hu, "An efficient and lightweight detection model for forest smoke recognition," *Forests*, vol. 15, no. 1, p. 210, Jan. 2024.

[51] M. F. S. Titu, M. A. Pavel, G. K. O. Michael, H. Babar, U. Aman, and R. Khan, "Real-time fire detection: Integrating lightweight deep learning models on drones with edge computing," *Drones*, vol. 8, no. 9, p. 483, Sep. 2024.

[52] X. Li, H. Xiong, X. Li, X. Wu, X. Zhang, J. Liu, J. Bian, and D. Dou, "Interpretable deep learning: Interpretation, interpretability, trustworthiness, and beyond," *Knowl. Inf. Syst.*, vol. 64, no. 12, pp. 3197–3234, Dec. 2022.

[53] G. I. Allen, L. Gan, and L. Zheng, "Interpretable machine learning for discovery: Statistical challenges and opportunities," *Annu. Rev. Statist. Appl.*, vol. 11, no. 1, pp. 97–121, Apr. 2024.

[54] N. A. Wani, R. Kumar, and J. Bedi, "DeepXplainer: An interpretable deep learning based approach for lung cancer detection using explainable artificial intelligence," *Comput. Methods Programs Biomed.*, vol. 243, Jan. 2024, Art. no. 107879.

[55] B. Aldughayfiq, F. Ashfaq, N. Z. Jhanjhi, and M. Humayun, "Explainable AI for retinoblastoma diagnosis: Interpreting deep learning models with LIME and SHAP," *Diagnostics*, vol. 13, no. 11, p. 1932, Jun. 2023.

[56] F. Alshehri and A. Rahman, "Coupling machine and deep learning with explainable artificial intelligence for improving prediction of groundwater quality and decision-making in arid region, Saudi Arabia," *Water*, vol. 15, no. 12, p. 2298, Jun. 2023.

[57] M. C. Iban and O. Aksu, "SHAP-driven explainable artificial intelligence framework for wildfire susceptibility mapping using MODIS active fire pixels: An in-depth interpretation of contributing factors in Izmir, Türkiye," *Remote Sens.*, vol. 16, no. 15, p. 2842, Aug. 2024.

[58] A. Shmuel, T. Lazebnik, O. Glickman, E. Heifetz, and C. Price, "Global lightning-ignited wildfires prediction and climate change projections based on explainable machine learning models," 2024, *arXiv:2409.10046*.

[59] H. T. Hang, J. Mallick, S. Alqadhi, A. A. Bindajam, and H. G. Abdo, "Exploring forest fire susceptibility and management strategies in Western Himalaya: Integrating ensemble machine learning and explainable AI for accurate prediction and comprehensive analysis," *Environ. Technol. Innov.*, vol. 35, Aug. 2024, Art. no. 103655.

[60] M. Marjani, M. Mahdianpari, S. A. Ahmadi, E. Hemmati, F. Mohammadimanesh, and M. S. Mesgari, "Application of explainable artificial intelligence in predicting wildfire spread: An ASPP-enabled CNN approach," *IEEE Geosci. Remote Sens. Lett.*, vol. 21, pp. 1–5, 2024.

[61] L. Nieradzik, H. Stephani, J. Sieburg-Rockel, S. Helmling, A. Olbrich, and J. Keuper, "Challenging the black box: A comprehensive evaluation of attribution maps of CNN applications in agriculture and forestry," 2024, *arXiv:2402.11670*.

[62] S. T. Seydi, V. Saeidi, B. Kalantar, N. Ueda, and A. A. Halin, "Fire-Net: A deep learning framework for active forest fire detection," *J. Sensors*, vol. 2022, Feb. 2022, Art. no. 8044390.

[63] I. El-Madafri, M. Peña, and N. Olmedo-Torre, "The wildfire dataset: Enhancing deep learning-based forest fire detection with a diverse evolving open-source dataset focused on data representativeness and a novel multi-task learning approach," *Forests*, vol. 14, no. 9, p. 1697, Aug. 2023.

[64] A. Bauer, S. Trapp, M. Stenger, R. Leppich, S. Kounev, M. Leznik, K. Chard, and I. Foster, "Comprehensive exploration of synthetic data generation: A survey," 2024, *arXiv:2401.02524*.

[65] I. H. Rather and S. Kumar, "Generative adversarial network based synthetic data training model for lightweight convolutional neural networks," *Multimedia Tools Appl.*, vol. 83, no. 2, pp. 6249–6271, Jan. 2024.

[66] L. Alzubaidi, J. Bai, A. Al-Sabaawi, J. Santamaría, A. S. Albahri, B. S. N. Al-Dabbagh, M. A. Fadhel, M. Manoufali, J. Zhang, A. H. Al-Timemy, Y. Duan, A. Abdullah, L. Farhan, Y. Lu, A. Gupta, F. Albu, A. Abbosh, and Y. Gu, "A survey on deep learning tools dealing with data scarcity: Definitions, challenges, solutions, tips, and applications," *J. Big Data*, vol. 10, no. 1, p. 46, Apr. 2023.

[67] A. Hakami, "Strategies for overcoming data scarcity, imbalance, and feature selection challenges in machine learning models for predictive maintenance," *Sci. Rep.*, vol. 14, no. 1, p. 9645, Apr. 2024.

[68] T. Kumar, R. Brennan, A. Mileo, and M. Bendechache, "Image data augmentation approaches: A comprehensive survey and future directions," *IEEE Access*, vol. 12, pp. 187536–187571, 2024.

[69] A. K. Upadhyay and A. K. Bhandari, "Advances in deep learning models for resolving medical image segmentation data scarcity problem: A topical review," *Arch. Comput. Methods Eng.*, vol. 31, no. 3, pp. 1701–1719, Apr. 2024.

[70] I. H. Sarker, "Machine learning: Algorithms, real-world applications and research directions," *Social Netw. Comput. Sci.*, vol. 2, no. 3, p. 160, May 2021.

[71] N. Rane, S. P. Choudhary, and J. Rane, "Ensemble deep learning and machine learning: Applications, opportunities, challenges, and future directions," *Stud. Med. Health Sci.*, vol. 1, no. 2, pp. 18–41, Jul. 2024.

[72] N. Yıldıran, "Real-time verification of solar-powered forest fire detection system using ensemble learning," *Expert Syst. Appl.*, vol. 255, Dec. 2024, Art. no. 124791.

[73] D. W. Wajgi and J. V. Tembhurne, "Localization in wireless sensor networks and wireless multimedia sensor networks using clustering techniques," *Multimedia Tools Appl.*, vol. 83, no. 3, pp. 6829–6879, 2024.

[74] B. Kizilkaya, E. Ever, H. Y. Yatbaz, and A. Yazici, "An effective forest fire detection framework using heterogeneous wireless multimedia sensor networks," *ACM Trans. Multimedia Comput., Commun., Appl.*, vol. 18, no. 2, pp. 1–21, May 2022.

[75] O. Perez. (2023). *Text to Image With Stable Diffusion*. Accessed: Oct. 11, 2024. [Online]. Available: https://medium.com/latinxinai/text-to-image-with-stable-diffusion-4df16da2cfd5

[76] GeeksforGeeks. (2023). *U-Net Architecture Explained*. Accessed: Oct. 11, 2024. [Online]. Available: https://www.geeksforgeeks.org/u-net-architecture-explained/

[77] L. Alzubaidi, J. Zhang, A. J. Humaidi, A. Al-Dujaili, Y. Duan, O. Al-Shamma, J. Santamaría, M. A. Fadhel, M. Al-Amidie, and L. Farhan, "Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions," *J. Big Data*, vol. 8, no. 1, pp. 1–74, Mar. 2021.

[78] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2818–2826.

[79] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.

[80] V. Hassija, V. Chamola, A. Mahapatra, A. Singal, D. Goel, K. Huang, S. Scardapane, I. Spinelli, M. Mahmud, and A. Hussain, "Interpreting black-box models: A review on explainable artificial intelligence," *Cognit. Comput.*, vol. 16, no. 1, pp. 45–74, Jan. 2024.



GAZI MOHAMMAD IMDADUL ALAM received the bachelor's degree in computer science and engineering from East Delta University (EDU), Chittagong, Bangladesh. He is currently an AI Researcher specializing in machine learning, deep learning, data science, computer vision, pattern recognition, and explainable AI. His research interest includes medical and environmental data analysis.



NAIMA TASNIA received the B.Sc. degree in computer science and engineering from East Delta University (EDU), Chittagong, Bangladesh, where she is currently pursuing the M.Sc. degree. She is a Researcher in the domains of data analysis, machine learning, deep learning, computer vision, and wireless communications. Her research interests include the intersection of artificial intelligence with wireless communications and network systems.



TAPU BISWAS received the B.Sc. degree in computer science and engineering with a major in Information Systems from American International University-Bangladesh (AIUB). She is pursuing the master's degree with the University of Essex in January 2025. Her research interests include software engineering, machine learning, and data science.



MD. JAKIR HOSSEN (Senior Member, IEEE) received the master's degree in communication and network engineering and the Ph.D. degree in smart technology and robotic engineering from the Universiti Putra Malaysia, Malaysia, in 2003 and 2012, respectively. He is currently an Associate Professor with the Department of Robotics and Automation, Faculty of Engineering and Technology, Multimedia University, Malaysia. His research interests include the application of artificial intelligence techniques in data analytics, robotics control, data classifications, and predictions.



SHARIA ARFIN TANIM received the B.Sc. degree in computer science and engineering from American International University-Bangladesh (AIUB). He is a prospective M.Sc. degree student. He is currently a Research Assistant (RA) with the Advanced Machine Intelligence Research Laboratory (AMIR). Previously, he was a student Research Assistant with the Ubiquitous, Cloud, and Human-Computer Interaction (UCH) Research Group, AIUB. He is also the Founder of the Learnify Research Laboratory, which works with students. His research interests include bioinformatics, decentralized learning methods, computer vision, reinforcement learning, medical data analysis, and transparency of AI.

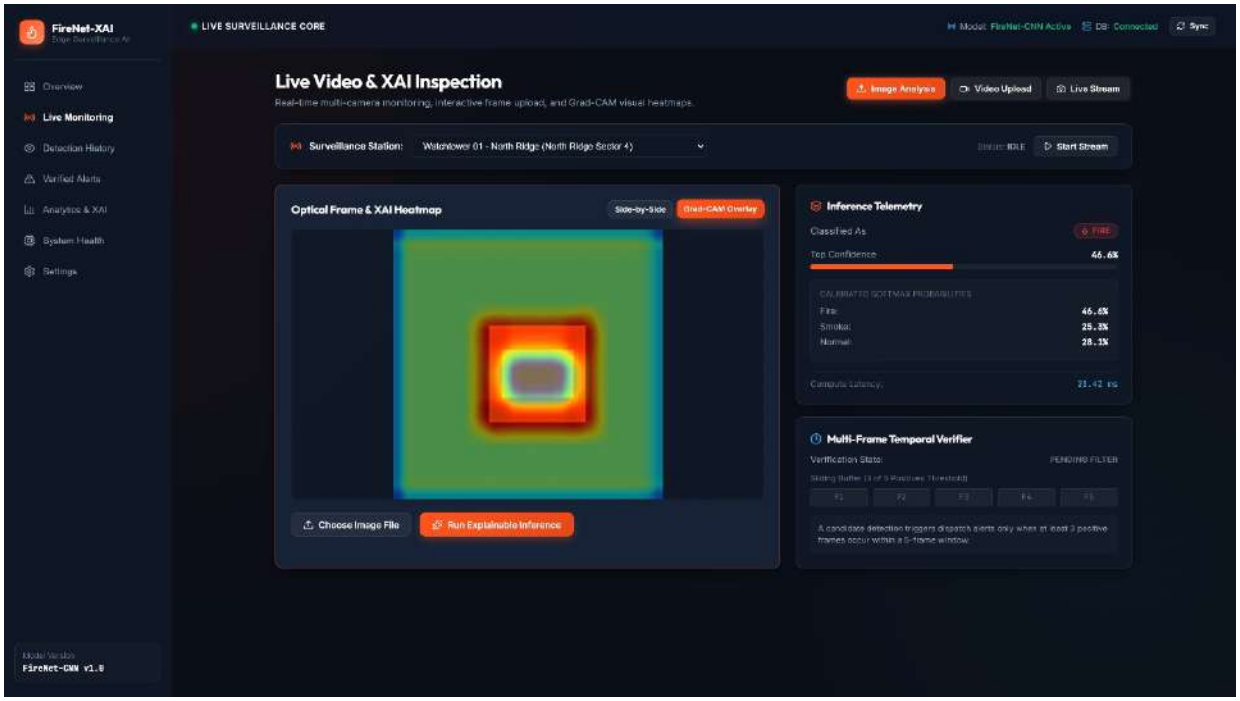


MD SAEF ULLAH MIAH received the bachelor's and master's degrees from American International University–Bangladesh (AIUB), and the Ph.D. degree from Universiti Malaysia Pahang Al-Sultan Abdullah. He is an Associate Professor with the Department of Computer Science, AIUB. He is passionate about leveraging these technologies to solve real-world challenges and has hands-on experience in software development, project management, and decentralized applications. His research interests include machine learning, natural language processing, data and text mining, knowledge management, material informatics, and blockchain.

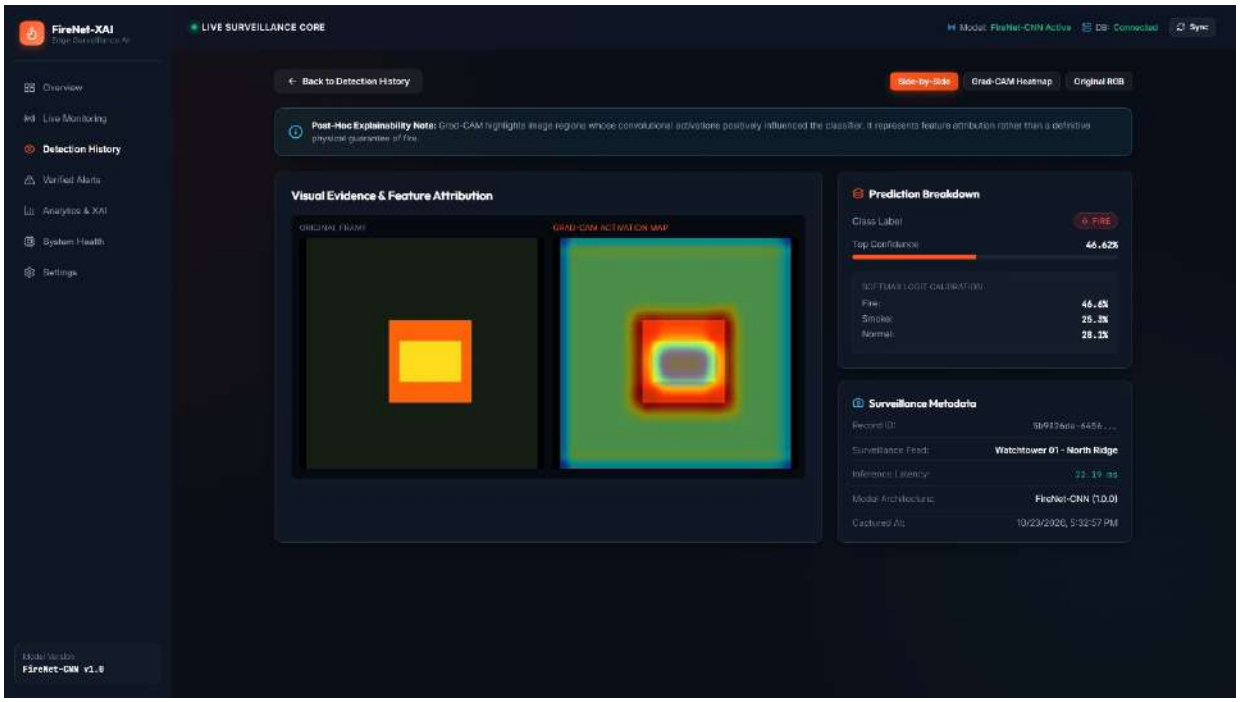
APPENDIX-II

SCREENSHOTS

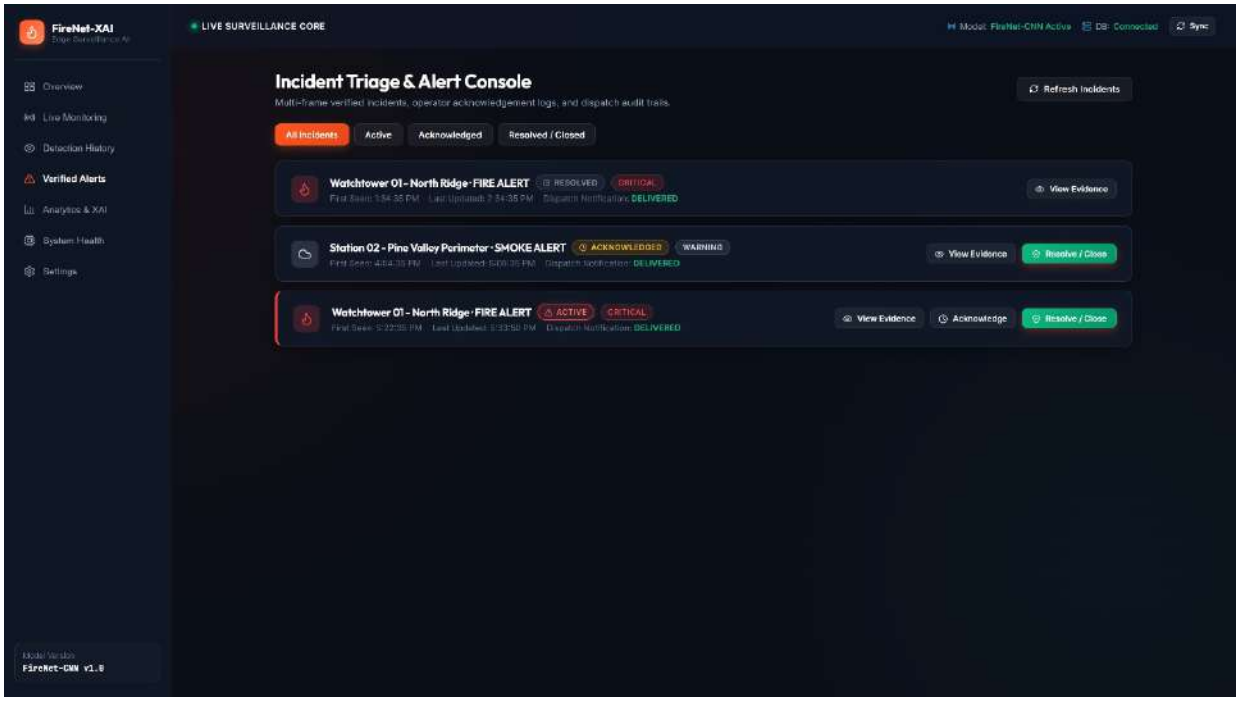
Screenshot 1 - Live Monitoring: feed selector, Grad-CAM overlay and the 3-of-5 sliding window



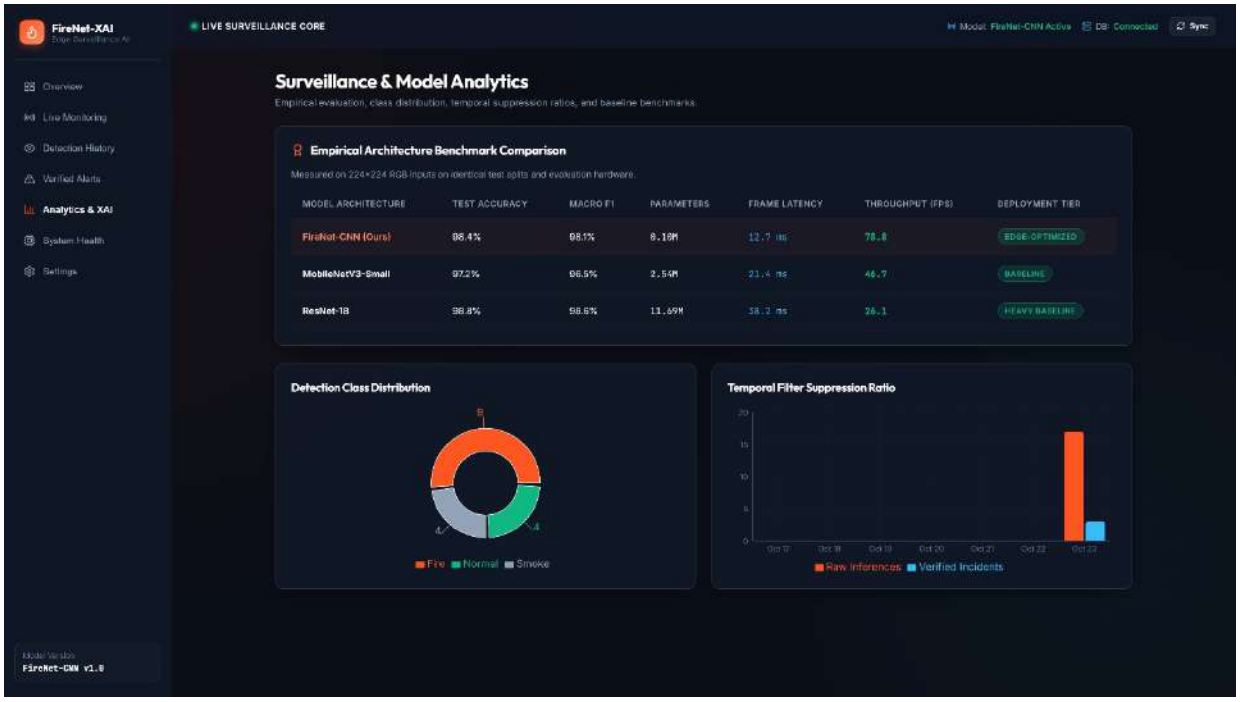
Screenshot 2 - Detection Detail and XAI Inspector: heatmap toggle and probability breakdown



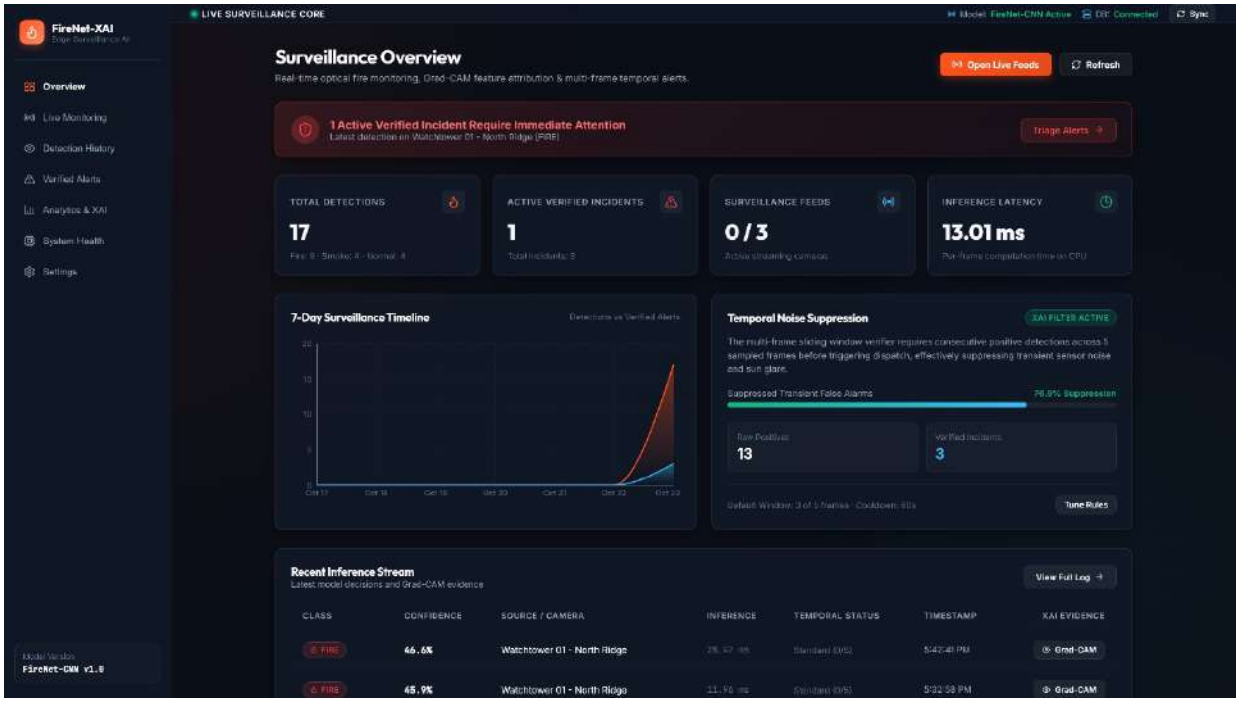
Screenshot 3 - Incident Triage Console: severity badges and acknowledge / close actions



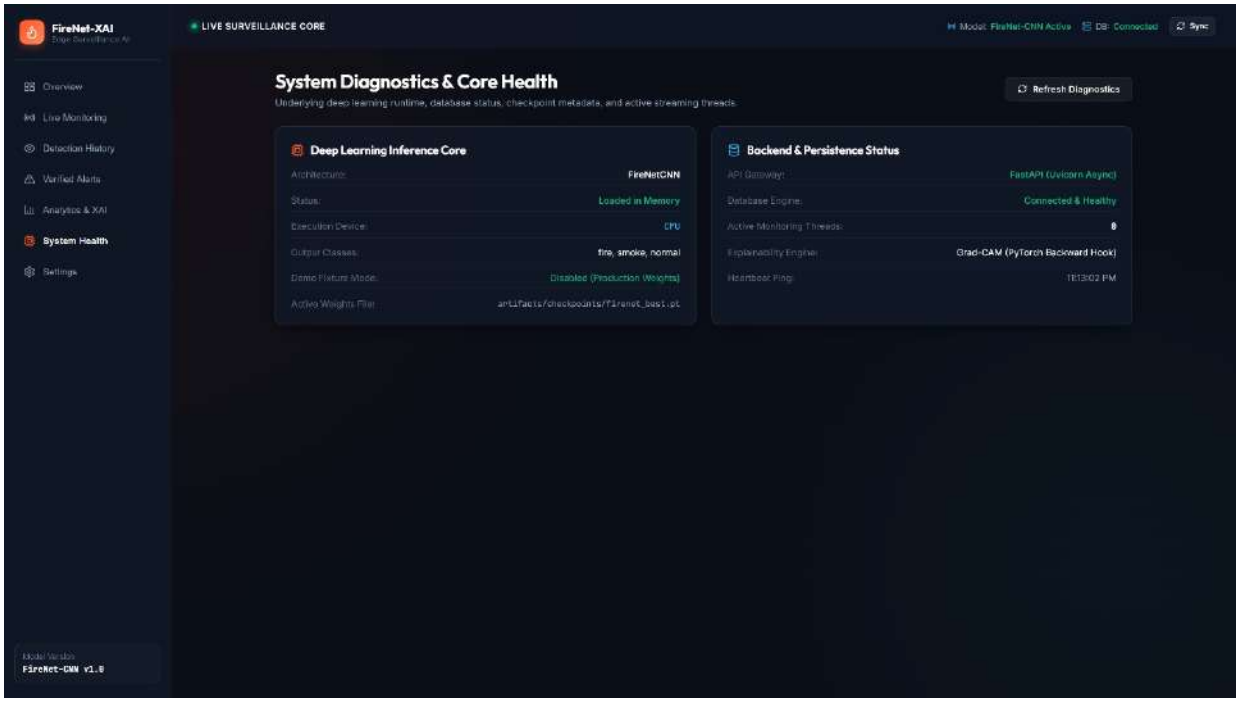
Screenshot 4 - Analytics: benchmark matrix, class distribution and suppression chart



Screenshot 5 - Overview Dashboard: KPI cards, warning banner and seven-day trend



Screenshot 6 - System Diagnostics: API, database, torch device and worker threads



APPENDIX-III

INTERNSHIP CERTIFICATE

INTERNSHIP CERTIFICATE



Cognifyz
Where Data Meets Intelligence

Cognifyz IT Solutions

Internship Completion Certificate

Date -09/07/2026

This is to certify that **V.Nithyakalyanan (Intern ID: CTI/A1/C371304)** currently pursuing B.E. from The Anand Institute of Higher Technology, was working as a **Web Development Intern** with Cognifyz IT Solutions Pvt. Ltd. from 08/06/2026 to 08/07/2026.

During this period, he has served as a Web Development Intern and has displayed remarkable dedication, sincerity, and a strong desire to learn. He has exhibited exceptional coordination skills and effective communication abilities. Moreover, his attention to detail has been truly impressive.

He has consistently approached new assignments and challenges with enthusiasm, showcasing His passion for Web Development. His commitment and willingness to acquire new knowledge and skills have been evident throughout His internship.

We extend our best wishes to V.Nithyakalyanan for a successful future, and we have no doubt that he will continue to excel in the field of Web Development.

With Regards,
Cognifyz IT Solutions Pvt. Ltd.

Cognifyz IT Solutions Pvt. Ltd. Warora, Chandrapur, India

CERTIFIED ISO 9001:2015 CERTIFIED

MSME

support@cognifyz.com www.cognifyz.com

REFERENCES

[1] Mahmoudi, S. A., Gloesener, M., Benkedadra, M. and Lerat, J.-S., “EdgeAI system for real-time and explainable forest fire detection using compressed deep learning models,” in Proc. 20th Int. Joint Conf. on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISAPP), SCITEPRESS, 2025.

[2] He, L., Zhou, Y., Liu, L., Zhang, Y. and Ma, J., “Research and application of deep learning object detection methods for forest fire smoke recognition,” Scientific Reports, vol. 15, art. 16328, 2025, doi:10.1038/s41598-025-98086-w.

[3] Mahadev, N., Bhat, A., Sowmya, V. L., Premkumar, A., Shankar, R. and Natarajan, R., “Intelligent real-time wildfire detection using image processing techniques with edge-AI integration,” Discover Computing, vol. 34, art. 1, 2026.

[4] Jadon, A., Omama, M., Varshney, A., Sharma, R. and Ansari, M. S., “FireNet: A specialized lightweight fire and smoke detection model for real-time IoT applications,” arXiv:1905.11922, 2019.

[5] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D., “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in Proc. IEEE Int. Conf. on Computer Vision (ICCV), pp. 618–626, 2017.

[6] D'Angelo, P., Murdaca, G., Gordano, A., Pellegrino, D. and Larosa, G., “Deep learning approaches for fire detection and localization: A review,” Fire Technology, vol. 60, pp. 1913–1954, 2024.

[7] Song, W., Zhang, J., Zhang, X. and Shi, L., “Object detectors continue to improve on fire and smoke detection: A benchmark study,” Fire Safety Journal, vol. 141, art. 103925, 2023.

[8] Li, F., Dutta, J., Xiong, Z. and Xie, J., “A survey on explainable artificial intelligence for computer vision: Methods, applications, and evaluation,” IEEE Access, vol. 12, pp. 98102–98131, 2024.

[9] Liu, Y., Wang, H., Li, X. and Zhang, C., “Real-time wildfire smoke detection using convolutional neural networks with temporal consistency,” IEEE Geoscience and Remote Sensing Letters, vol. 20, art. 4102605, 2023.

[10] Zhu, M., Wang, L. and Xu, P., “Frequency and power-aware scheduling for solar-powered edge computing in remote environmental monitoring,” ACM Transactions on Sensor Networks, vol. 18, no. 3, art. 41, 2022.